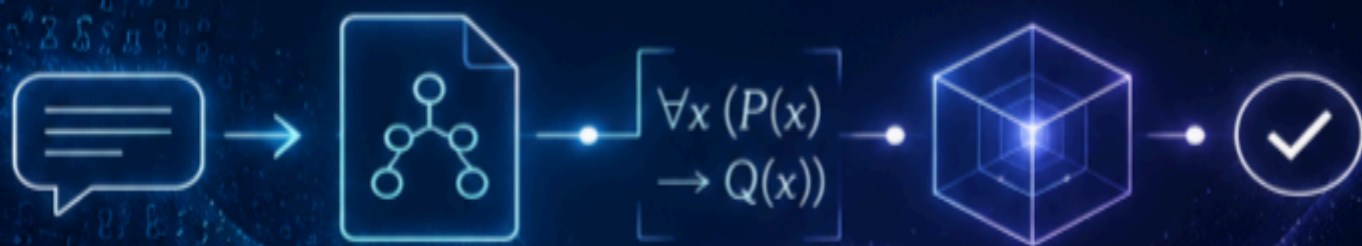


WYKONYWALNY JĘZYK NATURALNY



Krytyczny przegląd semantyki formalnej, interpretacji
abstrakcyjnej, wykonania symbolicznego i architektur
LLM ugruntowanych symbolicznie



WYKONYWALNY JĘZYK NATURALNY

Krytyczny przegląd semantyki formalnej, interpretacji
abstrakcyjnej, wykonania symbolicznego i architektur LLM
ugruntowanych symbolicznie

*Ku neuro-symbolicznemu środowisku wykonawczemu dla długich dokumentów,
reguł i systemów agentowych*



Prawa autorskie © [2026] Axiologic Research

Wszelkie prawa zastrzeżone.

KDP prawa wydawnicze posiadane przez Outfinity SRL (Iași, Rumunia).

Dostępne za darmo na stronie Axiologic (www.axiologic.net).

Nota autora

Praca ta została stworzona pod parasolem Axiologic Research w ramach dwóch połączonych ze sobą programów badawczych, jednego poświęconego Sztucznej Inteligencji, a drugiego Outfinitism, obu prowadzonych przez Șinică Alboai. Aby poznać szczegóły, w jaki sposób te dwa programy krzyżują się i opierają się na sobie, zapraszamy czytelników do odwiedzenia strony Outfinity Venture Studio przy www.outfinity.ch. Po

dczas gdy modele językowe sztucznej inteligencji były używane do wspomagania generowania tekstu, podstawowa filozofia, struktura tematyczna i podejście analityczne wywodzą się wyłącznie z dziesięciolecia dedykowanych przez autora badań zjawisk społecznych, technologii społecznych i autentycznych twardych technologii wynikających z różnych europejskich i samofinansowanych projektów badawczych. Ostatnio, w ramach projektu badawczego Achilles, głęboko badaliśmy AI literaturę generowaną i jej zautomatyzowaną weryfikację przy użyciu podejść neuro-symbolicznych; wszystkie bezpłatne książki udostępniane publicznie są częścią pakietu prac, których używamy do testowania naszych najnowszych technologii.

Spis treści

Streszczenie.....	5
Przedmowa: Zakres, metoda i pozycja krytyczna.....	8
1. Od płynnego języka do wykonywalnej semantyki.....	11
1.1 Czego wymaga wykonanie i gdzie należy umieścić granicę zaufania.....	11
1.2 Luka formalizacji: dlaczego poprawny solver nadal może wygenerować błędną odpowiedź.....	15
2. Intelktualny rodowód języka wykonywalnego.....	18
2.1 Języki kontrolowane, semantyka formalna, dyskurs i logika naturalna.....	18
2.2 Parsowanie semantyczne, zapytania wykonywalne, indukcja programów i wczesne systemy neuro-symboliczne.....	22
3. Interpretacja abstrakcyjna i wykonanie symboliczne dla języka.....	25
3.1 Interpretacja abstrakcyjna jako semantyka ograniczonej niepewności.....	25
3.2 Wykonanie symboliczne, rozwiązywanie ograniczeń, sprawdzanie modeli i doprecyzowanie semantyczne.....	29
4. Co naprawdę osiągnęła era LLM.....	33
4.1 Autoformalizacja, rozumowanie wspomagane programami, logika, specyfikacje temporalne i planowanie.....	33
4.2 Analiza programów wspomagana przez LLM, eksploracja symboliczna, transformatory abstrakcyjne i certyfikowane NLP.....	37
5. Dlaczego systemy języka wykonywalnego zawodzą.....	41
5.1 Ugruntowanie, tożsamość, pozorne formalizacje, ontologie i wiedza o świecie.....	41
5.2 Pragmatyka, niemonotoniczność, skala, bezpieczeństwo i porażka ewaluacji.....	45
6. Rodziny architektur dla wykonywalnego języka naturalnego.....	49
6.1 Konkurencyjne projekty: reprezentacje uniwersalne, portfele formalizmów, kontrolowana kompilacja i systemy query-first.....	49
6.2 Referencyjny pipeline od tekstu do symboli, wykonania i ograniczonych odpowiedzi w języku naturalnym.....	54
7. LongTextJS, CircuitJS i nllAgent: krytyczne studium przypadku.....	57
7.1 Co wdraża architektura publiczna i dlaczego jej granice mają znaczenie.....	58
7.2 Niezależna ocena: mocne strony, aktualne poważne problemy i wymagana ewolucja...62	62
8. Budowa i walidacja neuro-symbolicznego środowiska wykonawczego.....	66
8.1 Strategia implementacji: minimalny, lecz naukowo uczciwy system języka wykonywalnego.....	66
8.2 Pytania badawcze, benchmarki i program publikacji.....	73
Konkluzja: od formy probabilistycznej do konsekwencji symbolicznej.....	79
Bibliografia.....	83

Streszczenie

Duże modele językowe sprawiły, że język naturalny jest skutecznym interfejsem do obliczeń, ale nie uczyniły języka naturalnego wykonalnym w silnym sensie, w którym program, zapytanie bazy danych, system przejściowy lub teoria logiczna jest wykonalna. Model może wygenerować przekonujące wyjaśnienie, zmieniając jednocześnie zakres kwantyfikatora, pomijając wyjątek, wymyślając przesłankę lub stosując tę samą regułę inaczej w dwóch miejscach. Narzędzia zewnętrzne poprawiają niezawodność tylko wtedy, gdy model poprawnie przetłumaczył żądanie. Dowódca twierdzenia może doskonale udowodnić błędne sformalizowanie; interpreter Pythona może wykonać program, który przypadkowo odpowiada na przykład, jednocześnie błędnie przedstawiając pytanie; symboliczny wykonawca może odsłonić prawdziwą ścieżkę przez oprogramowanie, mimo że reguła języka naturalnego używana do skonstruowania warunku ścieżki została błędnie przetłumaczona. Trudnym problemem nie jest więc tylko dodanie solvera po LLM. Jest to skonstruowanie kontrolowanego łańcucha od języka źródłowego do symboli uziemionych, od uziemionych symboli po możliwe do wykonania obiekty formalne, a od symbolicznych wyników z powrotem do języka naturalnego, nie pozwalając, aby ostateczny model językowy zmienił to, co zostało ustanowione.

Ta książka bada odpowiednie tradycje jako jeden z powiązanych problemów badawczych. Wcześniejsze podstawy obejmują kontrolowane języki naturalne, semantykę model-teoretyczną i dyskursową, logikę naturalną, parsowanie semantyczne, programowanie logiczne, systemy tekstowe, syntezę programów z przykładów i języka, silniki reguł, tabele decyzyjne, logikę czasową, zautomatyzowane planowanie i systemy neuronowo-symboliczne. Fundamenty analizy programu obejmują abstrakcyjną interpretację, wykonanie symboliczne i jednokokowe, analizę przepływu danych, sprawdzanie modelu, SAT/SMT/CSP rozwiązywanie, Datalog i ocenę punktów końcowych, automaty i weryfikację dowodu. Niedawna literatura LLM dodaje wspomagane programowo rozumowanie, deklaratywne monitowanie solvera, autoformalizację, tłumaczenie logiczne pierwszego rzędu, tłumaczenie czasowo-logiczne, planowanie formalne, samonaprawę z przewodnikiem LLM, analizę programu wspomaganego przez LLM, eksplorację symboliczną z przewodnikiem LLM i syntezę zweryfikowanych transformatorów abstrakcyjnych. W tych rodzajach pracy najsilniejsze wyniki konsekwentnie pojawiają się, gdy formalizm docelowy jest wystarczająco wąski, aby mieć wyraźną semantykę, a akceptowana wydajność jest sprawdzana przez komponent niezależny od LLM.

Badanie zwraca szczególną uwagę na rozróżnienie między dwoma zastosowaniami interpretacji abstrakcyjnej. Pierwszy, już ustanowiony w NLP, weryfikuje model neuronowy nad formalnie określoną rodziną perturbacji tekstowych. Domeny abstrakcyjne interwałowe i relacyjne mogą poświadczyć, że przewidywanie klasyfikatora nie zmienia się w ramach każdej podstawienia dozwolonej przez określony zestaw kontryktoryjny. Jest to prawdziwa gwarancja formalna, ale dotyczy stabilności modelu w ramach skonstruowanej relacji perturbacji; nie zaświadcza, że arbitralne parafrazy ludzkie zachowują znaczenie, ani że pierwotna odpowiedź modelu jest prawdziwa. Drugie zastosowanie jest bardziej spekulacyjne i bardziej istotne dla możliwego do wykonania języka naturalnego: traktowanie zestawu dopuszczalnych interpretacji dokumentu jako semantyki zbierającej i konserwatywnej, niekoniecznie, i nieznanych wniosków nad tym zestawem. Wymaga to wyraźnych domen tożsamości, czasu, modalności, ilości, statusu źródła, siły normatywnej i pokrycia. Wymaga to

również uczciwej gwarancji warunkowej: analiza jest solidna tylko w stosunku do zestawu interpretacji, ontologii, polityki uziemienia i zasad transferu dostarczonych do niej.

Symboliczne wykonanie jest komplementarne. Interpretacja abstrakcyjna może tanie pokazać, że naruszenie jest niemożliwe w ramach obecnej abstrakcji lub że naruszenie pozostaje możliwe. Symboliczne wykonanie może następnie podjąć próbę skonstruowania konkretnej interpretacji i ścieżki, która ją realizuje. Jeśli taka droga nie istnieje, nieudany świadek może ujawnić, która abstrakcja lub semantyczny wybór był zbyt gruboskórny. Sugeruje to, że wzór badawczy zwany dopracowaniem semantycznym prowadzonym przez kontrprzykład: kontrprzykładowy, niezaspokojony rdzeń solvera lub nieudany dowód jest mapowany z powrotem do dokładnych fragmentów źródłowych i zobowiązań semantycznych, które go wygenerowały; LLM lub człowiek jest proszony o zrewidowanie tylko tych zobowiązań; analiza symboliczna jest odtwarzana. LLM służy do wyszukiwania przestrzeni formalizacji, podczas gdy jądro symboliczne określa, co wynika z każdego przyjętego formalizacji.

Ostatnie systemy pokazują ważne fragmenty tej wizji. PAL deleguje obliczenia proceduralne do Pythona. SatLM generuje ograniczenia deklaratywne dla silnika satysfakcjonującego. Logic-LM i LINC przekładają naturalne zadania rozumowania na logikę formalną i polegają na symbolicznych solverach lub twierdzeniach do dedukcji. NL2TL i nl2spec przekładają wymagania na specyfikacje czasowe, a nl2spec wyraźnie wspiera interaktywną korektę na poziomie podformuły. Prace związane z planowaniem formalnym wykazały duże zyski, gdy LLMs konstruowane instancje planowania, które są następnie rozwiązywane i sprawdzane za pomocą narzędzi dźwiękowych. SymGPT przekłada naturalne reguły językowe ERC na ograniczoną gramatykę i wykorzystuje symboliczne wykonanie, aby znaleźć konkretne naruszenia inteligentnych umów, zgłaszając tysiące naruszeń zasad i wyraźnych ścieżek ataku. W innym kierunku AutoBug rozkłada analizę programu na ścieżki symboliczne obsługiwane przez LLM, ale pozostaje przybliżona; SAIL wykorzystuje LLMs do wyszukiwania abstrakcyjnych transformatorów, podczas gdy twarde kontrole semantyczne i składniowe zachowują globalną solidność. Wyniki te są dowodem na to, że LLMs są skutecznymi mechanizmami składania wniosków i wyszukiwania wokół formalnych rdzeni, a nie dowodem na to, że samo pokolenie probabilistyczne stało się formalnym rozumowaniem.

Niedawny systematyczny przegląd neurosymbolicznych NLP twierdzi, że architektury federacyjne, w których moduły neuronowe i symboliczne pozostają autonomiczne i komunikują się za pośrednictwem uporządkowanych interfejsów, często wykazują większe zyski niż architektury, które tylko wstrzykują symboliczne uprzedzenia do sieci neuronowych. Jego autorzy ostrzegają również, że pozorna zaleta nie jest jeszcze rozstrzygająca, ponieważ systemy federacyjne i wstrzykiwacze są zwykle oceniane na różnych rodzajach zadań. Ta książka przyjmuje to samo zdyscyplinowane stanowisko. Struktura architektoniczna dla separacji jest silna, ponieważ wyjaśnia granicę zaufania, pozwala na niezależną powtórkę i utrzymuje symboliczny komponent do sprawdzenia; empiryczne twierdzenie, że jedna rodzina powszechnie przewyższa drugą, pozostaje niesprawdzone [Chatzikyriakidis and Lappin 2026] [Chatzikyriakidis and Lappin 2026].

Książka zawiera również krytyczne studium przypadku publicznej NaturalLanguageLinterAgent, lub nllAgent, architektury i jego LongTextJS i CircuitJS komponentów. Implementacja oddziela niezmienny program po stronie dokumentu od programu po stronie teorii wielokrotnego użytku. Zachowuje dokładne kotwice źródłowe, wersjonowane typy obserwacji, tożsamości producentów, pokrycie, luki i status epistemiczny. Kompilacja

obwodów wyprowadza obserwacje faktycznie wymagane przez regułę, sprawdza, czy istnieją zgodni producenci, sprawdza zgodność z przepisami, czy konkretny dokument przyniósł wystarczające obserwacje i zasięg, a niezależny weryfikator kontroluje, czy dane kandydujące jest opublikowane. Uczenie się odbywa się w jednorazowym obszarze roboczym i nie może publikować automatycznie. Są to niezwykle poważne decyzje dotyczące zarządzania systemem analizy dokumentów wspomaganych LLM w zakresie pomocy. Publiczne repozytorium rejestruje również nierozwiązane ograniczenia, zamiast je ukrywać. Jego obecny poważny rejestr zagadnień identyfikuje ograniczoną przyrostowość harmonogramu, niepełną propagację alternatyw interpretacji poprzez każdą ścieżkę wykonania, konserwatywną analizę zgodności dla nieprzejrzystego zachowania proceduralnego, minimalne planowanie zapytań i indeksowanie czasowe, celowo wąskie kontrolowane generowanie i wsparcie mutacji, które pozostaje specyficzne dla scenariusza. Są to wiarygodne ograniczenia badawcze: dotyczą one podłoża semantycznego i wykonawczego, a nie tylko brakującego słupka interfejsu użytkownika.

W wyszukiwaniach prowadzonych dla tego wydania nie przeprowadzono niezależnej oceny ani przeglądu technicznego nllAgent innych firm. W związku z tym ocena studium przypadku jest wyraźnie własnym przeglądem architektonicznym autora, opartym na publicznym repozytorium, dokumentacji, zakresie wykonywalnym i samozwańczych poważnych kwestiach. Wyrok jest korzystny dla rozdziału dowodów, teorii, weryfikacji i publikacji, ale ostrożny co do kompletności semantycznej. Najsilniejsze bieżące wnioski o wdrożenie dotyczą ograniczonych kontroli strukturalnych i kontrolowanych w języku angielskim. Ogólna rozdzielczość tożsamości, bogata semantyka zdarzeń, wiedza w świecie bieżącym, szeroki transfer domen i statystycznie wspierane twierdzenia o jakości pozostają pracą badawczą, a nie ustalonymi zdolnościami.

Centralną propozycją książki jest multiformalizm wykonywalno-naturalnojęzyczny czas działania. LLM najpierw proponuje mały zestaw kandydujących struktur semantycznych, z których każda jest połączona z dokładnymi rozpiętościami źródłowymi i wpisana przeciwko wybranej ontologii. Symboliczne jądro uziemienia przyznaje tylko przedmioty, których tożsamość, pochodzenie, zakres, czas, modalność i dowody zaspokajają wyraźne umowy. Router wybiera jeden lub więcej celów formalnych zgodnie z pytaniem: ocena relacyjna lub Datalog dla zapytań o zamknięcie i dokument; tabele decyzyjne dla polityk; SAT/SMT/CSP dla ograniczeń; automaty dla uporządkowanych wzorców; sprawdzanie modelu pod kątem zachowań czasowych; symboliczne wykonanie dla świadków ścieżki; oraz asystentów dowodów lub twierdzeń twierdzenia dla roszczeń dedukcyjnych. Formalne wykonanie daje wynik kanoniczny zawierający przyjęte fakty, odrzuconych kandydatów, niepewność, świadków, przedmioty dowodowe, limity pokrycia i założenia. Ostateczny LLM może wyjaśnić ten wynik, ale każde zdanie w wyjaśnieniu musi być licencjonowane przez strukturę kanoniczną i identyfikowalne do dowodów lub zweryfikowanego pochodzenia.

Wykonalny język naturalny nie powinien być zatem reklamowany jako wykonanie nieograniczonej prozy. Należy go zdefiniować jako kontrolowany proces kompilacji, uziemienia, wykonania i realizacji jednego lub więcej wyraźnych kandydatów semantycznych. Gwarancja jest warunkowa, ale przydatna: biorąc pod uwagę deklarowaną rewizję źródła, ontologię, uziemienie, zestaw interpretacji, semantykę formalną i granice wykonania, raportowany wynik następuje i może zostać powtórzony. Głównym problemem otwartych badań nie jest rozwiązanie. Jest to budowa, porównanie, dopracowanie i walidacja formalizacji, na których działa solver.

Przedmowa: Zakres, metoda i pozycja krytyczna

Ambicja wykonywalnego języka naturalnego jest łatwa do wyrażenia i łatwa do zawyżenia. Użytkownik pisze regułę, politykę, hipotezę naukową, procedurę lub pytanie w zwykłym języku; system to rozumie; obliczenia są następujące; wynik jest zwracany jako zwykły język. Współczesne modele językowe sprawiają, że początek i koniec tego rurociągu wydają się rozwiązane. Mogą czytać nieograniczoną prozę, generować kod i formuły, wywoływać narzędzia i płynnie wyjaśniać wyjścia. Niebezpieczną iluzją jest to, że środek zniknął. W rzeczywistości środek zawiera prawie każdy trudny problem: zdecydować, które znaczenie ma znaczenie, zidentyfikować obiekty w różnych przejściach, odróżnić wyraźne dowody od założeń tła, zachowując modalność i wyjątki, dobierając odpowiedni formalizm, udowadniając, że wygenerowany artefakt jest dobrze uformowany, i uniemożliwiając ostatecznemu wyjaśnieniu stwierdzenie więcej niż ustalone wykonanie.

Przydatną ankietą nie może być katalog małych systemów przedstawionych w krótkich odizolowanych podrozdziałach. Pole jest fragmentaryczne właśnie dlatego, że te same słowa są używane do różnych operacji. „Symboliczne wykonanie” może oznaczać eksplorację ścieżki w stylu Kinga nad symbolicznymi wejściami programu, wykonanie wygenerowanej formy logicznej na bazie wiedzy lub po prostu użycie zewnętrznego interpretera. „Formalna weryfikacja” może odnosić się do poprawności wyniku solvera, poprawności skompilowanego programu, wierności tłumaczenia w języku naturalnym lub solidności sieci neuronowej w ramach predefiniowanej rodziny perturbacji. „Uziemienie” może oznaczać łączenie słów z kolumnami bazy danych, podmiotami do rozpiętości dokumentów, propozycjami do wiedzy światowej lub instrukcjami robotów do obiektów percepcyjnych. Poważne leczenie musi pokazać, jak te warstwy się odnoszą i gdzie gwarancje się zatrzymują.

Metoda badania przebiega zatem zgodnie z mechanizmami, a nie z kategoriami marketingowych. Dla każdej linii pracy analiza zadaje pięć pytań. Jaki artefakt jest produkowany z języka? Jaką semantykę ma ten artefakt niezależny od LLM? Jak artefakt jest ugruntowany w źródle lub środowisku? Który składnik faktycznie określa wynik? Co jest i nie jest gwarantowane, gdy system odniesie sukces? Dokumenty są traktowane jako dowód na zgłoszone zadania, a nie jako dowód ogólnej architektury. Silne wyniki liczbowe są uwzględniane, gdy wyjaśniają mechanizm, ale nie są one przenoszone do domen o różnej strukturze semantycznej. Horyzont ankiety to 2 sierpnia 2026 r. i obejmuje recenzowaną literaturę, oficjalne postępowania, podstawowe prace badawcze oraz publiczne nllAgent repozytorium i dokumentację.

Termin „przegląd” jest używany ostrożnie. Obecnie istnieje kilka szerokich badań, ale każda obejmuje tylko część problemu end-to-end. Badanie kontrolowanego w języku naturalnym firmy Kuhn organizuje języki zgodnie z precyzją, ekspresją, naturalnością i prostotą [Kuhn 2014]. Hamilton i współpracownicy oceniają, czy neuro-symboliczny NLP faktycznie dostarczył rozumowanie, transfer, interpretację, wydajność danych i uogólnienie pozadystrybucyjne, podczas gdy nowsze badania klasyfikują zorientowane na zadania i oparte na danych i wiedzy systemy neuro-symboliczne [Hamilton et al. 2024; Delvecchio et al. 2025; Wang et al. 2025b]. Badania wykonania symbolicznego i analizy programu wspomaganego LLM opisują inżynierskie granice eksploracji ścieżek oraz rosnącą rolę wyuczonych komponentów [Baldoni et al. 2018; Wang et al. 2025a]. Dedykowane badania LLMs jako formalizatory do zautomatyzowanego planowania [Tantakoun et al. 2025], a przegląd autoformalizacji koncentruje się na przełożeniu

nieformalnej matematyki na formy sprawdzane maszynowo [Weng et al. 2025]. Dwukierunkowa mapa drogowa dla inżynierii wymagań bada zarówno zastosowanie LLMs w celu udostępnienia metod formalnych, jak i wykorzystanie formalnych metod w celu ograniczenia wymagań LLM wytwarzanych przez [Ferrari and Spoletini 2025]. Skoncentrowana przegląd obejmująca trzydzieści pięć artykułów mapuje formalizację wymagań i specyfikacji oprogramowania wspomaganą LLM w zakresie oprogramowania, w tym pracy skierowanej do Dafny, C i Java; jej zakres potwierdza, że obsługa tłumaczenia rośnie szybciej niż kompleksowa pewność [Beg et al. 2025] semantyczna. Przegląd systematyczny 2026 Frontiers rozwija nową taksonomię dla neuro-symbolicznego NLP i argumentuje za większą dbałością o systemy federacyjne [Chatzikyriakidis and Lappin 2026].

Żaden ankietowany przegląd nie ujednolici kontrolowanego języka, uziemienia semantycznego, abstrakcyjnej interpretacji, symbolicznego wykonania, selekcji formalizmu, wyników przenoszenia dowodów i ograniczonej regeneracji jako architektury zaufania. Wniosek ten jest raczej syntezą literatury niż twierdzeniem o wyczerpującym istnieniu: terminologia różni się w różnych dziedzinach, a odpowiednie prace są rozprowadzane między językoznawstwem obliczeniowym, językami programowania, bazami danych, metodami formalnymi, inżynierią oprogramowania, planowaniem i robotyką. Tam, gdzie nie istnieje żaden niezależny przegląd, zwłaszcza w przypadku nllAgent, tekst określa swoje osądy jako oryginalną analizę, a nie sugeruje zewnętrzne poparcie.

Stanowisko książki jest opiniotwórcze, ale konserwatywne. Odrzuca ideę, że duży model powinien być zaufany jako ostateczny powód tylko dlatego, że jego proza przypomina dowód. Odrzuca również odwrotną ideę, że tylko ręcznie napisany kompletny parser semantyczny może wspierać użyteczne formalne wykonanie. LLM jest często najlepszym dostępnym mechanizmem proponowania wydarzeń, relacji, schematów formalnych i mapowania kandydatów. Pytanie architektoniczne brzmi, jak ograniczyć tę moc. Najbardziej obronną odpowiedzią jest system federacyjny z wyraźnymi artefaktami pośrednimi, interfejsami typowanymi i wersjonowanymi, niezależnym wykonaniem symbolicznym, konserwatywnymi nieznanymi stanami i dowodami wielokrotnego odtwarzania.

Stanowisko to ma istotne konsekwencje. Celem systemu nie zawsze jest stworzenie jednej ostatecznej formalizacji. Czasami właściwy artefakt jest zestawem alternatyw. Zaimiek może odnosić się do dwóch możliwych podmiotów; zwrot prawny może przyznać się do konkurencyjnych zakresów; wymóg może być niekompletny; dwa źródła mogą się nie zgodzić; dokument może nie powiedzieć, czy ma zastosowanie wyjątek. Symboliczny czas działania, który arbitralnie wybiera jedną interpretację, jest mniej rygorystyczny niż LLM, który szczerze stwierdza dwuznaczność. Możliwość wykonania staje się cenna, gdy może obliczyć, co następuje pod każdym dopuszczalnym odczytem, co następuje pod co najmniej jednym, które wnioski zależą od spornego wyboru i jakie dodatkowe dowody rozstrzygną wybór.

Proponowana architektura traktuje zatem niepewność strukturalnie. Wyniki zaufania mogą klasyfikować kandydatów, ale nie przekształcają proponowanej interpretacji w poświadczony fakt. Status elementu rejestruje, w jaki sposób stał się znany: bezpośrednio wyodrębniony z deterministycznego parsera, zaproponowany przez model, założony dla scenariusza, wyprowadzony regułą, sprawdzony przez weryfikatora lub potwierdzony przez osobę. Zabezpieczenie rejestruje, czy domena została w pełni zbadana, pobrana, odzyskana lub nieprzetworzona. Negatywny wniosek jest dozwolony tylko wtedy, gdy odpowiednie

wyszukiwanie zostanie zamknięte zgodnie z deklarowaną polityką. Jest to różnica między niezawodnym systemem symbolicznym a płynnym klasyfikatorem o wyjściu w kształcie JSON.

Dyskusja pozostaje dostępna, a nie dowodowa. Interpretacja abstrakcyjna, wykonanie symboliczne, sprawdzanie modeli i semantyka logiczna są wyjaśnione poprzez ich istotę operacyjną i obowiązki, które tworzą. Formalna notacja pojawia się tylko wtedy, gdy wyjaśnia rozróżnienie, które proza może się rozmyć. Celem nie jest przedstawienie matematyki dekoracyjnej. Ma to na celu zapewnienie projektantom systemów, recenzentom i badaczom wystarczającej precyzji koncepcyjnej, aby zdecydować, gdzie proponowana gwarancja jest ważna, gdzie jest warunkowa, a gdzie jest tylko aspiracyjna.

Wynikająca z tego teza można stwierdzić jasno. LLMs powinien odkryć prawdopodobne kształty formalne; deterministyczne i formalnie ograniczone komponenty powinny kontrolować uziemienie, kompilację, wykonanie i akceptację; LLMs może następnie zrealizować wynik kanoniczny jako język, ale tylko w ramach umowy, która uniemożliwia nieobsługiwane dodatki. Nie jest to kompletne rozwiązanie znaczenia. Jest to wiarygodna droga od probabilistycznego przetwarzania języka do ograniczonego, możliwego do wykonania i audytu rozumowania.

1. Od płynnego języka do wykonywalnej semantyki

1.1 Czego wymaga wykonanie i gdzie należy umieścić granicę zaufania

Słowo „wykonać” niesie silniejsze zobowiązanie niż „interpretować”, „odpowiedzi” lub „rozum”. Program można wykonać, ponieważ jego działanie ma określony wpływ na stan. Zapytanie bazy danych jest możliwe do wykonania, ponieważ operatorzy relacyjni określają, które krotki są wybierane, łączone, grupowane lub wykluczane. Problem z planowaniem jest możliwy do wykonania, ponieważ działania mają warunki wstępne i skutki, a planista szuka sekwencji, która osiąga cel. Teoria logiczna jest wykonalna w szerszym znaczeniu, ponieważ procedura wnioskowania określa, czy następuje wniosek, czy istnieje model, czy też ograniczenia są zadowalające. W każdym przypadku decydująca operacja jest zewnętrzna w stosunku do retoryki używanej do opisanego zadania. Wynik można odtworzyć z artefaktu i semantyki.

Odpowiedź LLM jest inna. Model mapuje kontekst do rozkładu nad kontynuacjami. Może internalizować wzorce, które przypominają algorytmy, i może prawidłowo rozwiązywać wiele zadań, ale reguła stosowana na określonym kroku nie jest oddzielnie sprawdzalna lub gwarantowana, że będzie używana konsekwentnie. Podążanie za myślami może poprawić wydajność, ale łańcuch zdań nie jest obiektem dowodowym tylko dlatego, że jest zorganizowany jako kroki. Samospójność próbek kilka kontynuacji, a nie kilka formalnie zdefiniowanych egzekucji. Użycie narzędzia może sprawić, że wartość będzie dokładna, pozostawiając interpretację, która wytworzyła wywołanie narzędzia, niezawierającą. Kluczowe rozróżnienie to model, który proponuje artefakt możliwy do wykonania, a model, którego własne pokolenie jest traktowane jako wykonanie.

Wykonalny język naturalny zaczyna się zatem od materializacji. System musi utworzyć obiekt z deklarowaną gramatyką, systemem typu, semantyką i mapą źródłową. Obiekt ten może być zapytaniem relacyjnym, teorią logiczną, maszyną stanu, zestawem ograniczeń czasowych, tabelą decyzyjną, programem Datalog, skończonym automatem lub wpisanym wykresem z zarejestrowanymi operatorami. Szczególny wybór zależy od zadania. Ważne jest to, że zachowanie obiektu nie jest już określane przez rozkład następnego tokenu modelu językowego. Można go odrzucić, wykonać, przetestować, porównać i niezależnie odtwarzać.

Wymóg ten jest silniejszy niż zwykła produkcja strukturalna. Model może emitować prawidłowe składniowo JSON, które zawiera wymyślone jednostki lub wzajemnie niespójne pola. Schemat potwierdza kształt, a nie prawdę. Podobnie, model może generować legalne SQL, które łączą niewłaściwe kolumny, ważną logikę pierwszego rzędu, która odwraca kwantyfikator, lub formułę czasową, która wyraża przeciwną relację precedensową. Przydatna granica zaufania nie jest zatem po prostu „po zgenerowaniu JSON”. Leży po uziemieniu źródła, sprawdzaniu typów, zgodności semantycznej i formalnej kompilacji. Każdy symbol, który wpływa na odpowiedź, musi mieć dopuszczalne pochodzenie i określoną rolę.

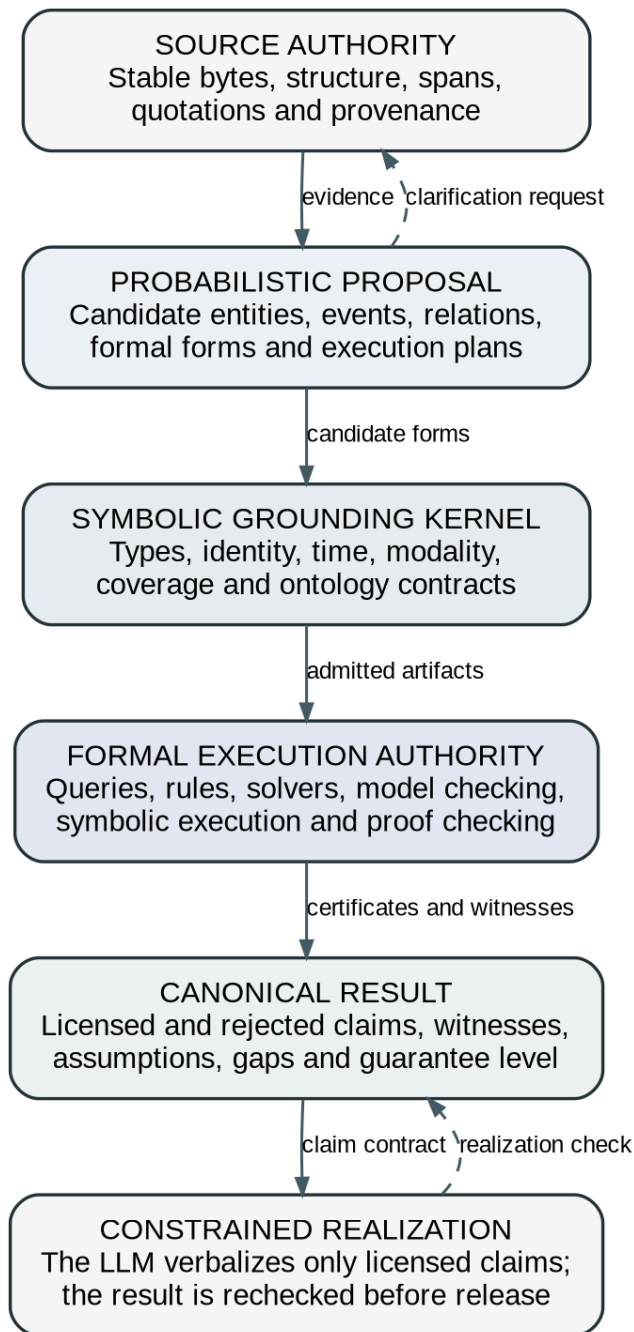
Architektura naturalnie oddziela cztery władze. Autorytet źródłowy określa, jaki tekst, dane lub obserwacje zostały faktycznie dostarczone. Władza semantyczna określa, które typy, relacje, sposoby i polityka tożsamości są dozwolone. Władza reguł określa, które ograniczenia lub przekształcenia stanowią teorię, która jest wykonywana. Organ egzekucyjny określa, czy wynik kandydata wynika z tej teorii. LLM może pomóc wszystkim czwórkom, ale nie powinien ich pomilczeć. Kiedy ten sam model wymyśla założenie, wybiera ontologię, stosuje regułę i pisze werdykt, nie ma znaczącego rozdziału między dowodami a wnioskiem.

Władza Źródłowa zaczyna się od stabilnych bajtów i rozpiętości. System analizy dokumentów powinien zachować dokładną rewizję, strukturę i lokalizację każdego cytowanego fragmentu. Powinien on odróżnić cytaty od podsumowania i podsumowanie od wniosku. Gdy model proponuje, aby zdanie wyraziło obowiązek, obserwacja musi wskazać zdanie i zachować model, szybki, schemat i alternatywy, które doprowadziły do klasyfikacji. Jeśli cytatu nie można znaleźć w źródle, obserwacja jest raczej odrzucana niż akceptowana jako prawdopodobna rekonstrukcja. Brzmi to jak przyziemna inżynieria pochodzenia, ale jest to warunek wstępny dla każdej późniejszej gwarancji.

Władza semantyczna jest zwykle dystrybuowana w górnej ontologii i pakietach domen. Uniwersalna ontologia jest atrakcyjna, ponieważ obiecuje interoperacyjność, ale ma tendencję do stawania się zbyt abstrakcyjną, aby decydować o przydatnych pytaniach lub zbyt dużą i sporną, aby rządzić. Lepsza konstrukcja definiuje małą wspólną warstwę dla rozpiętości źródeł, wzmianek, podmiotów, wydarzeń, czasu, światów, statusu, pochodzenia, alternatyw, zasięgu i luk. Następnie pakiety domen definiują pojęcia takie jak interwencja kliniczna, obowiązek regulacyjny, zdarzenie obiektu narracyjnego, wywołanie oprogramowania API lub transakcja finansowa. Każdy pakiet musi zawierać nie tylko nazwy pól, ale także zobowiązania semantyczne i testy. Typ zwany „zatwierdzeniem” nie ma znaczenia, chyba że system określa, czy oznacza dokument, akt, państwo, decyzję organu lub jakąś kombinację.

Władza zasad powinna być wersjonowana oddzielnie od interpretacji dokumentów. Polityka, teoria nadpociągu lub model naukowy zmienia się w innym tempie niż źródło, które analizuje. Jeśli reguły są osadzone w niewidocznie w monitach, trudno jest wiedzieć, czy dwa runy zastosowano tę samą teorię. Wersjonowane artefakty reguł umożliwiają porównawczą publikację, separacyjne różnice i wycofanie. Pozwalają również na wiele teorii dotyczących tej samej reprezentacji dokumentów. Rękopis można sprawdzić pod kątem spójności terminologii, spójności przyczynowej i wsparcia cytowania bez przekompilowania go w inny nieprzejrzysty monit dla każdej reguły.

Organ egzekucyjny powinien być tak mały i mechaniczny, jak i praktyczny. W przypadku zapytań relacyjnych może to być deterministyczny ewaluator. W przypadku ograniczeń, rozwiązanie SMT lub CSP. Dla zachowania czasowego, model checker. Dla właściwości programu specyficznego dla ścieżki, symboliczny wykonawca. W przypadku zobowiązań dowodowych, twierdzenie dowództwo plus mały sprawdzacz dowodowy. W przypadku niestandardowego języka obwodów może to być ograniczony interpreter z zarejestrowanymi operatorami i niezależnymi weryfikatorami. Składnik akceptacji nie powinien pytać LLM, czy wynik „wygląda poprawnie”. Powinien sprawdzić świadka w stosunku do źródła i teorii.



Rysunek 1. Zaufaj granicy dla wykonalnego języka naturalnego. LLM proponuje formularze semantyczne, podczas gdy uziemienie źródła, formalne wykonanie, akceptacja i ostateczne licencjonowanie roszczeń pozostają niezależnie sprawdzalne.

Przydatnym sposobem klasyfikacji systemów jest lokalizacja ostatecznej wyroczni. W układzie czysto neuronowym LLM jest wyrocznią. W systemie augmentowanym narzędziem narzędzie oblicza wartość, ale LLM może pozostać wyrocznią, czy wywołanie narzędzia reprezentuje pytanie. W systemie wspieranym przez solvera solver jest wyrocznią dla konsekwencji wygenerowanej formalizacji. W systemie opartym na dowodach, jądro solver i jądro uziemienia wspólnie określają przyjęty wynik. W systemie przenoszenia proofów nawet wyjście solvera może być sprawdzane przez mniejsze zaufane jądro. Każdy krok zawęża przestrzeń, w której błąd probabilistyczny może milcze zmienić werdykt.

Nie oznacza to, że każda aplikacja wymaga zapewnienia na poziomie twierdzenia-przysłowia. Asystent pisma może używać lekkich obwodów deterministycznych, podczas gdy system regulacyjny lub system bezpieczeństwa może wymagać formalnych artefaktów dowodowych. Ważną zasadą jest to, że gwarancja powinna pasować do mechanizmu. Deterministyczny czek regex może poświadczyć obecność dosłownego wzorca, a nie brak koncepcji. Wyciąg zdarzenia wspomagany modelem może obsługiwać proponowane ostrzeżenie o ciągłości, a nie mechanicznie pewną relację o tożsamości narracyjnej. Rozwiązywacz może ustalić niezaspokojenie ograniczeń, a nie kompletność tłumaczenia w języku naturalnym. Rigor polega w dużej mierze na odmowie przeniesienia zaufania przez te granice.

Najsilniejsza architektura jest więc federacyjna. Składnik neuronowy nie jest rozpuszczony w sieci symbolicznej, a symboliczny komponent nie jest zredukowany do zróżnicowanego regularyzatora. Pozostają odrębnymi systemami z interfejsem typowanym. LLM jest odpowiedzialny za szerokość, generowanie hipotez, normalizację semantyczną i odzyskiwanie z niezwykłego języka. Rdzeń formalny jest odpowiedzialny za dopuszczalność, wykonanie, niezmienniki, świadków i odtwarzalność. Przegląd 2026 przeprowadzony przez Chatzikyriakidis i Lappin nazywa powiązane systemy federacyjne i zauważa, że często wykazują one silne zyski w zadaniach, w których postrzeganie można oddzielić od wnioskowania. Autorzy podkreślają również, że porównania pozostają zdeorientowane różnicami zadań. Argument architektoniczny dla federacji jest jednak niezależny od przewagi porównawczej: oddzielne moduły umożliwiają audytowanie roszczeń i umożliwiają jednemu składnikowi odrzucenie innego [Chatzikyriakidis and Lappin 2026].

Ostateczna produkcja naturalnojęzyczna wymaga własnej granicy. Gdy solver zwróci wynik, kuszące jest dostarczenie śladu do LLM i poproszenie o wypolerowaną odpowiedź. Bez ograniczeń model może dodać wyjaśnienia w tle, wnioskować o motywach lub uprościć niepewność do pewności. Wynik kanoniczny powinien zatem wyliczyć twierdzenia, które mogą być zwerbalizowane, ich status, dowody, założenia i zakres. Renderer może wybrać sformułowanie i organizację, ale należy go sprawdzić w stosunku do struktury kanonicznej. Jedną z praktycznych metod jest realizacja na poziomie roszczenia: model generuje zdania powiązane z identyfikatorami wyników; weryfikator zapewnia, że każde zdanie jest pociągnięte za sobą lub parafrazuje dozwolone twierdzenie; nieobsługiwany materiał jest usuwany lub określany jako komentarz. W ustawieniach o wysokiej pewności szablonowy język kontrolowany może być lepszy niż nieograniczona proza.

Granice zaufania zaproponowaną w tej książce można podsumować bez kodu. LLMs może zaproponować formy semantyczne, odwzorowania ontologiczne, wybory formalizmu, naprawy i wyjaśnienia. Nie ustanawiają dowodów, zamykają przeszukanie w otwartym świecie, certyfikują tożsamość, decydują o zadowalalności ani nie zezwalają na ostateczne roszczenie. Funkcje te należą do wyraźnych umów symbolicznych. Ta separacja zachowuje unikalną wartość modeli językowych, jednocześnie zapobiegając ich płynnościom, która może zostać pomyłona z dowodem.

1.2 Luka formalizacji: dlaczego poprawny solver nadal może wygenerować błędną odpowiedź

Centralną słabością większości neuro-symbolicznych systemów rozumowania nie jest silnik wnioskowania. Rozwiązywacze SAT, solvery SMT, silniki baz danych, twierdzenia, sprawdzanie modeli i symboliczne wykonawcy są często znacznie bardziej niezawodne niż model językowy, który je poprzedza. Słabość tkwi w luce formalizacyjnej: różnica między tym, co oznacza źródło w kontekście, a formalnym przedmiotem zgłoszonym do wykonania. System może być doskonale dźwiękowy od formalnego wejścia do formalnego wyjścia, a jednocześnie niedźwiękowy z języka naturalnego, aby odpowiedzieć.

Rozważ proste zdanie polityczne: „Recenzent może zatwierdzić wydanie dopiero po zakończeniu raportu bezpieczeństwa, chyba że dowódca incydentu upoważnia do wyjątku awaryjnego”. Formalizator musi zachować co najmniej aktora, działanie, przedmiot, stan czasowy, stan ukończenia, wyjątek, autorytet i zakres „tylko po”. Musi odróżnić pozwolenie od obowiązku. Musi zdecydować, czy wyjątek usuwa tylko stan czasowy, czy też upoważnia do zatwierdzenia. Musi połączyć „raport bezpieczeństwa” z prawidłowym artefaktem i „dowodcą incydentu” z tożsamością lub rolą. Jeśli którykolwiek z tych wyborów jest błędny, rozwiązujący może prawidłowo egzekwować niewłaściwą politykę.

Zakres kwantyfikatorów jest klasycznym źródłem błędu. „Każdy badacz dokonał przeglądu pracy” może oznaczać, że każdy badacz dokonał przeglądu co najmniej jednego artykułu, niekoniecznie tego samego. „Praca została przejrzana przez każdego badacza” ma inną strukturę. Negacja stwarza podobne problemy: „nie każda przeprowadzona próbka” nie oznacza, że żadna próbka nie została przekazana. Warunki, porównania, modalność, stosunki czasowe, anafora i założenie tworzą rozróżnienia, które tłumaczenie na poziomie powierzchni może wymazać. Formalne tradycje semantyczne opracowały wyrafinowane mechanizmy dla tych zjawisk, ale język otwartej domeny dodaje wiedzę o świecie, konwencje gatunkowe i ukryte cele, których żadna pojedyncza reprezentacja nie oddaje całkowicie.

Luka pojawia się również w semantycznym parsowaniu z denotacji. System przeszkolony tylko na pytanie i ostateczną odpowiedź może znaleźć program, który zapewnia poprawną odpowiedź w środowisku treningowym. Takie fałszywe programy są znanym problemem w słabo nadzorowanym parsowaniu semantycznym [Pasupat and Liang 2016; Guu et al. 2017; Goldman et al. 2018]. Wykonanie nie odróżnia zamierzonego programu od przypadkowego, jeśli oba dają taką samą denotację. Systemy Text-to-SQL napotykają to samo ryzyko: zapytanie może zwrócić odpowiednie wiersze z powodu aktualnych wartości danych, jednocześnie zawierających niewłaściwe łączenie lub filtr. Dekodowanie prowadzone przez wykonanie usuwa programy, które ulegają awarii lub są sprzeczne z ograniczeniami pośrednimi, ale nie może udowodnić wierności semantycznej [Wang et al. 2018].

Luka formalizacyjna najlepiej rozkłada się na odrębne zobowiązania. Uziemienie leksykalne pyta, które przewidywania lub pola odpowiadają słowom i zwrotom. Referencyjne uziemienie pyta, które wzmianki oznaczają tę samą jednostkę. Uziemienie strukturalne pyta, które argumenty, zakresy i zależności należą do siebie. Uziemienie czasowe łączy frazy z momentami, interwałami, kalendarzami i uporządkowaniem. Modal grounding rozróżnia fakty, możliwości, obowiązki, uprawnienia, intencje, zgłoszone roszczenia i kontrfaktyczne. Uziemienie ontologiczne określa, czy pojęcie należy do wybranego modelu domeny. Dowodowe

uziemiać łączy każdą przesłankę z rozpiętością źródła, czujnikiem, wierszem bazy danych lub deklarowanym założeniem. Każdy obowiązek może nie zostać niezależnie.

System musi również zdecydować, jak duże znaczenie jest sformalizowane. Całkowita semantyczna reprezentacja nieograniczonej prozy nie jest ani praktyczna, ani konieczna dla większości zadań. Reguła, która jest wykonywana, stwarza popyt. Obwód terminologiczny może wymagać tylko dokładnych rozpiętości i znormalizowanych terminów. Reguła zgodności czasowej może wymagać zdarzeń, aktorów, dat i wyjątków. Analiza przyczynowa może wymagać roszczeń, interwencji, wyników i niepewności. Formalizacja oparta na żądaniu zmniejsza koszty i ogranicza liczbę nieuzasadnionych zobowiązań. Stwarza również wyraźny problem z pokryciem: jeśli reguła wymaga koncepcji, której dostępni producenci nie mogą zmaterializować, bieg musi zatrzymać lub zwrócić nieznany, a nie cicho przybliżony.

Dwuznaczność powinna być reprezentowana jako alternatywa, a nie natychmiast załamana. Załóżmy, że „Alex powiedział Jordanowi, że powinni odejść” pojawia się w procedurze. Zaimek może odnosić się do Alex, Jordani lub grupy, w zależności od kontekstu i konwencji językowych. Model może plasować kandydatów, ale wysokie zaufanie nie jest dowodem. Jeśli werdykt reguły niższego szczebla jest taki sam pod wszystkimi prawdopodobnymi wiązaniami, dwuznaczność jest nieistotna, a wykonanie może być kontynuowane. Jeśli werdykt się zmieni, wynik powinien ujawnić zależność i zażądać wyjaśnienia lub dodatkowego kontekstu. Jest to miejsce, w którym abstrakcyjna interpretacja staje się pożyteczna koncepcyjnie: może obliczać wnioski nad zestawem interpretacji, nie wyliczając każdej kombinacji w pełnych szczegółach.

Kompletne mapowanie jest często niemożliwe, ponieważ język naturalny jest otwartym światem. Dokument może pominąć fakty, które są prawdziwe, przyjąć powszechną wiedzę lub polegać na praktyce instytucjonalnej. Rozumowanie w zamkniętym świecie traktuje nieodnotowane fakty jako fałszywe; rozumowanie w otwartym świecie traktuje je jako nieznanne. Obie polityki są odpowiednie w różnych kontekstach. Inwentaryzacja bazy danych może zostać zamknięta dla określonej tabeli i czasu. Zbiór odzyskanych wiadomości e-mail nie jest ogólnie kompletnym dowodem na to, że nie wysłano powiadomienia. Warstwa uziemiać musi zatem przywiązywać zasięg do obserwacji i wyszukiwań. Negatywne wnioski wymagają żetonu pokrycia, który określa domenę, rewizję źródła, zakres, metodę, wykluczenia i politykę kompletności.

Wiedza zewnętrzna wprowadza kolejne komplikacje. Ogólne LLM zawiera obszerne, ale niewywrócone i potencjalnie nieświeże informacje. System symboliczny nie może traktować pamięci modelowej jako ponadczasowego faktu. Światowa wiedza powinna zawierać się poprzez pozyskiwane, datowane, specyficzne dla jurysdykcji i treści. Pakiet może stwierdzić, że reguła prawna była skuteczna w określonej jurysdykcji w określonym przedziale lub że stała naukowa została zaciągnięta z nazwanego źródła. Konflikty między opakowaniami powinny pozostać widoczne. Światy fikcyjne lub hipotetyczne muszą być wybierane bez skażenia przez fakty z obecnego świata. Dokumentacja nllAgent prawidłowo traktuje takie pakiety jako granicę, a nie udawanie, że pamięć modelu fundamentów zapewnia formalny autorytet.

Wierność formalizacji może być testowana, choć nie jest to powszechnie udowodnione. Jedną z metod jest dwukierunkowe sprawdzenie: przetłumacz obiekt formalny z powrotem na język kontrolowany i porównaj go ze źródłem. Innym jest test kontrastywny: wygeneruj minimalne zmiany tekstowe, które powinny zmienić jeden element formalny i zachować resztę. Trzecie to porównanie rozszerzające: poproś o konkurencyjne formalizacje, aby wygenerowały wyróżniające przykłady lub wyniki zapytań. Czwarty to stabilność parafrazy: semantycznie

równoważne preparaty powinny kompilować się do równoważnych lub prawdopodobnie powiązanych artefaktów. Piąta jest kontrprzykładnym wyjaśnieniem: kiedy solver wytwarza świadka, odwzoruj każdą zależność od rozpiętości źródeł i zapytaj, czy scenariusz jest dopuszczalny językowo. Szósty to przegląd człowieka skupiony na małym zestawie wyborów, które wpływają na wynik, a nie na cały program formalny.

Systemy interaktywne, takie jak nl2spec, ilustrują wartość lokalnej naprawy. Zamiast prosić użytkownika o zrozumienie lub przepisanie całej formuły czasowej, interfejs kojarzy podformuły z fragmentami źródła, umożliwiając korektę [Cosler et al. 2023] błędnego komponentu. Informacje zwrotne Solvera mogą odgrywać podobną rolę. Logic-LM wykorzystuje błędy składni i wykonania, aby uzyskać monit o reformalizacji [Pan et al. 2023]. Bardziej zaawansowane systemy mogą wykorzystywać niezaspokojone rdzenie lub kontrprzykłady, a nie ogólne komunikaty o błędach. Podstawowym wglądem jest to, że formalne informacje zwrotne powinny zawęzić kwestię językową.

Luka formalizacyjna nakłada również ograniczenia na kompleksowe roszczenia dotyczące „dźwiękowości”. System może okazać się twierdzeniem formy: jeśli każda przyjęta obserwacja dokładnie reprezentuje jej źródło, a ontologia i zasady są prawidłowe, następuje werdykt. To jest cenne. Nie należy go przepisywać jako „werdykt jest udowodniony z dokumentu”, chyba że obowiązki związane z obserwacją i uziemieniem również zostały wypłacone. Gwarancja powinna wymieniać jej założenia i niewiadome. Koperta dźwiękowa jest bardziej uczciwa niż binarna etykieta zaufania.

Ta książka traktuje zatem formalizację jako proces kompilacji z wieloma artefaktami pośrednimi, a nie jednym monitem. Źródło jest segmentowane i zakotwiczone. Obserwacje kandydatów są tworzone w ramach schematów. Reprezentowane są alternatywy tożsamości i czasu. Kontrola zgodności ontologicznej. Wybierany jest formalizm docelowy. Kompilator tworzy obiekt wykonywalny i mapę źródłową. Wykonawca wytwarza świadka, dowód, model lub porażkę. Wynik jest odtwarzany i renderowany. Na każdym etapie błąd ma określoną kategorię i ewentualną naprawę. Architektura nie eliminuje niepewności semantycznej; zapobiega ukrywaniu się niepewności wewnątrz płynnego wytwarzania.

Następuje najważniejszy wniosek badawczy. Systemy LLM zabezpieczone przez Solvera powinny być oceniane oddzielnie pod kątem wierności formalizacji i formalnego wykonania. Wysoka ocena ostatecznej odpowiedzi może ukryć fałszywe programy. Niski wynik może być spowodowany dobrym sformalizowaniem z nieodpowiednim solverem lub przez złą formalizację z doskonałym solverem. Punkty odniesienia potrzebują złotych lub dopuszczalnych struktur formalnych, wyrównań dowodów i semantycznych minimalnych par, a nie tylko odpowiedzi. Dopóki separacja ta nie stanie się standardem, wiele twierdzeń o „formalnym rozumowaniu z LLMs” pozostanie trudnych do interpretacji.

2. Intelktualny rodowód języka wykonywalnego

2.1 Języki kontrolowane, semantyka formalna, dyskurs i logika naturalna

Pomysł, że język czytelny dla człowieka może wspierać wnioskowanie mechaniczne, poprzedza duże modele językowe przez dziesięciolecia. Wcześniejsza praca zbliżyła się do problemu z przeciwnego kierunku: zamiast prosić model statystyczny o interpretację nieograniczonej prozy, naukowcy zaprojektowali języki lub teorie semantyczne, które ograniczyły dwuznaczność na tyle, aby wspierać tłumaczenie na logikę. Tradycje te są niezbędne, ponieważ ujawniają, co system nauczania musi zachować, nawet gdy LLM przeprowadza analizę front-end.

Kontrolowane języki naturalne są najczystszy precedensem inżynieryjnym. Ograniczają słownictwo, gramatykę lub interpretację, zachowując jednocześnie powierzchnię, która przypomina zwykły język. Attempto Controlled English jest najbardziej znanym przykładem. Zdania są przekładane na struktury reprezentacji dyskursu, a następnie na logikę formalną, umożliwiając zapytania i wnioskowanie, pozostając czytelny dla nielogików [Fuchs and Schwitter 1996; Fuchs et al. 2008]. Języki kontrolowane były używane do reprezentacji wiedzy, specyfikacji technicznych, wiki semantycznej, zasad biznesowych i wymagań. Badanie Kuhna pokazuje, że rodzina jest zróżnicowana: niektóre języki są zaprojektowane z myślą o ludzkiej jasności, a nie na formalnej precyzji, podczas gdy inne handlują naturalnością dla deterministycznej semantyki [Kuhn 2014].

Trwała lekcja nie polega na tym, że nieograniczony język powinien być zastąpiony jednym kontrolowanym dialektem. Chodzi o to, że system wykonywalny potrzebuje warstwy normalizacyjnej, której semantyka jest bardziej rygorystyczna niż zwykła proza. LLM może służyć jako tłumacz od otwartego języka do kontrolowanej formy semantycznej, ale kontrolowana forma musi mieć niezależną gramatykę i interpretację. Bez tej granicy „kontrolowany język naturalny” staje się tylko preferowanym stylem pisania i nie daje gwarancji obliczeniowej.

Języki kontrolowane również ujawniają znaczenie opinii autorów. Jeśli zdanie znajduje się poza kontrolowanym fragmentem, tradycyjny system odrzuca je lub wytwarza wyraźną dwuznaczność. Nowoczesne interfejsy LLM mają tendencję do robienia czegoś przeciwnego: wybierają wiarygodny odczyt i kontynuują. Odrzucenie jest często traktowane jako wada użyteczności, ale w systemie o wysokiej pewności jest to forma integralności epistemicznej. Interfejs w języku wykonywalnym nowej generacji powinien łączyć elastyczność parafrazy LLM z uczciwością kompilatora w kontrolowanym języku. Może zaproponować kilka znormalizowanych odczytów, wyjaśnić rozróżnienia i zapytać tylko wtedy, gdy rozróżnienie zmieni wynik.

Semantyka formalna zapewnia głębszą linię teoretyczną. Model-teoretyczna semantyka reprezentuje znaczenie w warunkach, w których wyrażenia są prawdziwe w modelu. Kompozycjonalność łączy interpretację złożonego wyrażenia ze znaczeniami jego części i ich kombinacją składniową. Semantyka w stylu Montague, typowane kalkulacje lambda, uogólnione kwantyfikatory i powiązane ramy oferują precyzyjne relacje z zakresu, struktury argumentów i operatorów logicznych. Wykazują, że znaczenie w języku naturalnym nie jest zredukowane do płaskiej listy potrójnych. Kwantyfikatory, negacja, czasowniki intensywne, modalność i kontekst mogą zmienić logiczne zachowanie pozornie prostych fraz.

W przypadku języka naturalnego podlegającego wykonaniu, semantyka formalna spełnia dwa wymagania. Po pierwsze, reprezentacja pośrednia musi zachować strukturę operatora, a nie tylko podmioty i relacje. „Każdy”, „niektóre”, „nie”, „tylko”, „chyba”, „może”, „musi” i „przed” nie są adnotacjami wokół propozycji; określają przestrzeń modeli. Po drugie, konstrukcja semantyczna powinna być świadoma typu. Czasowy odstęp czasowy, osoba, organizacja, propozycja i obowiązek nie mogą być swobodnie podstawione tylko dlatego, że ich etykiety powierzchniowe przypominają sobie nawzajem. Sprawdzenie typu jest jednym z najtańszych sposobów odrzucenia niespójnych formalizacji przed wywołaniem solvera.

Teoria reprezentacji dyskursu i związane z nią podejścia dynamiczne-semantyczne odnoszą się do zjawisk, które logika zdań według zdania źle sięga. Dyskurs wprowadza referentów, przenosi je przez zdania, aktualizuje kontekst i tworzy ograniczenia dostępności dla zaimków i opisów. Parserzy Boxer i DRS zoperacjonalizowali te pomysły do szeroko zakrojonej analizy semantycznej [Bos 2008; Liu et al. 2018]. Równoległy Bank Znaczenia i wspólne zadania parsowania DRS umożliwiły ocenę zakresowych reprezentacji znaczenia w różnych językach [Abzianidze et al. 2019]. Wysiłki te pokazują zarówno postęp, jak i trudności: zmienne wiązanie, przydział zmysłów, zakres i relacje dyskursu są znacznie trudniejsze niż płytkie wydobycie.

System wykonywalny na poziomie dokumentu powinien uczyć się z dynamicznej semantyki, nie wymagając jednego monolitycznego DRS dla każdego źródła. Potrzebuje uporczywych obiektów dyskursu, hipotez podmiotów, wydarzeń, światów i aktualizacji relacji. Musi odróżnić wzmiankę od podmiotu i łączyć tożsamości kandydata od ustalonego. Musi zachować cytowane, hipotetyczne, negowane i fikcyjne konteksty. Płaski wykres wiedzy, który łączy równe ciągi w jeden węzeł, będzie popełniał systematyczne błędy w długich dokumentach. Nazwy powtarzają się, pojawiają się aliasy, zmieniają się role, a dokumenty opisują wzajemnie niespójne światy.

Praktyczny koszt pełnego formalnego parsowania semantycznego wyjaśnia, dlaczego wiele systemów zmierzało w kierunku płytszych reprezentacji. Semantyczne oznakowanie ról identyfikuje strukturę predykat-argument. Abstrakcyjne znaczenie reprezentacji reprezentuje pojęcia i relacje na wykresie. Wydobycie informacji identyfikuje podmioty, zdarzenia i relacje. Te reprezentacje są przydatne, ale ich semantyka jest często niekompletna do wykonania. Mogą one pominąć zakres kwantyfikatora, modalność, zakotwiczenie czasowe oraz różnicę między zaopatrzoną i raportowaną zawartością. Runtime w języku wykonywalnym powinien traktować takie struktury jako obserwacje dowodowe, a nie jako kompletny formularz logiczny. Wykres może obsługiwać wiele lokalnych zasad, pozostawiając nierozwiązane aspekty.

Logika naturalna daje kolejny ważny kompromis. Zamiast tłumaczyć zdania na nieograniczony język formalny, rozum systemów naturalno-logicznych poprzez ustrukturyzowane przemiany w pobliżu postaci powierzchni. MacCartney i Manning rozwinęli relacje wnioskowcze oparte na monotoniczności, zaręczynach leksykalnych i projekcji [MacCartney and Manning 2007, 2009]. NaturalLI później wykorzystało naturalne operacje logiczne do skutecznego wnioskowania nad tekstem [Angeli and Manning 2014]. LangPro łączy w sobie formalną gramatykę, wiedzę leksykalną i twierdzenie świadczące o wnioskowaniu w języku naturalnym [Abzianidze 2017].

Naturalna logika jest atrakcyjna, ponieważ wiele pytań dotyczących dokumentów dotyczy relacji związanych z tym, które można uchwycić bez konstruowania dużego modelu światowego. Zastąpienie „psa” „zwierzęciem” jest ważne w kontekście uwikłanym w górę, ale niekoniecznie w wyniku negacji lub uniwersalnych ograniczeń. Obliczenia te są bardziej przejrzyste niż wynik podobieństwa osadzania. Mogą one wspierać kontrole sprzeczności,

rozumowanie taksonomiczne i kontrolowane wnioskowanie. Jednak logika naturalna nie jest uniwersalnym wykonawcą. Dobrze radzi sobie z pewną monotonicznością i relacjami leksykalnymi, ale nie wyraża naturalnie arbitralnych procedur, arytmetyki, planowania lub złożonych zachowań czasowych. W architekturze multiformalizmu powinien być jednym silnikiem spośród kilku.

Tradycje semantyczne wyjaśniają również, dlaczego „uziemia” ma kilka poziomów. Semantyka formalna może określać, w jaki sposób wyrażenie przyczynia się do spełnienia warunków prawdy, ale nie określa automatycznie, który obiekt w świecie rzeczywistym oznacza nazwę lub czy źródło jest wiarygodne. DRS może reprezentować referent dyskursu, ale rozdzielczość tożsamości pozostaje zadaniem empirycznym. Logika naturalna może propagować pociągnięcie leksykalne, ale relacja leksykalna może być zależna od domeny. Wykonalność wymaga zatem partnerstwa między strukturą językową, ontologią dziedzinową i zarządzaniem dowodami.

Przydatna architektura może rozróżnić co najmniej trzy produkty semantyczne. Pierwszy to model źródłowy zawierający dokładną strukturę, rozpiętości i cytaty. Druga to warstwa interpretacyjna zawierająca wpisane wzmianki, wydarzenia, relacje, sposoby i alternatywy. Trzecia to teoria wykonywalna, która wykorzystuje wybrane interpretacje, aby odpowiedzieć na konkretne pytanie. Formalne techniki semantyczne przede wszystkim informują drugą warstwę. Kontrolowane języki i twierdzenia łączą drugie i trzecie. Pochodzenie i zasięg łączą się z powrotem do pierwszego.

Wcześniejsza literatura ostrzega również przed założeniem, że jedna reprezentacja może służyć każdemu zadaniu. DRS jest wyrazisty dla dyskursu, ale drogi w produkcji i rozumie. Logika pierwszego rzędu jest dobrze zrozumiała, ale niezręczna dla zjawisk czasowych, probabilistycznych lub możliwych do krawstwa. Logika opisu oferuje rozstrzygające rozumowanie ontologiczne, ale ogranicza ekspresyjność. Logika naturalna pozostaje bliska językowi, ale jest wyspecjalizowana. Kontrolowany angielski poprawia przewidywalność poprzez ograniczenie danych wejściowych. Prawidłowym wnioskiem nie jest wybranie zwycięzcy, ale zachowanie wystarczającej ilości informacji semantycznych, aby obniżyć zadanie do formalizmu, który go pasuje.

LLMs zmienia ekonomię tego projektu. Mogą generować parsy kandydatów, przepisywać niekontrolowany język na kontrolowane formy, sugerować odwzorowania ontologii i wyjaśniać formalne alternatywy dla użytkowników. Uczynią one praktycznym próbowaniem semantycznej interpretacji na dokumentach, które odrzuciłyby ręcznie pisane gramatyki. Nie usuwają one potrzeby rozróżnień semantycznych opracowanych przez wcześniejsze prace. Wręcz przeciwnie, te rozróżnienia stają się umowami, za pomocą których zCXQ0001QXZ są oceniane propozycje.

Powtarzającym się błędem we współczesnych systemach jest traktowanie potrójnego sklepu generowanego przez LLM jako kompletnej reprezentacji semantycznej. Potrójne są wygodne dla zapytań wyszukiwania i wykresów, ale nie kodują one samodzielnie zakresu, pochodzenia, modalności, czasu, negacji, alternatyw tożsamości lub pokrycia. Wykonalna warstwa semantyczna powinna być bogatsza, ale nadal skończona i sprawna. Może przechowywać twierdzenie jako typowany obiekt z uczestnikami, kontekstem czasowym, polaryzacją, modalnością, światem, dowodami, producentem i statusem. Może reprezentować alternatywne wiązania bez wyboru jednego. Może ogłosić, które wymiary są nieobecne. Jest to mniej

elegancka niż pojedyncza forma logiczna, ale jest bardziej wierna temu, jak prawdziwe systemy nabierają znaczenia.

Najgłębszym wkładem linii sprzed LLM jest zatem dyscyplina koncepcyjna. Czytelność człowieka nie oznacza determinacji semantycznej. Formalną determinację osiąga się poprzez ograniczenie, pisanie, wyraźny kontekst i odrzucenie niepodniesionych konstrukcji. Znaczenie dokumentu jest dynamiczne i zakresowe. Wnioskowanie w pobliżu języka powierzchni jest możliwe, ale wyspecjalizowane. Wiele formalnych reprezentacji jest uzasadnionych, ponieważ odpowiadają na różne pytania. Te lekcje definiują architekturę, którą kompilator oparty na LLM musi odzyskać, a nie ominąć.

2.2 Parsowanie semantyczne, zapytania wykonywalne, indukcja programów i wczesne systemy neuro-symboliczne

Parsowanie semantyczne przekształca wypowiedzi w języku naturalnym w obiekty formalne, które można wykonać w oparciu o bazę danych, bazę wiedzy, środowisko lub interpreter. Jest to najbliższy historyczny poprzednik proponowanego tutaj rurociągu w języku wykonywalnym. Pole obejmuje odpowiedzi na pytania dotyczące tabel i wykresów wiedzy, tekst-do-SQL, instrukcje, regularna synteza ekspresji, interpretacja zrobotyzowanych poleceń i generowanie programu. Jego sukcesy pokazują siłę zewnętrznego wykonania; jego porażki ujawniają, dlaczego samo wykonanie nie ustanawia zrozumienia.

Wczesne parsery semantyczne polegały na gramatyce, leksykonach i nadzorowanych formach logicznych. Mogły one wytwarzać przejrzyste reprezentacje, ale wymagały drogich adnotacji i często przenosiły się słabo w różnych dziedzinach. Słaby nadzór zmniejszył obciążenie adnotacją dzięki szkoleniom z denotacji: pytanie jest połączone z odpowiedzią, a system szuka programu, który go daje. Takie podejście umożliwiło szersze zbiory danych, ale stworzyło problem programu fałszywego. Wiele programów może przypadkowo zwrócić tę samą odpowiedź, szczególnie w małych tabelach lub bazach wiedzy. Następnie sygnał szkoleniowy nagradza zachowanie bez identyfikacji zamierzonej semantyki [Pasupat and Liang 2016].

Problem jest strukturalny, a nie tylko statystyczny. Załóżmy, że odpowiedzią na „Which city goszczyło wydarzenie 2012?” jest Londyn. Przeznaczony jest program wybierający wiersz z rokiem 2012. Program wybierający pierwszy wiersz może również zwrócić Londyn w tabeli treningowej. Więcej danych może zmniejszyć przypadkowe korelacje, ale nie może zagwarantować, że wyuczony program odzwierciedla język. Abstrakcyjne przykłady, modele wyrównania, ograniczenia typu i filtrowanie oparte na wykonaniu zostały opracowane w celu tłumienia fałszywych kandydatów [Goldman et al. 2018; Lee et al. 2023]. Techniki te są również ważne dla LLM formalizacji. Wynik solvera należy traktować jako jeden test formalizacji, a nie jako jego certyfikat semantyczny.

Neural Symbolic Machines łączą generowanie programów neuronowych z symbolicznym interpreterem [Liang et al. 2017] Lisp. Składnik neuronowy proponuje programy, podczas gdy wykonawca odrzuca nieważnych kandydatów i zwraca denotacje. Mou i współpracownicy wyraźnie połączyli rozproszone wykonanie neuronowe z symbolicznym wykonaniem dla zapytań w języku naturalnym, zgłaszając poprawę dokładności, wydajności i interpretowalności [Mou et al. 2017]. Systemy te ustanowiły trwały wzór: modele neuronowe obsługują zmienność języka; symboliczni tłumacze egzekwują znaczenie operacyjne.

Tekst-do-SQL sprawił, że wzór był ważny komercyjnie. Pytanie użytkownika jest skompilowane w zapytanie, którego wykonanie zwraca dane. Dekodowanie prowadzone przez wykonanie może uruchamiać częściowych lub kompletnych kandydatów SQL i odrzucać te, które dają błędy lub niemożliwe wyniki [Wang et al. 2018]. Schemat łączący ograniczenia nazw i relacji. Nowoczesne LLMs znacznie poprawiają generowanie, ale uogólnienie między domenami, niejednoznaczne pytania, konwencje specyficzne dla bazy danych i prośby o brak odpowiedzi pozostają trudne. Kluczową lekcją dla wykonalnego języka naturalnego jest to, że samo środowisko formalne zapewnia użyteczne ograniczenia. Typy kolumn, klucze obce i błędy zapytań dostarczają informacji zwrotnych, których brakuje modelowaniu czystego języka.

Jednak tekst-do-SQL pokazuje również granice pojedynczej ostatecznej metryki dokładności. Dokładne dopasowanie ciągu może odrzucić równoważne zapytania. Dokładność

wykonania może akceptować semantycznie błędne zapytania, które zdarzają się zwracać ten sam wynik. Rygorystyczna ocena wymaga równoważności w różnych instancjach bazy danych lub starannie skonstruowanych kontrprzykładach. Jest to bezpośrednio analogiczne do oceny formalizacji reguł: wygenerowany program formalny powinien być testowany na scenariuszach rozróżniających, nie tylko na jednej zaobserwowanej odpowiedzi.

Synteza programu z języka naturalnego uogólnia ideę. Systemy generują wyrażenia regularne, fragmenty kodu, transformacje danych lub małe programy specyficzne dla domeny. Przykłady mogą ograniczać przestrzeń kandydującą, a systemy typu mogą wyeliminować bezsensowne programy. Neuronowe generowanie wyrażen regularnych wykazało, że mapowanie języka do programu może działać nawet przy ograniczonej wiedzy o domenach, a jednocześnie ilustruje kruchość, gdy opisy są podespecyfikowane [Locascio et al. 2018]. Programowany LLMs później przeniósł obliczenia poza model, ale podstawowy problem syntezy pozostał: model musi wybrać program, którego semantyka pasuje do żądania.

Solidny kompilator powinien zatem szukać wielu form dowodów. Opisy w języku naturalnym ograniczają intencje. Przykłady wejścia-wyjścia ograniczają zachowanie rozszerzenia. Rodzaje ograniczają dopuszczalne operacje. Schematy domenowe ograniczają słownictwo. Testy przypadków krawędzi sondy. Wygenerowany program, który spełnia wszystkie te ograniczenia, jest bardziej wiarygodny niż ten, który tylko wykonuje. Tę zasadę wielu dowodów można zastosować do reguł dokumentowania: autorytet tekstowy, przykłady pozytywne i negatywne, oczekiwane ustalenia i mutacje semantyczne powinny kształtować obwód przed publikacją.

Programowanie logiczne zapewnia kolejny cel wykonywalny. Datalog określa konsekwencje dla punktu rozpoznawczego i wspiera ocenę przyrostową. Prolog i powiązane języki reprezentują relacje i wyszukiwanie. Interpretacja abstrakcyjna od dawna jest wykorzystywana do analizy programów logicznych, pokazując pomost koncepcyjny między semantyką deklaratywną a analizą programu. W przypadku zastosowań w języku naturalnym programowanie logiczne jest dobrze dostosowane do monotonicznych zamknięć: relacji przejściowych, taksonomii, propagacji zależności i reguł polityki bez skomplikowanych domyślności. Jest mniej odpowiedni, gdy rozumowanie zależy w dużej mierze od niepewności, niespójnych źródeł lub niewyjątków niemonotonicznych, chyba że rozszerzono je o wyraźne mechanizmy.

Tabele decyzyjne i systemy przepisów produkcyjnych są często pomijane w przeglądach akademickich, ale są bardzo istotne. Wiele polityk nie jest głębokimi teoriami logicznymi; są to skończone kombinacje warunków i wyników. Tabela decyzyjna sprawia, że pokrycie, konflikty i brakujące kombinacje są widoczne. Można go skompilować w predykaty wykonywalne i systematycznie testować. Język naturalny może być używany do autorstwa lub wyjaśnienia tabeli, podczas gdy tabela pozostaje autorytatywną semantyką. W przypadku kłócenia dokumentów przedsiębiorstwa tabele decyzyjne mogą być bezpieczniejsze i łatwiejsze do zweryfikowania niż arbitralna logika pierwszego rzędu.

Instrukcje czasowe i proceduralne wymagają bogatszych celów. Maszyny o stanie skończonym, drzewa zachowań, języki planowania i logika czasowa reprezentują uporządkowane działania i warunki. Robotyka od dawna używa parserów językowych, które składają się na prymitywy wykonywalne. Ostatnie neurosymboliczne systemy manipulacji kontynuują ten schemat: parser językowy mapuje instrukcję do programu, którego składniki mogą obejmować percepcję neuronową i liczenie symboliczne, rozumowanie przestrzenne lub kontrolę działania. Sukces takich systemów zależy od ograniczonej biblioteki umiejętności i

środowiska uziemionego, a nie od nieograniczonego zasięgu semantycznego [Tziafas and Kasaei 2026].

Probabilistyczne programowanie logiczne i zróżnicowane systemy neuro-symboliczne próbują ściślejszej integracji. DeepProbLog, Logic Tensor Networks i powiązane systemy łączą wyuczone predykaty z logiczną strukturą [Manhaeve et al. 2018; Badreddine et al. 2022]. Są cenne, gdy niepewność i uczenie się są nieodłączne i mogą wprowadzać ograniczenia w trening. Różnicowość nie jest jednak równoznaczna z formalną pewnością. Rozmyte wartości prawdy i miękkie kary mogą kierować modelami bez wytwarzania dyskretnych zobowiązań dowodowych. W przypadku możliwego do wykonania języka naturalnego takie systemy mogą oszacować lub uszeregować kandydatów semantycznych, ale granica akceptacji o wysokiej pewności nadal korzysta z oddzielnego symbolicznego sprawdzania.

Przegląd neuro-symboliczny NLP 2026 wyróżnia układy wtryskowe, które kompilują informacje symboliczne w modele neuronowe, od systemów federacyjnych, które zachowują autonomiczne komponenty. Donosi, że systemy wpływowe dominują w literaturze, ale często wykazują skromne zyski w szerokich zadaniach NLU, podczas gdy systemy federacyjne wykazują większe zyski w zadaniach rozumowania. Autorzy wyraźnie ostrzegają, że porównanie jest zakłócanie przez różne poziomy odniesienia, a generalizacja poza domeną pozostaje poważnym problemem [Chatzikiriakidis and Lappin 2026]. Zrównoważona interpretacja polega na tym, że federacja jest architektonicznie atrakcyjna dla możliwości audytu i rozkładu zadań, ale jeszcze nie jest empirycznie udowodniona jako uniwersalna strategia skalowania.

Linia semantyczno-parsingowa wyjaśnia również rolę języka pośredniego. Reprezentacja musi być wystarczająco wyrazista, aby uchwycić istotne dla zadania znaczenie, ale wystarczająco ograniczona, aby skompilować niezawodnie. Jeśli jest zbyt blisko tekstu powierzchniowego, wykonanie pozostaje domyślne. Jeśli jest zbyt ogólna, złożoność tłumaczenia i solvera staje się nie do opanowania. Beiser i Penz opisują to jako pośredniojęzyczny problem we współczesnych systemach neuro-symbolicznych: wybór formalizmu może się poruszać, a nie rozwiązywać trudności [Beiser and Penz 2025]. Architektura multiformalizmu zmniejsza presję na każdy język, ale wprowadza nowy wymóg typowania trasy i spójności międzyformalizmowej.

Łatwy zwrot semantyczny nie powinien zatem udawać ostatecznej logiki. Powinien zachować dowody i neutralne dla zadań rozróżnienia - jednostki, wzmiarki, wydarzenia, relacje, czas, modalność, ilości, światy, pochodzenie, alternatywy i zasięg - jednocześnie umożliwiając obniżenie wyspecjalizowanych wykonawców. Niektóre zapytania można odpowiedzieć bezpośrednio na pivot za pomocą operacji relacyjnych. Inne wymagają skompilowanej formuły czasowej, systemu ograniczeń lub maszyny stanowej. Przejście jest cenne, ponieważ oddziela zrozumienie języka od każdego konkretnego solvera, zachowując jednocześnie wystarczającą ilość informacji do wykonania opartego na źródłach.

Program badawczy pre-LLM osiągnął znaczny, ale ograniczony domeną sukces. Pokazano w nim, że język naturalny można skompilować w programy wykonywalne, że interpreterzy symboliczni poprawiają systematyczność, że typy i schematy zmniejszają błędy, a słaby nadzór tworzy semantyczne skróty. Wykazano również, że szeroki zasięg i dokładna semantyka są w napięciu. LLMs znacznie poprawia zasięg i generowanie kandydatów, ale dziedziczą każdy problem strukturalny zidentyfikowany przez parsowanie semantyczne. Prawidłową syntezą nie

jest odrzucenie wcześniejszej maszyny. Jest to umieszczanie LLMs przed nim i wokół niego, z formalnymi interfejsami, które sprawiają, że stare lekcje są egzekwowalne na nową skalę.

3. Interpretacja abstrakcyjna i wykonanie symboliczne dla języka

3.1 Interpretacja abstrakcyjna jako semantyka ograniczonej niepewności

Interpretacja abstrakcyjna została opracowana jako ogólna teoria do obliczania bezpiecznych przybliżeń zachowań [Cousot and Cousot 1977] programu. Program może mieć nieskończenie wiele wejść i stanów, co uniemożliwia wyczerpujące wykonanie. Zamiast wyliczać każdy stan, analiza reprezentuje zestawy stanów poprzez abstrakcyjne właściwości, takie jak interwały, znaki, możliwe typy obiektów, osiąganie definicji lub relacje aliasowe. Operacje abstrakcyjne są zaprojektowane tak, aby każde konkretne zachowanie istotne dla nieruchomości pozostało reprezentowane. Analiza może obejmować zachowania niemożliwe, a zatem zgłaszać fałszywe alarmy, ale dźwięk z dużym przybliżeniem nie pomija w milczeniu zachowania, na które pozwala konkretna semantyka.

Pomysł operacyjny jest prosty. Konkretnie wykonanie śledzi dokładne wartości. Analiza interwałowa może śledzić, że zmienna leży między zero a dziesięć. Gdy program dodaje pięć, abstrakcyjna operacja transferu wytwarza odstęp między pięcioma a piętnaściami. W oddziale państwa są dopracowane zgodnie z warunkiem. Przy połączeniu łączy się informacja. W przypadku pętli i procesów rekurencyjnych analiza wyszukuje punkt końcowy. Poszerzenie może wymusić zbieżność poprzez poświęcenie precyzji, podczas gdy zwężenie może później odzyskać niektóre szczegóły. Ramy matematyczne dostarczają zdyscyplinowanego słownictwa dla precyzji, solidności, zbieżności i dopracowania.

Zastosowanie ram do języka wymaga konkretnej semantyki. Obiekt przybliżony nie może mieć „znaczenia” w nieokreślonym sensie. W przypadku zadania ograniczonego, domena betonowa może być zestawem struktur semantycznych dopuszczalnych w określonej gramatyce, ontologii, polityce uziemienia, rewizji źródła i teorii tła. Każda struktura zawiera możliwe podmioty, zadania zdarzeń, relacje czasowe, modalności i linki źródłowe. Dwuznaczność tworzy kilka struktur. Brakujące informacje pozostawiają wymiary nieograniczone. Sprzeczne źródła mogą generować alternatywne światy, a nie jeden niespójny związek.

Abstrakcyjny stan semantyczny podsumowuje ten zestaw. Może się okazać, że zaimek może odnosić się do jednej z dwóch osób, że zdarzenie występuje w przedziale czasowym, że obowiązek jest obecny pod jedną interpretacją i nieobecny w innej, lub że ilość leży w zasięgu. Fakt obowiązkowy utrzymuje się w każdej reprezentowanej interpretacji. Fakt może zawierać co najmniej jeden. A fakt nie może się utrzymać w żadnym. Nieznany wskazuje, że systemowi brakuje reprezentacji, zasięgu lub zasobów obliczeniowych potrzebnych do podjęcia decyzji. To czterokierunkowe rozróżnienie jest bardziej pouczające niż samo prawdopodobieństwo i bardziej uczciwe niż zmuszanie każdego pytania do prawdziwego lub fałszywego.

Rozważ dokument polityki stwierdzający: „Operator wysyła powiadomienie po walidacji. Jeśli tryb awaryjny jest aktywny, przełożony może rzec się okresu oczekiwania. ” Przypuśćmy, że nie jest jasne, czy „okres oczekiwania” odnosi się do okresu przed walidacją, czy też do okresu późniejszego. Jedno parsowanie dokonuje kruchego wyboru. Stan abstrakcyjny może zachować dwa modele czasowe. Jeśli oba oznaczają, że zawiadomienie musi nastąpić po walidacji, wniosek ten jest koniecznością. Jeśli tylko jeden zezwoli na natychmiastowe

powiadomienie, natychmiastowe wysłanie może. Reguła zgodności może określić, który werdykt zależy od niejednoznacznego zwrotu i zażądać wyjaśnienia tylko wtedy, gdy jest to konieczne.

Podstawową korzyścią nie jest to, że abstrakcyjna interpretacja w magiczny sposób rozumie język. Jest to, że daje pryncypialną semantykę egzekucji, gdy został skonstruowany ograniczony zestaw odczytów. Mówi systemowi, jak agregować alternatywy bez mylenia możliwości z koniecznością. Obsługuje również domeny modułowe. Tożsamość, czas, ilość, modalność, pochodzenie i status normatywny mogą mieć swoją własną abstrakcję, a następnie być łączone poprzez zmniejszone produkty, które wymieniają informacje tam, gdzie są przydatne.

Domena tożsamości może reprezentować klasy równoważności kandydatów z wyraźnymi, różnymi i nierozwiązanymi relacjami. Nie może scalać jednostek tylko dlatego, że nazwy się zgadzają. Domena czasowa może wykorzystywać częściowe zamówienia, interwały lub ograniczenia różnic. Domena modalności może rozróżnić twierdzoną, zgłoszoną, zamierzoną, dozwoloną, wymaganą, zabronioną, hipotetyczną i kontrfaktyczną treść. Domena ilościowa może wykorzystywać dokładne wartości, interwały, jednostki i zgodność wymiarów. Domena pochodzenia może uporządkować statusy dowodów, takich jak wyodrębnione, zaproponowane, założone, pochodne, zweryfikowane i potwierdzone przez człowieka, nie pozwalając na usunięcie ich pochodzenia. Domena zasięgu może śledzić, czy wyszukiwanie jest wystarczająco kompletne, aby poprzeć roszczenia z tytułu nieobecności.

Domeny powinny być wybierane przez sprawdzaną nieruchomość. Linter terminologiczny nie potrzebuje bogatej sieci deontycznej. Czasowa reguła nie wymaga pełnej semantyki leksykalnej, jeśli wydarzenia i daty są już zmaterializowane. Ten wybór oparty na zadaniu zmniejsza złożoność i odzwierciedla analizę statyczną opartą na zapotrzebowaniu. W długim środowisku dokumentalnym obwody lub reguły deklarują typy obserwacji i abstrakcyjne wymiary, których wymagają. Kompilator materializuje tylko te wymiary i odrzuca przebieg, gdy nie można dostarczyć domeny krytycznej.

Funkcje transferu są najtrudniejszą częścią. W zwykłej analizie programu semantyka operacji przydziału lub arytmetyki jest ustalona przez język programowania. W analizie języka naturalnego operacja aktualizująca stan semantyczny może być zaproponowana przez LLM. Zdanie może wprowadzić zdarzenie, zmodyfikować poprzedni stan, ustalić relację czasową, stworzyć wyjątek lub zgłosić czyjeś przekonanie. Jeśli uczony tłumacz bezpośrednio staje się funkcją transferu, klasyczna dźwiękowość zostaje utracona.

Defensywna konstrukcja oddziela pokolenie kandydatów od dopuszczonego transferu. LLM proponuje skończony zestaw typowanych aktualizacji semantycznych z rozpiętością źródłową i alternatywami. Deterministyczny materializator potwierdza, że cytowane dowody istnieją, pola spełniają schemat, jednostki i typy są kompatybilne, odniesienia wskazują na dopuszczalne obiekty, a żaden status nie jest po cichu uaktualniony. Następnie tłumacz abstrakcyjny stosuje zarejestrowane funkcje transferu do dopuszczonych zapisów. Otrzymana analiza jest solidna w stosunku do przyjętego zestawu kandydatów, niekoniecznie w stosunku do każdej interpretacji, którą człowiek mógłby rozpoznać. Ta warunkowość musi być określona wprost.

Istnieje kilka sposobów na wzmocnienie założenia kandydata. Wiele niezależnych monitów lub modeli może proponować alternatywy. Generatory oparte na gramatyce lub ontologii mogą zapewnić pokrycie dla kontrolowanych fragmentów. Warianty parafrazy mogą sprawdzić stabilność. Kontrastywne przykłady mogą ujawnić brakujące interpretacje. Przegląd człowieka

może koncentrować się na niejasnościach związanych z wynikami. Kompilator może celowo przegenerować się w ramach ograniczonej gramatyki i pozwolić późniejszym ograniczeniom wyeliminować niemożliwych kandydatów. Każda metoda handluje kosztem zaufania, że zestaw zawiera wszystkie istotne prawdopodobne odczyty.

Interpretacja abstrakcyjna jest również istotna dla propagacji reguł w skali dokumentów. Wiele problemów związanych z kłyczeniem i zgodnością przypomina analizę przepływu danych. Fakty są generowane w lokalizacjach źródłowych, propagowane przez zakresy, łączone w różnych sekcjach i unieważniane przez późniejsze oświadczenia. Definicja może być dostępna po jej deklaracji, ale nie wcześniej. Obiekt może zmienić stan w całej narracji. Wymóg może mieć zastosowanie w ramach jednej jurysdykcji lub wersji. Roszczenie może uzyskać dowody później. Przyrostowy silnik fixpoint może obliczać konsekwencje i aktualizować tylko dotknięte regiony, gdy dokument się zmieni.

Ta perspektywa pozwala uniknąć ponownego uruchomienia ogólnego LLM całego dokumentu po każdej edycji. Stabilne artefakty strukturalne i semantyczne można buforować według trawienia treści. Zmieniony akapit unieważnia obserwacje lokalne, a niższe stany abstrakcyjne, które są od nich zależne. Wykres analizy określa, które obwody wymagają powtórki. Jest to analogiczne do przyrostowych ram kompilacji i analizy statycznej. Uściśla również wyjaśnienia: system może pokazać, które zmiany źródła zmieniły, który abstrakcyjny niezmiennik i dlaczego.

Poszerzenie i utrata precyzji mają analogi semantyczne. Długa narracja może zawierać setki możliwych tożsamości obiektowych lub stosunków czasowych. Zatrzymanie każdej kombinacji jest niepraktyczne. System może podsumować „jeden z tych obiektów”, „jakiś czas w tym odstępie czasu” lub „jedna z tych jurysdykcji”. Takie poszerzenie może wprowadzić fałszywe ostrzeżenia. Zawężenie lub lokalne udoskonalenie może odzyskać precyzję wokół podejrzenia naruszenia. Celem projektu nie jest unikanie abstrakcji, ale zapewnienie, że jego precyzja jest widoczna i odwracalna.

Ustanowiona literatura NLP na temat certyfikowanej solidności wykorzystuje pomysły abstrakcyjno-interpretacyjne w inny sposób. Jia i współpracownicy wykorzystali propagację przedziałową do szkolenia modeli, których przewidywania są certyfikowane w stosunku do każdego zastąpienia słowa w predefiniowanej rodzinie, zgłaszając 75-procentową dokładność kontrydiktoryjną na IMDB i SNLI pod ich modelem zagrożenia [Jia et al. 2019]. Późniejsze prace wykorzystywały randomizowane osadzanie, świadome solidności i specjalistyczne domeny Transformer w celu poprawy certyfikowanych granic [Ye et al. 2020; Wang et al. 2023; Huang et al. 2026]. Są to autentyczne gwarancje, ale domena betonowa jest zestawem zaburzonych wejść modelowych, a nie zestawem interpretacji semantycznych. Rodzina transformacji jest definiowana zewnętrznie i zakłada się, że zachowuje etykietę.

To rozróżnienie ma znaczenie. Poświadczenie, że przewidywanie klasyfikatora jest niezmiennie pod wszystkimi podmianami na liście synonimów, nie dowodzi, że każda podstawienie zachowuje znaczenie zdania w kontekście. Nie dowodzi, że klasyfikator jest prawidłowy na oryginalnym zdaniu. Poświadcza solidność względem formalnego zestawu perturbacji. Ta literatura jest nadal bardzo aktualna, ponieważ pokazuje, w jaki sposób domeny abstrakcyjne mogą skalować się do kombinatorskich danych wejściowych języka i jak precyzja weryfikacji zależy od przedstawienia korelacji. Zapewnia również ostrzeżenie metodologiczne: najsilniejsze twierdzenie formalne jest zawsze w stosunku do zestawu betonowego faktycznie zdefiniowanego.

Obiecującym kierunkiem badawczym jest połączenie obu zastosowań. Certyfikowana analiza neuronowa może wiązać zachowanie modelu propozycji semantycznej pod kontrolowanymi parafrazami, podczas gdy abstrakcyjna interpretacja semantyczna może wiązać konsekwencje alternatywnych interpretacji uziemionych. Ten pierwszy pyta, czy ekstrakcja modelu zmienia się w wyniku dozwolonych przekształceń powierzchni. Ten ostatni pyta, czy wniosek na poziomie dokumentu zmienia się w dopuszczalnych strukturach semantycznych. Razem mogli określić, czy niepewność pochodzi z niestabilności modelu lub prawdziwej dwuznaczności semantycznej.

LLMs może również pomóc w konstruowaniu abstrakcyjnych interpretatorów, ale tylko pod akceptacją mechaniczną. SAIL demonstruje tę zasadę, wykorzystując LLMs do wyszukiwania abstrakcyjnych transformatorów, jednocześnie egzekwując ograniczenia składniowe i semantyczne oraz optymalizując koszt, który mierzy nierozsądek. Zaakceptowane transformatory są globalnie solidne, a system może syntetyzować precyzyjne transformatory dla operatorów nieliniowych, które wcześniej nie były dostępne [Gu et al. 2026]. Szersza lekcja jest architektoniczna: modele generatywne mogą zbadać ogromną przestrzeń projektową, jeśli mały formalny walidator kontroluje przyjęcie.

Bezpośrednie poproszenie LLM o wykonanie abstrakcyjnej interpretacji jest mniej wiarygodne. Eksperymenty na benchmarkach analizy programu pokazują, że modele mogą podążać za stylami interpretacji abstrakcyjnej, ale popełniać błędy logiczne na złożonych strukturach i stanach z Halucynizacji [Mitchell et al. 2025]. Wynik ten powinien utemperować propozycje, w których model opowiada o niezmienniku, a narracja jest akceptowana jako analiza. Model jest cenny do sugerowania niezmienników, partycji domeny lub predykatów udoskonalenia. Obliczenia punktu stałego i kontrole solidności powinny pozostać symboliczne.

Dla wykonalnego języka naturalnego abstrakcyjna interpretacja nie jest więc ani metaforą, ani kompletnym rozwiązaniem. Jest to teoria zdyscyplinowanego przybliżenia po zdefiniowaniu przez system, co się liczy jako możliwa interpretacja. Obsługuje może / musi semantyka, stopniowa propagacja, skład domeny, konserwatywne rozumowanie nieobecności i ukierunkowane dopracowanie. Jego gwarancja jest uzależniona od pokrycia semantycznego. Ograniczenie to nie jest śmiertelne; jest to dokładny punkt, w którym interpretacja wspomagana modelem musi być oceniana i regulowana.

3.2 Wykonanie symboliczne, rozwiązywanie ograniczeń, sprawdzanie modeli i doprecyzowanie semantyczne

Klasyczne wykonanie symboliczne wykonuje program z wejściami symbolicznymi, a nie określone wartości [King 1976]. Stan symboliczny zawiera lokalizację kontrolną, wyrażenia symboliczne dla zmiennych programu oraz warunek ścieżki opisujący, które wejścia docierają do tego stanu. W gałęzi, widelce egzekucyjne. Jedna ścieżka dodaje warunek oddziału; druga dodaje jej negację. Rozwiązywacz ograniczeń określa, czy warunki są spełnione i mogą wytwarzać konkretne wejścia, które realizują wykonalną ścieżkę. Metoda jest potężna, ponieważ wytwarza świadków: nie tylko ostrzeżenie, że błąd może istnieć, ale wejście i ścieżka, które go demonstrują.

Trudność polega na wybuchu ścieżki. Gałęzie mnożą się, pętle i rekurencja tworzą nieograniczone ścieżki, złożone teorie rzucają wyzwanie solverom, a zewnętrzne biblioteki są trudne do modelowania. Badania wykonania symbolicznego opisują dużą przestrzeń projektową strategii wyszukiwania, łączenia stanu, podsumowań składu, jednokołowego wykonania, modeli środowiska i optymalizacji [Baldoni et al. 2018] solvera. Żaden pojedynczy silnik nie bada każdego zachowania prawdziwego oprogramowania. Oświadczenia o symbolicznym wykonaniu obejmują zatem granice, założenia modelowania i ograniczenia pokrycia.

Język naturalny łączy się z symbolicznym wykonaniem na kilka różnych sposobów. Najprostszym jest język do kodu: LLM generuje program, a konwencjonalny symboliczny wykonawca analizuje ten program. Innym jest język do specyfikacji: model generuje właściwość lub ograniczenie, które jest sprawdzane pod kątem istniejącego kodu. Trzeci to wykonanie programu semantycznego: dokument jest kompilowany w maszynie stanowej lub programie reguł, którego alternatywne interpretacje stają się symbolicznymi wyborami. Czwartą jest eksploracja wspomagana LLM: model priorytetyzuje ścieżki, proponuje podsumowania lub powody dotyczące ograniczeń, których konwencjonalne solvery nie mogą łatwo wyrazić.

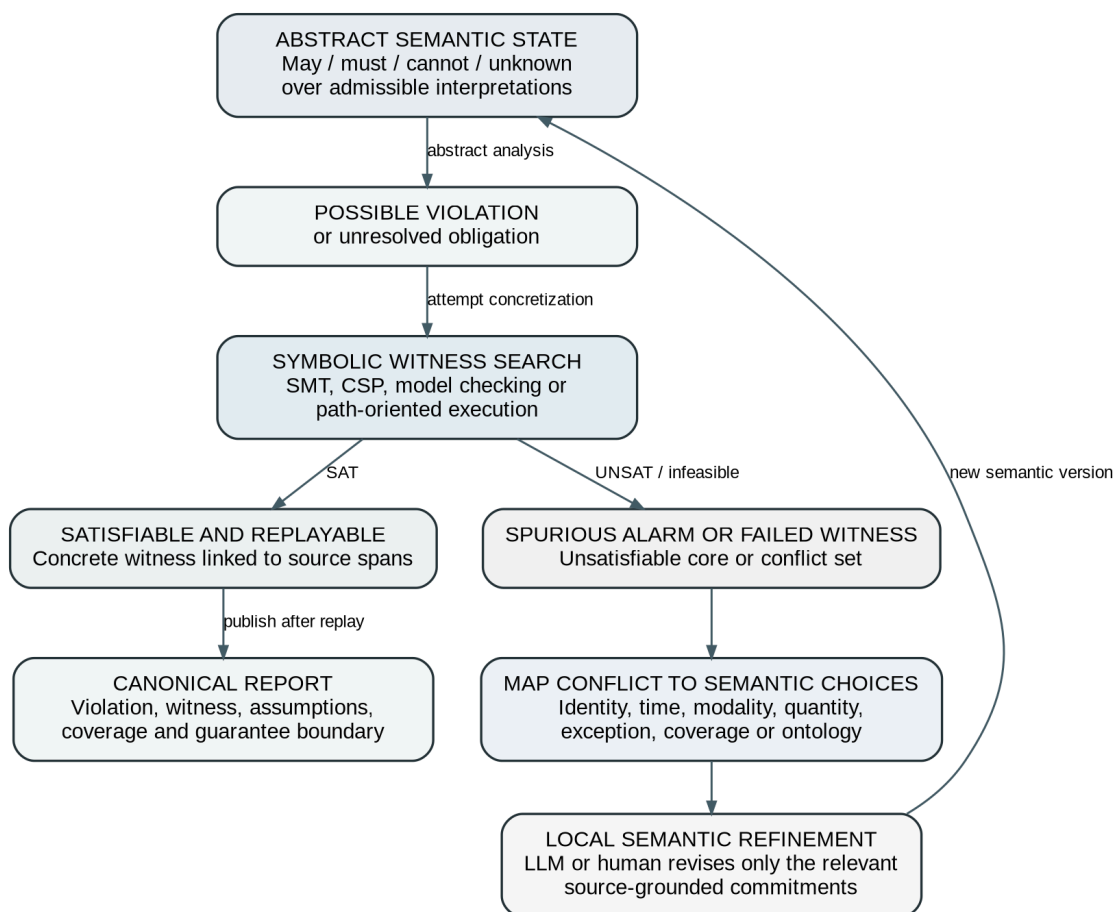
Terminologia musi pozostać precyzyjna. Wykonanie SQL, Pythona lub formularza logicznego wygenerowanego z pytania jest formalnym wykonaniem, ale niekoniecznie jest to wykonanie symboliczne w sensie wrażliwym na ścieżkę. Różnica ma znaczenie, ponieważ wykonanie symboliczne oferuje warunki ścieżki i konkretnych świadków, podczas gdy zwykłe wykonanie zapewnia jeden wynik dla jednego konkretnego stanu. Obie są użyteczne. Badanie, które grupuje ich razem bez rozróżnienia, sprawia, że gwarancje wydają się silniejsze niż są.

SymGPT zawiera jeden z najczystszych przykładów języka do specyfikacji. System bada 132 zasady języka naturalnego z trzech ERC standardów, tłumaczy je z LLM na ograniczoną gramatykę, syntetyzuje ograniczenia, które opisują naruszenia i wykorzystuje symboliczne wykonanie do wyszukiwania kontraktów Ethereum. Autorzy zgłaszają 5,783 naruszeń w 4,000 kontraktów, w tym 1,375 z wyraźnymi ścieżkami ataku na kradzież, i zgłaszają silniejszą wydajność niż sześć zautomatyzowanych technik i usług audytu ekspertów ds. Bezpieczeństwa [Xia et al. 2026]. Siła projektu polega na wąskiej gramatyce, systematycznej syntezie ograniczeń i konkretnych ścieżkach programowych. Jego pozostałe ryzyko semantyczne polega na tym, czy każda ERC reguła została przetłumaczona wiernie.

Ten wzór uogólnia. Polityka może być opracowana w warunkach wstępnych, niedozwolonych przejść i wyjątków. Procedura może być skompilowana w stany i działania. Reguła ciągłości narracyjnej może modelować obiekty i zdarzenia, a następnie wyszukiwać ścieżkę, w której obiekt jest używany po pozostawieniu bez wyszukiwania. Protokół kliniczny

może modelować sekwencje kwalifikowalności i interwencji. Naukowy przepływ pracy może modelować transformacje danych i wymagania dotyczące pochodzenia. W każdym przypadku symboliczny wykonawca powinien działać na typowanym programie, a nie na surowej prozie.

Symboliczne wykonanie jest szczególnie przydatne dla kontrprzykładów. Analiza abstrakcyjna może wskazywać, że naruszenie jest możliwe, ponieważ utraciło korelację. Poszukiwanie ścieżki może próbować wybrać konkretne tożsamości, czasy, warunki i wyjątki, które zdają sobie sprawę z naruszenia. Jeśli ścieżka jest zadowalająca, świadek może być oddany jako scenariusz oparty na źródle. Jeśli jest to niesatysfakcjonujące, system dowiadyuje się, że abstrakcyjne ostrzeżenie było fałszywe pod obecną semantyką.



Rysunek 2. Dopracowanie semantyczne z kontrprzykładowym naprowadzaniem. Analiza abstrakcyjna identyfikuje możliwe naruszenie; symboliczne przeszukiwanie świadka albo potwierdza przypadek wielokrotnego odtwarzania, albo ujawnia rozróżnienia semantyczne, które muszą zostać dopracowane.

Prowadzi to do kontrprzykładowego dopracowania semantycznego. Tradycyjna CEGAR zaczyna się od gruboziarnistej abstrakcji programu. Kontrola modelu znajduje abstrakcyjny kontrprzykład. Kontrola wykonalności określa, czy kontrprzykład odpowiada betonowemu wykonaniu. Jeśli nie, dodawane są predykaty w celu dopracowania abstrakcji, a analiza się powtarza. W ustawieniu językowym abstrakcja obejmuje wybory semantyczne. Może powstać fałszywy kontrprzykład, ponieważ dwie wzmianki zostały połączone, wyjątek został zignorowany, rozszerzono interwał czasowy lub potraktowano stwierdzenie modalne jako fakt.

Semantyczna pętla udoskonalenia powinna zachować symboliczne wyjaśnienie awarii. Załóżmy, że ostrzeżenie o zgodności zależy od założenia, że „tablica” w dwóch sekcjach oznacza ten sam organ. Symboliczna powtórka pokazuje, że ścieżka staje się niewykonalna, jeśli byłyby odrębne. System mapuje tożsamość zgodnie z odpowiednimi wzmiankami i zadaje

ukierunkowane pytanie: czy te odniesienia mają na celu określenie tej samej tablicy? LLM może ponownie zbadać kontekst lokalny i zaproponować alternatywy; człowiek może orzekać, jeśli to konieczne. Reszta modelu dokumentu pozostaje bez zmian. Jest to bardziej niezawodne i wydajne niż proszenie modelu o ponowne przeczytanie wszystkiego i „spróbowanie ponownie”.

Niesatysfakcjonujące rdzenie zapewniają powiązany mechanizm. Gdy zestaw ograniczeń jest niespójny, solver często może zwrócić podzbiór odpowiedzialny za konflikt. Mapowanie rdzenia z powrotem do rozpiętości źródła identyfikuje minimalny zestaw oświadczeń i interpretacji, które nie mogą współistnieć. W przeglądzie polityki może to ujawnić sprzeczne obowiązki. W analizowaniu semantycznym może ujawnić, że kwantyfikator, łącznie tożsamości i tymczasowe założenie wspólnie tworzą niemożliwość. Użytkownik otrzymuje wyjaśnienie oparte na konflikcie formalnym, a nie ogólną krytykę modelu.

Nie każdy problem najlepiej przedstawiać jako wykonanie ścieżki. Kontrola modelu analizuje systemy przejściowe pod kątem właściwości czasowych. Liniowa logika czasowa może wyrazić, że coś w końcu się dzieje, zawsze trzyma lub występuje przed innym wydarzeniem. Logika obliczeniowo-drzewna i probabilistyczna wyrażają rozgałęzienie lub niepewne zachowanie. Systemy logiczne z języka naturalnego do czasowego, takie jak NL2TL i nl2spec, pokazują, że LLMs może konstruować specyfikacje czasowe, gdy słownictwo i domena są kontrolowane [Chen et al. 2023; Cosler et al. 2023]. Wykonawcą jest wtedy sprawdzanie modelu lub planer, a nie LLM.

Rozwiązywanie ograniczeń obejmuje szeroki środek. SAT reprezentuje wybory logiczne. SMT dodaje teorie takie jak arytmetyka, tablice, ciągi i funkcje nieinterpretowane. CSP i CP-SAT obsługują domeny, harmonogramowanie i optymalizację. MaxSAT i miękkie ograniczenia wyrażają preferencje. Wiele zadań w języku naturalnym - harmonogramowanie, alokacja zasobów, sprawdzanie spójności, relacje rodzinne, zagadki i kombinacje polityk - jest lepiej zestawionych w ograniczenia niż w kod proceduralny. SatLM pokazał, że specyfikacje deklaratywne plus solver może przewyższać podejścia wspomagane programem w zakresie twardych zadań arytmetycznych i logicznych [Ye et al. 2023]. Wynik obsługuje centralny wybór architektoniczny: LLM powinien opisać problem, podczas gdy solver wykonuje wyszukiwanie.

Formalne planowanie zapewnia semantykę działania i wyszukiwanie ukierunkowane na cel. Ostatnie stanowiska pracy z LLMs jako formalizatory planowania, a nie planiści. Hao i współpracownicy przekładają ograniczenia planowania w świecie rzeczywistym na formalne narzędzia weryfikacji, używają niesatysfakcjonujących rdzeni do naprawy i zgłaszają 93,9-procentowy sukces w TravelPlannerrr w swoim otoczeniu, w porównaniu ze znacznie niższymi bazami z modelami bezpośrednimi [Hao et al. 2025]. Badanie z 2025 r. stwierdza, że główną szansą nie jest zastąpienie planistów, ale wykorzystanie LLMs do skonstruowania i udoskonalenia modeli planowania [Tantakoun et al. 2025]. Właśnie za tym opowiada się podział.

Silniki przepływu danych i Datalog zapewniają wykonanie fixpoint, a nie wyszukiwanie ścieżki. Są one przydatne do monotonicznych konsekwencji i propagacji w całym dokumencie. Silniki Automata i pattern obsługują uporządkowane sekwencje zdarzeń. Tabele decyzyjne obsługują skończone polityki. Architektura powinna kierować każdy problem do najprostszego formalizmu, który go wyraża. Symboliczne wykonanie nie powinno stać się uniwersalną etykietą dla każdego symbolicznego komponentu.

LLMs może pomóc w tradycyjnym wykonaniu symbolicznym w bezpieczniejszy i bardziej ryzykowny sposób. Bezpieczniejszą rolą jest priorytetyzacja ścieżki. Model może zidentyfikować

regiony kodu, które mogą zawierać luki w zabezpieczeniach lub ścieżki, które mogą osiągnąć cel, podczas KLEE lub innego silnika, zachowuje ostateczną decyzję o wykonalności. Ostatnie prace, takie jak KLECOpilót, informują o znacznych ulepszeniach w zakresie zasięgu z wyszukiwaniem z przewodnikiem LLM, chociaż status preprintu i czułość modelu wymagają ostrożnej interpretacji [Chen et al. 2026]. Zasada jest solidna: wyuczone heurystyki decydują, gdzie szukać, a nie czy ścieżka jest ważna.

Bardziej ryzykowną rolą jest zastąpienie solvera. AutoBug, przedstawiony przez jego autorów jako wielkojęzyczne modelowe wykonanie symboliczne, rozkłada kod na zadania rozumowania oparte na ścieżce i prosi model o bezpośrednie uzasadnienie ich bez tłumaczenia wszystkiego na SMT. Podejście to jest lekkie i agnostyczne językowo, a zgłoszone eksperymenty poprawiają analizę LLM z programem OPARTĄ na [Li et al. 2025]. Autorzy określają ją jako przybliżoną. Może analizować konstrukcje, które są trudne dla konwencjonalnych solverów, ale nie zapewniają klasycznej solidności. W przypadku zastosowania o wysokiej pewności taki model jest lepiej traktowany jako jeden silnik w portfolio, którego wyniki wymagają testów, powtórki lub niezależnego potwierdzenia.

LLMs może również generować testy, podsumowania lub modele dla zewnętrznych interfejsów API. Badania donoszą o przydatnym zasięgu programów, z którymi tradycyjne narzędzia mają trudności, ale wydajność spada z długimi ścieżkami, złożonymi interfejsami API, rekurencją i niewykonalną klasyfikacją ścieżek. Metody te są obiecujące, ponieważ środowiska programowe zawierają semantykę, która jest kosztowna do aksjomatyzacji. Ich status wyniku powinien pozostać „kandydatem” do czasu potwierdzenia przez konkretną ścieżkę lub udowodnienie stanu przez rozwiązywacz.

Bezpieczeństwo jest poważnym problemem. Zasady języka naturalnego i dokumenty źródłowe są niezaufanymi danymi wejściowymi. Szkodliwy dokument może próbować wyświetlić wstrzyknięcie, sfabrykować referencje lub wykorzystać tłumacza generującego kod. Formalne artefakty powinny być danymi, a nie nieograniczonym kodem wykonywalnym. Gdy wymagane są niestandardowe algorytmy, należy je przejrzeć rozszerzenia hosta z zablokowanymi przetwornikami, wpisanymi wejściami i wyjściami, limitami zasobów i brakiem autorytetu otoczenia. Wezwania Solvera potrzebują budżetu na czas i pamięć. Świadek powinien zawierać dokładną wersję teorii, kompilatora i silnika, który go wytworzył.

Połączony wzór analizy jest teraz jasny. Gruboziarnista analiza semantyczna i przepływ danych identyfikuje możliwe problemy w długim dokumencie. Specjalistyczne silniki formalne testują tylko odpowiednie plasterki. Symboliczna egzekucja konstruuje świadków roszczeń zależnych od ścieżki. Kontrola modelu zajmuje się zachowaniami czasowymi. Rozwiązywacze ograniczeń zajmują się wyborami kombinatorycznymi. Nieudani świadkowie i rdzenie solvera napędzają lokalne wyrażenie semantyczne. Każdy przyjęty wniosek jest zwracany ze statusem i pochodzeniem, które odzwierciedla najsłabsze ogniwo w łańcuchu.

Ta architektura nie obiecuje całkowitej weryfikacji nieograniczonego języka. Obiecuje coś bardziej operacyjnego: każde formalne roszczenie ma znaczenie wykonalne; każdy symbol ma źródło lub deklarowane założenie; każdy kontrprzykład może zostać powtórzony; każde twierdzenie o nieobecności jest związane z pokryciem; a każda pozostała dwuznaczność jest reprezentowana, a nie ukryta. Interpretacja abstrakcyjna zapewnia skalę i konserwatywną agregację. Symboliczne wykonanie zapewnia precyzję i świadków. Ich połączenie jest jednym z najbardziej prawdopodobnych podstaw do wykonywania języka naturalnego.

4. Co naprawdę osiągnęła era LLM

4.1 Autoformalizacja, rozumowanie wspomagane programami, logika, specyfikacje temporalne i planowanie

Pojawienie się dużych modeli językowych zmieniło praktyczną granicę kompilacji semantycznej. Wcześniejsze systemy wymagały gramatyki specyficznej dla zadania, nadzorowanych form logicznych lub starannie zaprojektowanych leksykonów. LLMs może zaproponować formalne artefakty z otwartych opisów z niewielkim lub żadnym szkoleniem specyficznym dla zadania. To znacznie rozszerza zasięg, ale także stwarza pokusę, aby wywnioskować więcej od zysków porównawczych niż uzasadniają mechanizmy. Istotne pytanie nie jest tylko to, czy metoda poprawia dokładność odpowiedzi. Jest to, które błędy zostały przeniesione do komponentu symbolicznego, które błędy pozostają w formalizacji i jakie niezależne dowody istnieją, że formalny artefakt reprezentuje źródło.

Programowe modele językowe wspomagane lub PAL zapewniają czysty punkt wyjścia. LLM odczytuje problem, generuje program Python i deleguje obliczenia do interpretera [Gao et al. 2022]. W GSM8K zgłoszony system znacznie się poprawił w ramach szybkiego działania. Mechanizm jest przekonujący, ponieważ modele językowe są często lepsze w rozkładaniu problemu niż w prowadzeniu długiej arytmetyki. Gdy rozkład jest prawidłowy, Python dostarcza deterministyczne operacje i stan.

Ograniczenie PAL jest równie pouczające. Tłumacz poświadcza jedynie, że wykonał wygenerowany program. Nie poświadcza, że program wiernie modeluje pytanie w języku naturalnym. Błędna zmienna, pominięty warunek lub niewłaściwy wzór może działać bez błędu. Sukces systemu zależy zatem od dystrybucji, w której programy są krótkie, przykłady zdecydowanie ograniczają strukturę, a błędy wykonawcze ujawniają wiele błędów. W przypadku rozumowania dokumentów, analizy polityki i interpretacji naukowej odwzorowanie jest mniej ograniczone, a tłumacz zapewnia słabsze informacje zwrotne.

SatLM zastępuje kod proceduralny ograniczeniami deklaratywnymi [Ye et al. 2023]. Model określa zmienne i relacje, a silnik satysfakcji szuka rozwiązania. Może to być łatwiejsze niż generowanie algorytmu, ponieważ model nie musi decydować o kolejności operacji. Reprezentacje deklaratywne umożliwiają również wiele modeli, dowodów niewykonalności i debugowanie na poziomie ograniczeń. Zgłoszone zyski w zadaniach hard thinking wspierają ideę, że wybór solvera powinien podążać za strukturą problemu. Harmonogram jest naturalnie problemem ograniczającym; dynamiczny proces może być maszyną stanową; arytmetyczny problem słowny może być programem lub ograniczeniami.

Logic-LM formalizuje problemy logiczne w języku naturalnym i wywołuje symboliczne solvery [Pan et al. 2023]. Wykorzystuje również błędy solvera, aby uzyskać monit samodopracowania. W kilku zbiorach danych autorzy informują o dużych średnich ulepszeniach w stosunku do standardowych i łańcuchów myślenia. Architektura pokazuje dwie użyteczne zasady. Po pierwsze, symboliczny silnik usuwa niekonsekwencję dedukcyjną po istnieniu ważnej formalizacji. Po drugie, formalne komunikaty o błędach zapewniają lepszy sygnał naprawczy niż ogólne żądanie ponownego rozpatrzenia.

Jednak naprawa składni nie jest naprawą semantyczną. Formuła może być dobrze uformowana, zadowalająca i błędna. Potok Logic-LM może skorygować nieprawidłowe

predykaty lub awarie solvera, ale może nie wykryć, że „niektóre” stały się „wszystkimi” lub że wyjątek został pominięty. Późniejsze prace, takie jak Logic-LM++, dodają wieloetapowe udoskonalenie, a LTRAG pobiera przykłady formalizacji kierowanej myślami, zgłaszając ulepszenia w stosunku do Logic-LM i LINC na FOLIO i AR-LSAT [Kirtania et al. 2024; Hu et al. 2025]. Zyski te sugerują, że sama formalizacja korzysta z przykładów i ustrukturyzowanej krytyki. Nie eliminują potrzeby bezpośredniej oceny wierności semantycznej.

LINC sprawia, że modułowość jest wyraźna: LLM służy jako parser semantyczny, logika pierwszego rzędu reprezentuje problem, a twierdzenie wykonuje odliczenie [Olausson et al. 2023]. W ProofWriter, zgłaszana kombinacja wytworzyła duże zyski w porównaniu z systemami łańcucha myśli, w tym modelami znacznie większymi niż parser. Analiza błędów wykazała uzupełniające się tryby awarii: system symboliczny pozwala uniknąć wielu błędów rozumowania, podczas gdy niepowodzenia w formalizacji pozostają. To są dokładnie dowody oczekiwane od architektury federacyjnej. System jest najsilniejszy, gdy zadanie w języku naturalnym odpowiada już zwartej teorii pierwszego rzędu.

FOLIO stanowi użyteczny punkt odniesienia, ponieważ zawiera oparte na człowieku problemy z rozumowaniem w języku naturalnym w połączeniu ze zweryfikowaną logiką pierwszego rzędu [Han et al. 2024]. Ocenia zarówno rozumowanie, jak i tłumaczenie NL-to-FOL. Zbiór danych pozostaje wyzwaniem nawet dla silnych modeli, co podcina ideę, że formalizacja pierwszego rzędu jest rozwiązaniem krokiem wstępnym przetwarzania. FOLIO ilustruje również znaczenie złożoności autorstwa człowieka. Syntetyczne zestawy danych reguł mogą sprawić, że język będzie niezwykle regularny i pozwoli modelom na wykorzystanie szablonów. Przykłady ludzkie wprowadzają zmienność leksykalną, wiedzę światową i bardziej realistyczne wybory zakresu.

FoVer przyjmuje inne podejście, weryfikując rozumowanie w języku naturalnym poprzez logikę pierwszego rzędu [Pei et al. 2025]. Zamiast traktować płynny tekst rozumowania jako autoryzujący, konstruuje formalne obowiązki i sprawdza je. Jest to obiecujący wzór dla pokolenia ograniczonego dowodowo: LLM może zaproponować argument, ale weryfikator akceptuje tylko inferencyjne kroki, które odpowiadają ważnym formalnym konsekwencjom. Pozostałą kwestią jest wyrównanie między każdym zdaniem, a formalne predykaty używane do jego sprawdzania.

Badania autoformalizacji coraz częściej traktują tłumaczenie jako odrębną dyscyplinę, a nie szybką sztuczkę. Ankiety opublikowane w 2025 roku w przekładzie katalogowym na temat twierdzeń, języków logicznych i specyfikacji formalnych, podkreślając potrzebę pętli sprzężenia zwrotnego, wyszukiwania, reprezentacji pośrednich i wskaźników porównawczych, które oddzielają składnię od semantyki [Weng et al. 2025]. Bezpośrednie NL-to-FOL prace wskazują znaczący postęp, ale także systematyczne błędy w wiedzy niejawnej, zakresie kwantyfikatorów i projektowaniu ontologii [Yang et al. 2024; Lalwani et al. 2025]. Dodanie ukrytej wiedzy w tle może poprawić wydajność zadania, jednocześnie zwiększając ryzyko, że system formalny okaże się przesłanką nie licencjonowaną przez źródło.

Traktowanie wiedzy podstawowej jest decydującym wyborem architektonicznym. W benchmarku łamigłówek można w domyśle oczekiwać przesłanek zdrowego rozsądku. W weryfikacji prawnej lub naukowej wprowadzenie nieokreślonych przesłanek może unieważnić wynik. Rygorystyczny system powinien wyróżniać pomieszczenia źródłowe, lokale typu authority-pack, aksjomaty domeny i założenia sugerowane modelowo. Rozwiązywacz może rozsądzić wszystkie z nich, ale ostateczne wyjaśnienie musi pokazać, które wnioski zależą od

tego, który autorytet. Jeśli założenie jest niezbędne i niezwyfikowane, wynik powinien być warunkowy.

Tłumaczenie czasowo-logiczne stawia różne wyzwania. NL2TL wykorzystuje podniesione reprezentacje, aby nauczyć się struktury czasowej w różnych dziedzinach i raportuje silną adaptację przy stosunkowo niewielkich danych [Chen et al. 2023] do domeny docelowej. Strategia podnoszenia ukrywa konkretne propozycje atomowe, jednocześnie ucząc się wzorców logicznych wielokrotnego użytku. Jest to cenny wgląd w projekt: oddziel strukturę „zawsze przed”, „w końcu po”, lub „do” od specyficznego dla domeny uziemienia wydarzeń.

nl2spec kładzie nacisk na interakcję [Cosler et al. 2023]. System mapuje części wymogu języka naturalnego do części wygenerowanej formuły czasowej, umożliwiając użytkownikom naprawę lokalnych błędnych tłumaczeń. Interfejs ten potwierdza, że niejednoznaczność nie zawsze może być rozwiązana automatycznie. Sugeruje również, jak powinno działać dopracowanie semantyczne oparte na LLM: użytkownik powinien zobaczyć decyzję językową, a nie tylko szereg operatorów czasowych.

Uziemienie pozostaje kruchym punktem. Formuła czasowa może mieć prawidłowy kształt, podczas gdy jego twierdzenia atomowe odnoszą się do niewłaściwych zdarzeń lub obiektów. Prace nad uziemioną równoważnością logiczną potwierdzają, że wyraźne mechanizmy uziemienia poprawiają poprawność, wzmacniając rozróżnienie między składnią wzoru a przywiązaniem semantycznym. W przypadku środowiska wykonawczego w języku wykonywalnym atomy czasowe powinny być odniesieniami do typowanych obserwacji z rozpiętościami i tożsamościami, a nie etykietami o swobodnym kształcie wymyślonymi przez tłumacza.

Zautomatyzowane planowanie zapewnia szczególnie dojrzałe symboliczne zaplecze. Języki planowania definiują typy, predykaty, działania, warunki wstępne, efekty, stany początkowe i cele. Następnie planiści szukają poprawnej sekwencji akcji. LLMs są słabymi planistami o długim poziomie horyzontu, gdy muszą utrzymywać stan i ograniczenia wewnętrznie, ale mogą być skuteczne w sformalizowaniu problemu dla planisty. W badaniu z 2025 r. zidentyfikowano budowę modelu i dopracowanie jako centralną rolę LLMs w tym obszarze [Tantakoun et al. 2025].

Hao i współpracownicy demonstrują rygorystyczną wersję wzoru na temat planowania [Hao et al. 2025] w świecie rzeczywistym. System formalizuje ograniczenia, wywołuje narzędzia weryfikacji i wykorzystuje niezaspokojone rdzenie do naprawy problemów. Autorzy informują o 93,9-procentowym sukcesie TravelPlannerrr w ich warunkach eksperymentalnych, znacznie powyżej bezpośrednich LLM linii bazowych, z którymi się porównują. Rezultat jest ważny, ponieważ pokazuje, że narzędzia formalne mogą przekształcić niezawodność planowania, gdy problem jest reprezentowany. Nie oznacza to, że LLM stał się sound plannerem. Oznacza to, że planer i weryfikator przenoszą stan długiego horyzontu, podczas gdy LLM konstruuje i naprawia formalną instancję.

Planowanie obnaża również ukryte ograniczenia. Wnioski w języku naturalnym obejmują zdroworozsądkowe oczekiwania, preferencje i warunki wykonalności, które mogą nie być określone jako wymogi formalne. Systemy takie jak LRPlan próbują współpracować między modelami językowymi i rozumowania w celu uzyskania wyraźnych i dorozumianych ograniczeń, podczas gdy benchmarki, takie jak PlaningArena, pokazują, że nawet silne modele zmagają się z wyborem narzędzi, pamięcią kontekstową i spójnością [Karthikeyan et al. 2025; Zheng et al. 2025] logiczną. System wysokiej gwarancji musi albo sformalizować ukryte ograniczenia z

zaufanego źródła, albo oznaczyć je jako założenia. „Zdrowotne” nie jest wystarczającą kategorią pochodzenia.

RuleArena i powiązane benchmarki testują LLMs na tle realistycznych systemów reguł i stwierdzają, że augmentacja narzędzi pomaga, ale nie sprawia, że formalizacja jest błaha. SemEval 2026 zadania dotyczące treści i formalnego rozumowania dodatkowo pokazują, że na modele mają wpływ treści semantyczne nawet wtedy, gdy formularz logiczny powinien określać odpowiedź. Modułowe wpisy neuro-symboliczne wykorzystują FOL i Z3 do redukcji stroniczości treści, ale formalne tłumaczenie i hałas wielojęzyczny pozostają trudne [Grimaldi 2026; Kartáč et al. 2026; Wang et al. 2026]. Wyniki te wzmacniają potrzebę odłączenia wykładni językowej od wniosku formalnego.

Jedną z najbardziej znaczących obserwacji w tej literaturze jest to, że skuteczne systemy używają specyficznych dla celu języków pośrednich. PAL używa Pythona. SatLM wykorzystuje ograniczenia deklaratywne. Logic-LM i LINC używają logiki. NL2TL wykorzystuje logikę czasową. Systemy planowania wykorzystują modele podobne do PDDL. SymGPT wykorzystuje gramatykę dostosowaną do reguł ERC. Pole nie odkryło uniwersalnej formalnej reprezentacji języka naturalnego. Odkryto, że LLMs może kierować język do wielu formalnych ekosystemów, gdy dostępne są przykłady, schematy i informacje zwrotne.

Obsługuje to architekturę portfela, a nie uniwersalny „językowy kod bajtowy”. Semantyczny pivot może zachować dowody, podmioty, zdarzenia, zakres i modalność, podczas gdy selektor formalizmu wybiera cel wykonania. Sam selektor może być wspomagany przez LLM, ale jego decyzja powinna zostać zwalidowana w stosunku do kontraktów zdolności. Formalizm jest zgodny tylko wtedy, gdy może wyrazić wymagane rozróżnienia i istnieją niezbędne uziemione obserwacje. Planer nie powinien być wybierany do zapytania czysto taksonomicznego; logika pierwszego rzędu nie powinna po cichu przybliżać niepewnej dynamiki czasowej; prosty silnik relacyjny nie powinien twierdzić, że decyduje o możliwych do krycia wyjątkach prawnych.

Dowody empiryczne potwierdzają zatem węższe, ale istotne twierdzenie. LLMs może radykalnie usprawnić budowę obiektów formalnych wykonywalnych. Zewnętrzne systemy symboliczne mogą poprawić poprawność, spójność i interpretowalność. Informacje zwrotne Solver mogą prowadzić naprawę. Najmocniejsze wyniki pojawiają się w ograniczonych domenach z językami wyraźnymi, wyraźnymi schematami i możliwymi do oceny odpowiedziami. Nierozwiązanym wyzwaniem jest kompilacja semantyczna: zapewnienie, że obiekt formalny zachowuje istotne znaczenie źródła i że każda przesłanka jest uziemiona i autoryzowana.

4.2 Analiza programów wspomagana przez LLM, eksploracja symboliczna, transformatory abstrakcyjne i certyfikowane NLP

Interakcja między LLMs a analizą programu przebiega w obu kierunkach. LLMs może pomóc w obsłudze specyfikacji, wzorców kodu i środowisk, które są trudne do modelowania. Formalne narzędzia mogą sprawdzać kod generowany przez LLMs, weryfikować sieci neuronowe i ograniczać wyuczone komponenty. Ta literatura jest szczególnie cenna dla wykonywalnego języka naturalnego, ponieważ testuje to samo pytanie architektoniczne pod wyraźniejszą semantyką: gdzie model probabilistyczny może przyczynić się, nie stając się ostateczną wyrocznią?

Tradycyjna analiza statyczna zależy od ręcznie zaprojektowanych abstrakcji, funkcji transferu, podsumowań i modeli środowiska. Symboliczne wykonanie zależy od wyszukiwania ścieżki i kodowania solvera. Narzędzia te oferują pryncypialne gwarancje, ale zmagają się z dynamicznymi funkcjami językowymi, kodem biblioteki, złożoną manipulacją ciągami, usługami zewnętrznymi i skalowalnością. LLMs może rozpoznawać intencje kodu, proponować inwarianty, podsumowywać funkcje, identyfikować prawdopodobne lokalizacje błędów i generować testy. Badanie LLM analizy programu wspomaganego przez LLM w 2025 roku kategoryzuje pracę w metody statyczne, dynamiczne i hybrydowe oraz podkreśla zarówno szansę, jak i ryzyko związane z probabilistycznymi wyjściami [Wang et al. 2025].

Jedną z ról architektonicznych jest wskazówki heurystyczne. Symboliczny wykonawca może zachować swoją semantykę i użyć LLM do szeregowania ścieżek, identyfikacji wrażliwych regionów lub wyboru granic pętli. Jeśli heurystyka jest błędna, zasięg może ucierpieć, ale akceptowane ścieżki pozostają sprawdzone przez solvera. Jest to analogiczne do użycia modelu neuronowego do porządkowania wyszukiwania w twierdzeniu dowodzenia. Rozróżnienie między wskazówkami a akceptacją jest krytyczne. Wyuczony wynik może określać, gdzie są wydawane zasoby; nie powinien określać, czy warunek ścieżki jest zadowalający.

KLEECopilot reprezentuje ten kierunek. Preprint 2026 wykorzystuje wytyczne LLM do kierowania KLEE w kierunku prawdopodobnych luk w zabezpieczeniach i wprowadza heurystykę do ucieczki pętli, zgłaszając znaczną poprawę w zakresie pokrycia bloków podstawowych i linii oraz dziesiątki unikalnych naruszeń [Chen et al. 2026]. Jako niedawny preprint, wyniki wymagają niezależnej replikacji i wrażliwości na modelowanie spraw rodzinnych. Niemniej jednak wzór projektu jest prawdopodobny: LLMs przyczynia się do semantycznego priorytetyzacji, której brakuje w klasycznym wyszukiwaniu, podczas KLEE zachowując semantykę wykonania.

Kolejną rolę jest generowanie testów satysfakcjonujących drogę. Model może odczytać kod i zaproponować dane wejściowe, które mają przemierzyć ścieżkę docelową. Konkretnie wykonanie potwierdza następnie, czy ścieżka została osiągnięta. Takie podejście może obsługiwać konwencje API lub formaty danych, które są uciążliwe dla SMT. Badania pokazują przydatne wskaźniki sukcesu, ale także porażki na długich ścieżkach, rekurencja, niewykonalne gałęzie i złożone biblioteki. Bezpieczna interpretacja jest taka, że LLM jest generatorem testowym. Betonowy bieg, ślad oprzyrządowania lub solver pozostaje walidatorem.

AutoBug przyjmuje bardziej radykalne podejście, stosując LLMs do bezpośredniego [Li et al. 2025] rozwiązywania problemów. Zamiast przełożyć wszystkie ograniczenia na teorię solvera, przedstawia warunki ścieżki na poziomie kodu do modelu. Może to być agnastyka językowa i

może analizować semantykę, którą trudno jest zakodować. Autorzy zgłaszają ulepszenia w zakresie kilku wskaźników analizy programu, szczególnie w przypadku mniejszych modeli. Charakteryzują również analizę jako przybliżoną. System nie oferuje klasycznej solidności, a jego wnioski mogą się różnić w zależności od modelu.

AutoBug jest cenny, ponieważ pokazuje, gdzie metody symboliczne zawodzą operacyjnie. Wiele prawdziwych programów nazywa nieprzezroczyste biblioteki, manipuluje złożonymi obiektami lub zależy od nieformalnych umów. Model językowy może wywnioskować prawdopodobne zachowanie z nazw, komentarzy i poznanych wzorców kodu. Ale ta elastyczność jest tym samym źródłem niepewności. W portfolio zorientowanym na pewność przybliżona LLM analiza powinna generować kandydatów, podsumowania lub priorytetowe kontrole. Wyniki powinny zostać potwierdzone przez testy, umowy, monitorowanie środowiska wykonawczego lub silnik formalny za każdym razem, gdy zostanie zgłoszone roszczenie dotyczące bezpieczeństwa.

LLMs może zaproponować niezmienniki do interpretacji abstrakcyjnej. Kandydujący niezmiennik może przyspieszyć dowód, jeśli sprawdzanie weryfikuje, że początkowo się utrzymuje, jest zachowywane przez przejścia i oznacza, że nieruchomość. Jest to potężny podział pracy: niezmiennie odkrycie jest twórcze i kombinatoryczne, podczas gdy niezmiennie sprawdzanie jest mechaniczne. Podobne wzorce pojawiają się w twierdzeniu udowodnienia i syntezie programu. Model języka szuka; weryfikator akceptuje.

SAIL rozszerza tę zasadę na syntezę abstrakcyjnych transformatorów [Gu et al. 2026]. Konstruowanie transformatora, który jest zarówno dźwiękowy, jak i precyzyjny dla operatora nieliniowego, jest trudne. SAIL prosi LLMs o generowanie kandydatów, ale nakłada twarde ograniczenia składniowe i semantyczne i wykorzystuje matematycznie uziemiony koszt za nierozsądek. Zgłoszony system syntetyzuje globalnie transformatory dźwiękowe w kilku domenach, pasuje do projektów ręcznych i znajduje precyzyjne transformatory nie wcześniej w literaturze. Znaczenie wykracza poza weryfikację sieci neuronowej. Jest to dowód na to, że LLMs może zautomatyzować budowę metodą formalną, gdy każdy przyjęty artefakt podlega niezależnym obowiązkom dowodowym.

Kontrast z prośbą do LLM o „działanie jako abstrakcyjny interpreter” jest ostry. Mitchell i współpracownicy skłonili modele do rozsądku poprzez abstrakcyjną interpretację programów SV-COMP [Mitchell et al. 2025]. Modele mogą śledzić style kompozycyjne lub fixpoint w niektórych przypadkach, ale popełniały błędy logiczne i halucynowały na złożonych strukturach. Wynik jest spójny z ogólnym wzorcem: modele mogą naśladować procesy formalne i generować przydatnych kandydatów, ale niezawodność spada, gdy proces wymaga trwałego dokładnego stanu. Prawdziwy abstrakcyjny interpreter powinien wykonywać operacje kratowe; model może sugerować domeny, predykaty lub wyjaśnienia.

Formalna weryfikacja modeli neuronowych NLP to kolejna główna gałąź. Certyfikowana praca nad solidnością pyta, czy przewidywanie modelu pozostaje niezmiennie we wszystkich wejściach w zdefiniowanym zestawie perturbacji. Jia i współpracownicy używali propagacji interwałowych dla podstawień słów i zgłaszali 75-procentową dokładność kontradyktoryjną na IMDB i SNLI w ramach skonstruowanych rodzin zstępujących [Jia et al. 2019]. SAFER zastosowano randomizowane wygładzenie i może dotyczyć modeli czarnoskrzynkowych, w tym BERT [Ye et al. 2020]. Świadomość solidności osadza zaostrome granice certyfikacji, poprawiając certyfikowaną solidną dokładność na IMDB z 76,73 do 84,78 procent w zgłoszonym otoczeniu [Wang et al. 2023].

Weryfikacja transformatora jest trudna, ponieważ samouwaga zawiera produkty, operacje softmax i korelacje w różnych pozycjach. Wspecjalizowane domeny poprawiają granice. Parametryzowana abstrakcyjna interpretacja dla weryfikacji transformatora wprowadziła parametryzowane domeny afine dla produktów wewnętrznych i zgłosiła ściślejsze granice i weryfikacja instancji, których wcześniejsze metody nie mogły udowodnić [Huang et al. 2026]. Ta linia pracy pokazuje, że analiza formalna może skalować się do nietrywialnych architektur NLP, gdy właściwość i zestaw wejściowy są precyzyjnie zdefiniowane.

Nie wolno zatrzeć zakresu gwarancji. Certyfikowany klasyfikator może być solidny do każdej podstawienia narysowanej z określonego zestawu i nadal się mylić. Lista substytucyjna może zawierać kontekstowo nieodpowiednie „synonimy” lub pominąć znaczące parafrazy. Certyfikat mówi, że model jest niezmienny w stosunku do formalnej rodziny perturbacji. Nie ustanawia równoważności semantycznej w ludzkim języku. Rozróżnienie to powinno być wyraźne w każdym systemie wykonywalno-językowym, który wykorzystuje certyfikowane modele ekstrakcji.

Przydatna integracja przywiązuje certyfikat solidności do obserwacji semantycznej. Załóżmy, że wyciąg zdarzenia identyfikuje, że zdanie określa obowiązek. Certyfikowany model może wykazać, że klasyfikacja ta jest stabilna pod kontrolowaną rodziną substytucji leksykalnych. Obserwacja jest nadal proponowana modelowo, ale ma dodatkową właściwość stabilną. Analiza symboliczna w dole może rejestrować te dowody bez podnoszenia obserwacji do faktów. Taki warstwowy status jest bardziej pouczający niż jeden wynik zaufania.

Formalne metody weryfikują również kod generowany przez LLMs. Generowanie testów, analiza statyczna, sprawdzanie typów i sprawdzanie dowodów mogą wykryć wiele błędów. Systemy syntezy programu coraz częściej wykorzystują kompilator i informacje zwrotne do iteracyjnej naprawy. Ten sam wzór powinien regulować generowane obwody semantyczne i kod formalizacji. Agent kodowania może być autorem implementacji reguły, ale publikacja powinna wymagać kompilacji, testów mutacji, naturalnych przypadków referencyjnych, powtórek i niezależnych weryfikatorów. Nie należy dopuścić do tego, aby LLM tworzył ukryte zachowania wykonywalne wewnątrz artefaktu teorii deklaratywnej.

Literatura na temat modelowania protokołu z języka naturalnego daje oświecający wynik. Prace prezentowane na ICSE 2025 zbadały generowanie modeli Tamarin lub ProVerif z dokumentacji protokołu. Naiwne stosowanie LLM było nieskuteczne; udany rurociąg wymagał rozkładu semantycznego, ograniczonej ujednoznacznienia człowieka i przekształceń pośrednich dźwięku. Wygenerował poprawne modele o umiarkowanej skali dla 10 z 18 rzeczywistych protokołów [Mao et al. 2025]. Rezultat jest znaczący, ponieważ protokoły zawierają role, wiadomości, założenia kryptograficzne, stan i porządek czasowy. Pokazuje również, że nawet silna dokumentacja strukturalna nie kompiluje się automatycznie w formalne modele bezpieczeństwa.

Przypadki awarii są pouczające. Dokumentacja pomija założenia, które eksperci wnioskuje. Ten sam termin może oznaczać typ wiadomości, stan lub instancję. Właściwości bezpieczeństwa mogą być podane nieformalnie lub rozproszone po sekcjach. Formalny model protokołu musi wyraźnie określać możliwości przeciwnika. Są to bezpośrednie analogie rozumowania dokumentów. Rozwiązaniem nie był większy monit, ale architektura, która oddziela parsowanie semantyczne, ujednoznacznianie i zweryfikowaną transformację.

Zewnętrzne przeglądy szerszego pola zbiegają się na podobnym wniosku. Neuro-symboliczny NLP przegląd 2026 sprzyja silniejszemu badaniu architektur federacyjnych, ale ostrzega, że obecne porównania nie są kontrolowane i że nie ustanowiono żadnego

podstawowego systemu federacyjnego. Badanie planowania-formalizatora traktuje LLMs jako konstruktory i rafinerie modeli formalnych, a nie zamienniki dla planistów [Tantakoun et al. 2025]. Badanie analizy programu podkreśla systemy hybrydowe oraz konieczność zarządzania halucynacją i odtwarzalnością [Wang et al. 2025]. Są to niezależne linie dowodów na ten sam podział pracy.

Pole to osiągnęło zatem więcej niż powierzchowny wzór „LLM plus solver”. Wykazała się naprawą kierowaną przez solver, formalizacjami sprawdzanymi przez dowód, planowaniem formalnym, prowadzeniem ścieżki, niezmiennym wnioskiem, syntezą transformatorów i certyfikowaną solidnością. Obnażyła również stabilną granicę. Gdy LLM kontroluje akceptację końcową, gwarancje stają się przybliżone. Kiedy LLM proponuje artefakty, które sprawdza odrębny system formalny, można zachować silne gwarancje dla sprawdzonej nieruchomości. Pytanie bez odpowiedzi brzmi, jak sprawić, by granica natural-język-artefakt była porównywalnie zdyscyplinowana.

Architektura zaproponowana w dalszej części tej książki stosuje bezpośrednio lekcje analizy programu. Transformacje semantyczne kandydatów są traktowane jak zsyntetyzowany kod. Mają wpisywane kontrakty, mapy źródłowe, granice zasobów i testy. Domeny abstrakcyjne są rejestrowane, a nie improwizowane podczas biegu. Predykaty dopracowania są sugerowane przez LLMs, ale zatwierdzane poprzez konkretne rozróżnienia. Portfele Solvera eksponują przedmioty dowodowe lub świadków. Renderer językowy zużywa kanoniczne wyniki, a nie surowe ślady. W tym projekcie elastyczność LLM jest atutem właśnie dlatego, że jest otoczona symbolicznymi bramkami odbioru.

5. Dlaczego systemy języka wykonywalnego zawodzą

5.1 Ugruntowanie, tożsamość, pozorne formalizacje, ontologie i wiedza o świecie

Dominującym błędem w systemach wykonywalnych nie jest nieprawidłowe odliczenie od przyjętych przesłanek. Jest to przyznanie się do niewłaściwej przesłanki, przedmiotów lub relacji. Uziemienie to proces, w którym symbole nabywają odniesienie operacyjne: predykat jest połączony z frazą, jednostka do jednej lub więcej wzmianek, zdarzenie do dowodów, wyrażenie czasowe do interwału, pole bazy danych do koncepcji lub działanie do zdolności środowiska. Formalny program bez niezawodnego uziemienia jest dokładny w najmniej użytecznym sensie. Wykonuje konsekwentnie, opisując niewłaściwy świat.

Uziemienie zaczyna się od lokalizacji źródła. Każda obserwacja semantyczna powinna być powiązana z dokładnymi dowodami źródłowymi lub wyraźnie oznaczona jako zewnętrzne założenie. Należy zwrócić uwagę na niezmienną rewizję źródła. Podsumowanie dokumentu nie może zastąpić źródła, ponieważ może pominąć kwalifikacje. Wyniki pobierania nie mogą poprzeć roszczenia dotyczącego nieobecności, chyba że proces pobierania jest zakończony dla odpowiedniej domeny. Ta dyscyplina źródłowa zapobiega powszechnej awarii w systemach LLM: model wytwarza wiarygodny tekst podtrzymujący, który nie występuje w dokumencie, a wyjście strukturalne w dół strumienia daje wynalazkowi wygląd autorytetu.

Uziemienie leksykalne łączy język z ontologią. Zwrot „zatwierdzenie” może oznaczać działanie, państwo, podpisany artefakt, pozwolenie lub decyzję instytucjonalną. Schemat domeny musi wybrać i udokumentować interpretację potrzebną zgodnie z regułą. LLMs są przydatne, ponieważ mogą wnioskować o kontekście i proponować odwzorowania, ale mapowanie powinno być testowane pod kątem przykładów i alternatyw. Zapalka ciągu między frazą a nazwą typu nie jest kompatybilnością semantyczną. Dokumentacja nllAgent wyraźnie ostrzega przed mapowaniem dwóch pojęć tylko dlatego, że ich etykiety pasują; wymaga adaptera lub sondy semantycznej, gdy znaczenia się różnią.

Uziemienie tożsamości jest trudniejsze i bardziej znaczące. Analiza dokumentów często musi wiedzieć, czy dwie wzmianki oznaczają tę samą osobę, organizację, obiekt, zbiór danych, wymagania lub zdarzenie. Równe nazwy nie oznaczają tożsamości, a różne nazwy nie oznaczają różnicy. Tytuły zmieniają się, pseudonimy występują, zaimki są niejednoznaczne, organizacje mają podpodziały, a fikcyjne lub cytowane wzmianki mogą nie należeć do świata źródłowego. Bezpieczna warstwa tożsamości oddziela wzmianki od hipotez podmiotów i kandydatów na tożsamość. Rejestruje producenta, dowody, zakres, świat i alternatywy dla każdego powiązania.

Przedwczesne łączenie tożsamości powoduje błędy kaskadowe. W narracji można scalić dwa znaki o tym samym nazwisku, powodując niemożliwe naruszenia ciągłości. W dokumencie regulacyjnym „Urząd” może odnosić się do różnych organów w różnych jurysdykcjach. W dokumentacji oprogramowania „klient” może oznaczać użytkownika, komponent aplikacji lub punkt końcowy HTTP. Po połączeniu późniejsze reguły traktują wszystkie właściwości jako należące do jednego obiektu. Błąd staje się trudny do zdiagnozowania, ponieważ solver widzi spójny identyfikator. System konserwatywny powinien zezwalać na nierozwiązaną tożsamość i zwracać nieznany, gdy reguła wymaga powiązania, które nie zostało ustalone.

Uziemienie czasowe ma podobne warstwy. Daty mogą być wyraźne lub względne. „W ciągu trzydziestu dni” wymaga wydarzenia kotwicy i polityki kalendarza. „Przed wydaniem” może odnosić się do planowanego lub rzeczywistego wydania. „Po zatwierdzeniu” może oznaczać natychmiast po lub w dowolnym późniejszym czasie. Jurysdykcje inaczej określają dni robocze. Rozwiązywacz czasowy może zarządzać interwałami i zamawiać dokładnie po dokonaniu tych wyborów, ale nie może wnioskować o zamierzonej kotwicy z dowolnej etykiety. Obserwacje czasowe powinny zatem obejmować wyrażenie tekstowe, wartość znormalizowaną lub ograniczenie, kotwicę, kalendarz, strefę czasową, pewność siebie i alternatywy.

Normatywne uziemienie jest często źle traktowane. „Must”, „show”, „show”, „może”, „może” i „ma się mieć” ma siłę specyficzną dla domeny. Zjazdy prawne różnią się od zwykłego języka. Wyjątek może pokonać regułę, zawęzić jej zakres lub stworzyć oddzielne pozwolenie. Sprzeczne władze mogą mieć priorytety. Prosta boolowska reprezentacja zgodności traci te rozróżnienia. Logika deontyczna, priorytety reguł, tabele decyzyjne lub systemy definitywne mogą je modelować, ale mapowanie od źródła do reguł pozostaje osądem językowym i instytucjonalnym.

Fałszywe formalizacje pojawiają się, gdy niewłaściwy program daje oczekiwany wynik. Słabo nadzorowane parsowanie semantyczne wyraziło ten problem na jaw, ale jest wszechobecne w LLM ocenie. Wygenerowana formuła może rozwiązać punkt odniesienia, ponieważ przykłady mają regularne szablony. Zapytanie tekstowe do SQL może zwracać prawidłowe bieżące wiersze pomimo błędnego połączenia. Model planowania może znaleźć plan, ponieważ pominięte ograniczenie jest nieistotne w testowanym przypadku. Obwód reguł może przejść dziesięć przykładów podczas kodowania niewłaściwego pojęcia. Sama dokładność odpowiedzi nie może rozróżnić tych przypadków.

Odpowiednią obroną jest mutacja semantyczna. Oceniający zmienia jeden aspekt źródła lub środowiska, zachowując inne, a następnie sprawdza, czy formalizacja zmienia się w oczekiwany sposób. Jeśli „wszystko” stanie się „niektórymi”, kwantyfikator powinien ulec zmianie. Jeśli wyjątek zostanie usunięty, wcześniej zwolniona ścieżka powinna stać się zabroniona. Jeśli dwie jednostki zostaną przemianowane na ten sam ciąg znaków, tożsamość nie powinna automatycznie się scalać. Jeśli instancja bazy danych się zmieni, przypadkowo poprawne zapytanie SQL powinno ulec awarii. Testowanie mutacji ocenia znaczenie programu, a nie tylko jego zachowanie na jednym urządzeniu.

Kontrprzykładowe pokolenie może porównywać konkurencyjne formalizacje. Biorąc pod uwagę dwa programy logiczne lub wykonywalne, solver może wyszukiwać model, w którym różnią się ich wyjścia. Powstały scenariusz jest tłumaczony na język naturalny i przedstawiony recenzentowi. To zamienia nieprzejrzyste porównanie formuły w konkretne pytanie: „Pod interpretacją A, przełożony, który nie ukończył szkolenia, może zatwierdzić; zgodnie z interpretacją B, mogą nie. Który jest przeznaczony?” Metoda jest szczególnie przydatna, gdy kilka LLMs wytwarza wiarygodne, ale nierównoważne formy.

Projekt ontologii tworzy kolejny tryb awarii. Uniwersalny schemat jest atrakcyjny do ponownego użycia, ale może ukryć nieporozumienia dotyczące domeny i zmusić koncepcje do nieodpowiednich kategorii. Natomiast niezwykle wąski schemat wytwarza wiele niezgodnych języków lokalnych. Preferowana jest konstrukcja warstwowa. Górna warstwa definiuje źródło, dowody, hipotezy tożsamości, światy, czas, modalność, status, zasięg i luki. Pakiety domenowe definiują typy merytoryczne i relacje. Mapowanie pakietów deklaruje korespondencje, straty i założenia między domenami.

Zgodność z ontologii powinna być sprawna, a nie retoryczna. Obwód deklaruje rodzaje i gwarantuje, że zużywa. Producent deklaruje, które typy może zmaterializować, w ramach jakich statusów i trybów pokrycia. Publikacja sprawdza, czy istnieje kompatybilny producent. Czas działania sprawdza, czy konkretny dokument przyniósł wymagane obserwacje. Jeśli obwód wymaga tożsamości potwierdzonej przez człowieka, ale istnieją tylko kandydaci tożsamości zaproponowani przez model, nie może on działać z deklarowaną gwarancją. Jeśli reguła czasowa wymaga kalendarza jurysdykcyjnego, który jest nieobecny, wynikiem jest luka operacyjna, a nie odgadnięta data.

Wiedza światowa jest szczególnie niebezpieczna. LLMs zawiera szeroką wiedzę, ale nie ma stabilnego pochodzenia, daty wejścia w życie ani jurysdykcji. Traktowanie pamięci modelowej jako formalnej bazy faktów tworzy niewidzialne zależności i nieświeże wnioski. System symboliczny powinien łączyć jawne pakiety wiedzy z identyfikatorami źródeł, trawieniami, efektywnymi odstępami czasu, jurysdykcjami, polityką konfliktu i pułapami gwarancji. Opakowanie może być kuratorowane, pobierane lub wspomagane modelem, ale jej przyjęte twierdzenia są zewnętrznymi artefaktami. Pakiet prawny może wygasnąć. Pakiet naukowy można zastąpić. Fikcyjna paczka świata może celowo zaprzeczać obecnej rzeczywistości.

Konflikty między źródłami nie powinny być usuwane poprzez głosowanie większościowe, chyba że zadanie określa taką politykę. System może potrzebować ułaskawienia, priorytetu źródła, wersjonowania czasowego lub oddzielnych światów. Roszczenie może być poparte przez jedno źródło i sprzeczne z innym. Wynik kanoniczny powinien zachować zarówno, jak i stwierdzić, która polityka, jeśli w ogóle, rozwiązuje je. Ma to zasadnicze znaczenie dla badań naukowych i analiz regulacyjnych, w przypadku gdy niezgoda jest raczej informacją niż hałasem.

Założenia w tle muszą być widoczne. Logiczne benchmarki często zakładają, że nazwy oznaczają różne osoby lub te kategorie są rozłączne. Prawdziwe dokumenty rzadko mówią wszystkie takie aksjomaty. LLM może je wstawić, ponieważ sprawiają, że puzzle są rozwiązywalne. W systemie badań lub zgodności założenia te powinny pojawiać się w oddzielnej warstwie władzy. Ostateczna odpowiedź może następnie stwierdzić, że wniosek następuje tylko wtedy, gdy założenie się spełni. Zapobiega to przebieganiu się za dowody dokumentu przez niejawną wiedzę światową.

Zasięg to uziemienie nieobecności. Aby stwierdzić, że dokument nie zawiera wymaganej zgody, system musi wiedzieć, co zostało przeszukane, które sekcje i kanały zostały uwzględnione, w jaki sposób koncepcja została rozpoznana i czy metoda wyszukiwania była kompletna dla roszczenia. Skanowanie leksykalne może być zakończone dla dokładnej frazy, ale nie dla parafraz. Pobieranie może być wysokim wycofaniem, ale nie zamkniętym światem. Rejestr z człowiekiem może być kompletny w celu wydania. Zasięg powinien być przedmiotem pierwszej klasy pochłoniętym przez reguły negatywne.

Architektura nllAgent zawiera ważną wersję tej zasady. LongTextJS rejestruje zasięg i luki wraz z obserwacjami. Kompatybilność obwodu sprawdza, czy rzeczywisty program spełnia wymagany status i zasięg. Brak wiedzy nie jest zgłaszany jako zgodność. Jest to silniejsze niż powszechny wzór polegający na poproszeniu LLM o zwrot prawdziwego, fałszywego lub nieostwierdzonego, ponieważ nie zostanie zidentyfikowany w stosunku do udokumentowanej domeny [AssistOS-AI 2026] wyszukiwania.

Pozostałą trudność polega na tym, jak zweryfikować obserwacje produkowane w modelu. Walidacja schematu jest konieczna, ale słaba. Zakotwiczenie dowodów zapobiega wymyślonemu

cytatami, ale nie błędnej interpretacji. Wyniki kalibracji pomagają klasyfikować kandydatów, ale nie ustalają znaczenia. Najsilniejsze podejście praktyczne łączy w sobie niezależne sygnały: parsery w języku kontrolowanym dla fragmentów, modelowe zespoły lub różne odpowiedzi; ograniczenia ontologiczne; spójność między zdaniami; mutacje semantyczne; negatywne przykłady; ludzki przegląd dla rozróżnień o dużym wpływie; oraz kontrprzykłady, które ujawniają konsekwencje.

Uziemienie powinno być również napędzane przez zapytania. System nie musi rozwiązywać każdego podmiotu lub zdarzenia w 300-stronicowym dokumencie. Powinien on czerpać zapotrzebowanie semantyczne z wykonywanej reguły, pobierać odpowiednie zakresy i materializować tylko niezbędne typy. Jednak przetwarzanie query-first nie może zmylić trafności z kompletnością. Jeśli reguła pyta, czy jakkolwiek wyjątek istnieje w dowolnym miejscu w tekście autorytetu, pobieranie musi stanowić argument pokrycia lub odpowiedź pozostaje nieznana. Zapotrzebowanie na wykonanie zmniejsza pracę; nie usprawiedliwia przedwczesnego zamknięcia.

Podstawowa konkluzja jest poważna, ale konstruktywna. Nie ma czysto symbolicznego lekarstwa na symbol przywiązany do niewłaściwego referentu. Uziemienie nie jest wstępnym NLP szczegółem; jest częścią zaufanych obliczeń. System musi wystawiać łańcuch od źródła do symbolu, zachować alternatywy, śledzić zasięg i odmówić niekompatybilnego wykonania. Gdy ten łańcuch zostanie zdyscyplinowany, dojrzałe narzędzia formalne mogą dodać znaczną niezawodność. Bez niego narzędzia po prostu poświadczają konsekwencje wiarygodnej fikcji.

5.2 Pragmatyka, niemonotoniczność, skala, bezpieczeństwo i porażka ewaluacji

Nawet dobrze ugruntowana reprezentacja może się nie udać, ponieważ język naturalny robi więcej niż stwierdzać monotonne fakty. Mówcy sugerują, zakładają, zmieniają, cytują, żartują, negocjują i polegają na kontekście instytucjonalnym. Dokumenty zawierają wyjątki, priorytety, definicje, które zmieniają się później, oraz stwierdzenia, których siła zależy od gatunku. Wykonalny język naturalny musi zdecydować, które z tych zjawisk zamierza wesprzeć i musi zwrócić nieznane poza tym zakresem.

Pragmatyka jest pierwszą granicą. Zdanie może komunikować więcej niż jego dosłowne warunki prawdy. „Raport jest prawie kompletny” może sugerować, że nie jest kompletny. „Niektóre testy przeszły” mogą pragmatycznie sugerować, że nie wszyscy to zrobili, chociaż klasyczna logika tego nie pociąga za sobą. Ironia odwraca pozorne nastroje. Żądania pośrednie wykorzystują oświadczenia jako działania. Formalizator, który traktuje pragmatyczne wnioski jako fakty, może przesadzić; taki, który je ignoruje, może przegapić praktyczne intencje autora. Odpowiednie leczenie zależy od zastosowania. Silnik zgodności powinien zwykle uprzywilejować wyraźny autorytet i etykietę pragmatyczną interpretację, jak zaproponowano. Agent konwersacyjny może potrzebować bogatszych modeli pragmatycznych, ale nie powinien przedstawiać ich jako twardych dowodów.

Presupozycja stwarza związane z tym wyzwanie. „Komisja przestała przeglądać wniosek” zakłada, że dokonała jej przeglądu. Określone opisy często zakładają istnienie lub wyjątkowość. Dokumenty mogą wykorzystywać takie konstrukcje bez wyraźnego dochodzenia do tła. System formalny musi zdecydować, czy założenia wchodzą w zakres założeń, wniosków o defeasable, czy też wymogów do wyjaśnienia. Automatyczne ich zapewnianie może być niebezpieczne; ignorowanie ich może sprawić, że dyskurs będzie niespójny. System stanu powinien zachować ich pochodzenie.

Zasady języka naturalnego są często niemonotoniczne. Nowe informacje mogą wycofać wniosek. „Pracownicy mogą uzyskać dostęp do systemu” może zostać pokonany przez „chyba że ich certyfikacja wygasła”. Logika domyślna, programowanie typu answer-set, logika możliwa do wykwapienia i priorytetowe zasady zapewniają formalne mechanizmy dla wyjątków. Klasyczna logika pierwszego rzędu może wyraźnie kodować wyjątki, ale duże zestawy reguł stają się trudne do zarządzania, a niespójne informacje mogą trywializować wnioskowanie, jeśli dotyczy zwykłej eksplozji.

Praktyczny czas działania powinien wspierać wiele semantyki reguł. Monoton Datalog jest odpowiedni, gdy fakty gromadzą się tylko. Zasady możliwe do zniesienia są odpowiednie dla domyślnych i wyjątków. Równoznaczne rozumowanie jest właściwe, gdy źródła są sprzeczne, a system musi unikać wyprowadzania wszystkiego. Tabele decyzyjne są właściwe, gdy kombinacje polityk są skończone i wyraźne. Wybrana semantyka musi być wersjonowana z pakietem reguł; nie może być wywnioskowana ad hoc przez LLM podczas wykonywania.

Systemy normatywne dodają autorytetu i priorytetu. Późniejsza zasada może zastąpić wcześniejszą. Regulacja wyższego szczebla może uprzedzić procedurę lokalną. Jurysdykcja i data wejścia w życie. Uprawnienia i obowiązki mogą być sprzeczne. Deontyczne paradoksy pokazują, że naiwne kodowanie logiczne może przynieść sprzeczne z intuicją wyniki. Wiele praktycznych przypadków jest lepiej obsługiwanych przez wyraźne priorytety reguł, wyjątki i

warunki naruszenia niż przez uniwersalny rachunek deontyczny. Wykonywalny język naturalny powinien sprzyjać jasności operacyjnej w stosunku do teoretycznej wielkości.

Skala tworzy kilka oddziałujących wybuchów. Długie dokumenty zawierają wiele rozpiętości i obserwacji kandydatów. Niejednoznaczne wzmianki tworzą kombinatorskie zadania tożsamości. Stosunki czasowe tworzą alternatywy dla kolejności częściowej. Zasady tworzą pochodne konsekwencje. Symboliczne wykonanie tworzy ścieżki. Rozwiązywacze borykają się z dużymi systemami ograniczeń. LLM kontekstowe okna i koszty nakładają odrębne limity. Naiwna architektura, która formalizuje każde zdanie w każdą możliwą logikę, jest niewykonalna.

Materializacja napędzana popytem jest pierwszą odpowiedzią. Obwody deklarują, co konsumują. Struktura indeksów runtime i stabilne informacje leksykalne, a następnie wytwarzają bogatszą semantykę tylko dla odpowiednich zakresów. Obliczenia przyrostowe ponownie wykorzystują niezmiennie artefakty. Abstrakcyjna interpretacja podsumowuje alternatywy. Łączące się i ograniczone symboliczne ścieżki kontroli eksploracji. Portfele Solvera kierują proste przypadki do tanich silników. Twarde budżety dają wyraźne niepełne wyniki, a nie ukryte obciążenie.

Drugą odpowiedzią jest modułowość semantyczna. Dokument można analizować za pomocą widoków: sekcji, okien czasowych, jurysdykcji, scen narracyjnych, etapów eksperymentalnych lub warstw autorytetu. Tożsamość i konsekwencje reguł mogą być lokalne w widoku. Relacje między poglądami są wprowadzane tylko wtedy, gdy jest to wymagane. Zmniejsza to przypadkowe globalne scalanie i wspiera równoległość. System powinien jeszcze rejestrować, kiedy wniosek zależy od założenia, że widok jest kompletny.

Trzecią odpowiedzią jest inscenizowana precyzja. Niedrogie podanie strukturalne lub leksykalne eliminuje wiele przypadków. Przepustka wydobywcza wspomagana modelem produkuje kandydatów do pozostałych. Analiza abstrakcyjna identyfikuje potencjalne naruszenia. Symboliczne udoskonalenie obsługuje tylko niejasności istotne dla wyniku. Przegląd człowieka jest zarezerwowany dla nierozwiązanych wyborów o dużym wpływie. Przypomina to rurociągi statycznej analizy, które wykorzystują tanie analizy przed udowodnieniem kosztownego twierdzenia.

Groźby bezpieczeństwa pojawiają się na każdym etapie. Szybkie wstrzyknięcie w dokumentach źródłowych może poinstruować LLM, aby ignorować schematy lub fabrykować wyniki. Tłumacz powinien odbierać treści źródłowe jako dane w trybie wiersza specyficznego dla reguły, a jego dane wyjściowe powinny być zatwierdzone niezależnie. Źródło nie może nigdy uzyskać uprawnień do modyfikacji czasu pracy, pakietu reguł lub polityki publikacji. Każdy tekst twierdzący, że jest instrukcją systemową, jest nadal zawartością dokumentu.

Wygenerowany kod to jeszcze większe ryzyko. DSL, który umożliwia dowolny JavaScript, Python, import lub dostęp do sieci, zamienia naukę semantyczną w wykonywanie kodu. Obwody deklaratywne powinny normalizować się do zwykłych danych i wywoływać tylko zarejestrowanych operatorów. Algorytmy niestandardowe powinny być instalowane jako sprawdzone rozszerzenia z typowanymi umowami, efektami, limitami zasobów i zablokowanymi trawnikami. Sandboxing pozostaje konieczny, ale ograniczenie zdolności jest silniejsze niż poleganie tylko na granicy procesu.

Ataki Solvera obejmują odmowę służby poprzez ograniczenia patologiczne, nieuformowane obiekty proofowe i eksploatację natywnych bibliotek. Budżety, ograniczenia teoretyczne, sprawdzanie dowodów i przypinanie wersji są wymagane. Model może celowo lub przypadkowo

skonstruować formułę, która jest technicznie ważna, ale obliczeniowa. Wybór formalizmu powinien oszacować koszt i odrzucić zadania poza polityką.

Zatrucie danymi dotyczy wyuczonych producentów semantycznych i pakietów porównawczych. Jeśli agent kodujący generuje zarówno obwód, jak i przykłady, które go potwierdzają, artefakty mogą mieć to samo nieporozumienie. Niezależne generowanie testów, mutacje semantyczne, ręcznie kurowane przypadki kotwicy i oddzielne role przeglądowe zmniejszają to ryzyko. Architektura nllAgent uczenia się prawidłowo uniemożliwia agentowi kodowania publikowanie własnego kandydata. Publikacja ręczna jest kontrolą zarządzania, chociaż jej skuteczność zależy od jakości wskaźników referencyjnych i niezależnych kontroli.

Realizacja w języku naturalnym może ponownie wprowadzić niepoparte roszczenia. Model może przekształcić „możliwe pod jedną interpretację” na „dokument narusza regułę”. Może pominąć założenia lub przekształcić niepełne wyszukiwanie w nieobecność. Realizacja powinna być zatem ograniczona schematem wyników kanonicznych i sprawdzana na poziomie roszczeń. Wyjścia o dużym wpływie mogą wykorzystywać kontrolowane szablony. Wytłumaczający komentarz należy wyraźnie oddzielić od zweryfikowanych ustaleń.

Ocena jest chyba najbardziej systemową porażką. Wiele punktów odniesienia mierzy tylko ostateczną dokładność odpowiedzi. To zakłóca ekstrakcję, formalizację, rozumowanie i renderowanie. Model może dotrzeć do poprawnej odpowiedzi z niewłaściwego powodu. Formalny system może zostać ukarany za powrót nieznany, w przypadku gdy punkt odniesienia zakłada nieokreśloną konwencję. Syntetyczne zbiory danych mogą nagradzać rozpoznawanie szablonów. LLM-as-a-sugetry może dzielić się uprzedzeniami z ocenianymi systemami.

Rygorystyczny punkt odniesienia powinien zawierać wiele warstw. Warstwa źródłowa zawiera dokładny tekst i strukturę. Warstwa semantyczna zawiera jedną lub więcej dopuszczalnych interpretacji z rozpiętością dowodów. Warstwa formalna obejmuje programy docelowe lub testy równoważności rozszerzonej. Warstwa wykonania obejmuje oczekiwane modele, dowody, świadków lub niezaspokojone rdzenie. Warstwa wynikowa obejmuje statusy kanoniczne i pokrycie. Warstwa realizacji obejmuje dozwolone roszczenia i zakazane zawyżenia. Błędy można następnie przypisać, a nie zapaść w jeden wynik.

Minimalne pary są niezbędne. Zmiana jednego słowa lub relacji powinna przynieść przewidywalną różnicę formalną. Rodziny parafrazy testują niezmiennosc. Sekcja odwrócenia testu odwróć uwagę. Testowanie zderzeń tożsamości. Brak dowodów świadczy o nieznanym zachowaniu. Źródła sprzeczne testują parakonsekwencję lub politykę władzy. Klaszory czasowe graniczne testują kalendarze i interwały. Kontradyktoryjny tekst testuje izolację bezpieczeństwa. Mutacje semantyczne sprawdzają, czy obwód faktycznie implementuje swoją regułę.

Ocena powinna również mierzyć wstrzymanie się od głosu. System godny zaufania musi wiedzieć, kiedy brakuje mu on ontologii, producenta, pokrycia lub budżetu. Nieznane nie jest porażką, gdy pytanie jest niepoparte. Metryki powinny odróżniać uzasadnione wstrzymywanie się od możliwej do uniknięcia niekompletności. Krzywe jakości pokrycia mogą pokazać, jak zmienia się dokładność i fałszywe zapewnienie, ponieważ system przyznaje się do większej semantyki proponowanej przez model.

Niezależne wdrożenie ma znaczenie. Kompilator obwodu i jego „cień” oceniający nie powinny mieć każdej krytycznej funkcji, jeśli zostanie zgłoszone testy różnicowe. Wyniki mutacji powinny pochodzić z mutacji wykonywalnych, a nie opisów prozy. Profile modeli na żywo

powinny być oceniane oddzielnie od opaw deterministycznych. Planowanie i realizacja wymagają specyficznych dla zadań wskaźników referencyjnych, a nie wywnioskowania z wykonania audytu. Jest to dokładnie kilka poważnych problemów uznanych w repozytorium nllAgent [AssistOS-AI 2026].

Przeglądy zewnętrzne należy interpretować z podobną starannością. Badanie Frontiers 2026 jest recenzowane i przydatne, ale jej twierdzenie, że systemy federacyjne osiągają lepsze wyniki, jest wyraźnie kwalifikowane przez zajęcie zakłócające zadanie. Planowanie i analiza programów analizuje trendy, ale nie może ustalić, że proponowana architektura działa na nieograniczonych dokumentach. Nie znaleziono niezależnego przeglądu nllAgent. Odpowiedzialna książka powinna zatem wykorzystywać recenzje w celu zidentyfikowania konsensusu i niezgody, a następnie określać własne osądy, w których dowody są nieobecne.

Centralną zasadą oceny jest atrybucja przyczynowa. Kiedy system się poprawi, naukowcy powinni określić, czy zysk pochodzi z lepszego parsowania semantycznego, poprawności solvera, wyszukiwania, próbkowania w czasie testu lub wycieku porównawczego. Gdy się nie uda, powinni zidentyfikować warstwę. To nie jest tylko naukowa higiena. Warstwowe dane o błędach są tym, co pozwala na poprawę architektury bez zastępowania całego systemu lub ukrywania regresji w zbiorczych wynikach.

Wykonalny język naturalny pozostanie ograniczoną zdolnością. Pragmatyka, wiedza o otwartym świecie, kontekst społeczny i rozwijające się instytucje zapewniają, że żadne skończone symboliczne jądro nie oddaje wszelkiego znaczenia. Celem nie jest usunięcie tej granicy, ale uczynienie jej wyraźną i użyteczną. System może oferować silne gwarancje dla fragmentów strukturalnych, relacyjnych, czasowych, ilościowych i regulowanych reguł, jednocześnie zwracając wyniki warunkowe w innym miejscu. Najbardziej niebezpieczny system to nie taki, który przyznaje się do limitów. Jest to taki, który przekształca niezmodelowany język na pewne wyjście symboliczne i nazywa wynik zweryfikowanym.

6. Rodziny architektur dla wykonywalnego języka naturalnego

6.1 Konkurencyjne projekty: reprezentacje uniwersalne, portfele formalizmów, kontrolowana kompilacja i systemy query-first

Architektury w języku wykonywalnym mogą być zorganizowane w zależności od miejsca, w którym stawiają zaangażowanie semantyczne. Niektórzy próbują przełożyć każdy wkład na jedną uniwersalną reprezentację. Inni zachowują kilka formularzy kandydujących i wybierają formalizm na zadanie. Niektóre ograniczają język wejściowy, dopóki kompilacja nie będzie deterministyczna. Inni zachowują otwarty język, ale sformalizują tylko obserwacje wymagane przez zapytanie. Każdy projekt rozwiązuje inny problem, a dojrzały system prawdopodobnie połączy je, a nie wybierze wyłącznie.

Uniwersalna pośrednia reprezentacja jest najbardziej intuicyjną architekturą. Tekst jest kompilowany w bogaty wykres semantyczny lub język logiczny, a każda operacja w dół zapyła lub przekształca ten wspólny obiekt. Zaletą są jedność koncepcyjna, ponowne wykorzystanie i możliwość globalnych kontroli spójności. Pojedyncza reprezentacja może obsługiwać wiele aplikacji i tworzyć stabilny interfejs między NLP a rozumowaniem.

Jego słabością jest przesada semantyczna. Aby służyć każdemu zadaniu, reprezentacja musi obejmować jednostki, wydarzenia, dyskurs, czas, modalność, ilość, przyczynowość, normy, niepewność, pochodzenie i pojęcia specyficzne dla domeny. Albo język staje się niezwykle złożony, albo upraszcza rozróżnienia, których potrzebuje jakieś zadanie. Powszechna reprezentacja tworzy również jeden punkt porażki: błędne globalne przeanalizowanie zanieczyszcza każdą analizę. Presja, aby wybrać jedną tożsamość, zakres lub ontologię wcześniej, może ukryć dwuznaczność.

Semantyczny pivot jest skromniejszym wariantem. Nie twierdzi, że jest to kompletny formularz logiczny. Standaryzuje rozpiętości źródłowe, wzmianki, hipotezy podmiotów, obserwacje, światy, wartości nośne, modalność, pochodzenie, alternatywy, pokrycie i luki. Specjalistyczne kompilatory obniżają wybrane części do celów formalnych. Zachowuje to wspólną infrastrukturę, nie udając, że jedno wykonanie semantyki pasuje do wszystkich znaczeń. LongTextJS, jak udokumentowano w nllAgent, jest zbliżony do tego pomysłu: jest to adresowalny świat źródłowy i program obserwacji, a nie uniwersalny język [AssistOS-AI 2026] potwierdzający twierdzenie.

Architektura wielu kandydatów traktuje formalizację jako wyszukiwanie. LLM wytwarza kilka form logicznych, programów lub wykresów semantycznych. Kontrole typu i ograniczenia ontologiczne eliminują nieprawidłowych kandydatów. Egzekucja, przykłady i kontrprzykłady rangi lub rozróżnienia ocalałych. System może zwrócić wniosek konieczny, gdy wszyscy kandydaci się zgadzają i może się to zakończyć, gdy tylko niektórzy to robią. Ta architektura jest odpowiednia, gdy niejednoznaczność jest prawdziwa lub gdy niezawodność formalizacji jest ograniczona.

Jego koszt jest kombinatoryczny. Formy kandydujące mnożą się w różnych zdaniach i wyborach tożsamości. Naiwny produkt krzyżowy staje się niewykonalny. Rozwiązaniem jest zwarte reprezentowanie wspólnej struktury i lokalnych alternatyw, wykorzystanie abstrakcyjnych

domen do ich podsumowania i dopracowanie tylko istotnych dla wyniku wyborów. Zaufanie może kierować eksploracją, ale kandydaci o niskim prawdopodobieństwie nie powinni być usuwani z należytej analizy, chyba że polityka wyraźnie akceptuje probabilistyczną niekompletność.

Kompilacja w języku kontrolowanym przyjmuje odwrotne podejście. Wejście jest przepisywane w ograniczonym języku naturalnym, którego gramatyka i semantyka są deterministyczne lub wysoce ograniczone. Użytkownicy mogą być autorami bezpośrednio w kontrolowanym języku lub LLM może zaproponować kontrolowaną parafrazę potwierdzenia. Po zaakceptowaniu kompilacja jest wiarygodna. Ta architektura jest szczególnie silna w przypadku wymagań, umów, procedur i tworzenia reguł.

Jego ograniczeniem jest zasięg i przyjęcie. Zwykle dokumenty zawierają konstrukcje poza kontrolowanym fragmentem. Przepisanie może zmienić znaczenie. Użytkownicy mogą zatwierdzić parafrazę bez zauważania subtelnej różnicy. Architektura działa najlepiej, gdy kontrolowana forma jest wyświetlana obok źródła, różnice są zlokalizowane, a formalne kontrprzykłady sprawiają, że ważne wybory są konkretne. Język kontrolowany powinien być wyraźnym punktem kontrolnym semantycznym, a nie niewidzialną normalizacją.

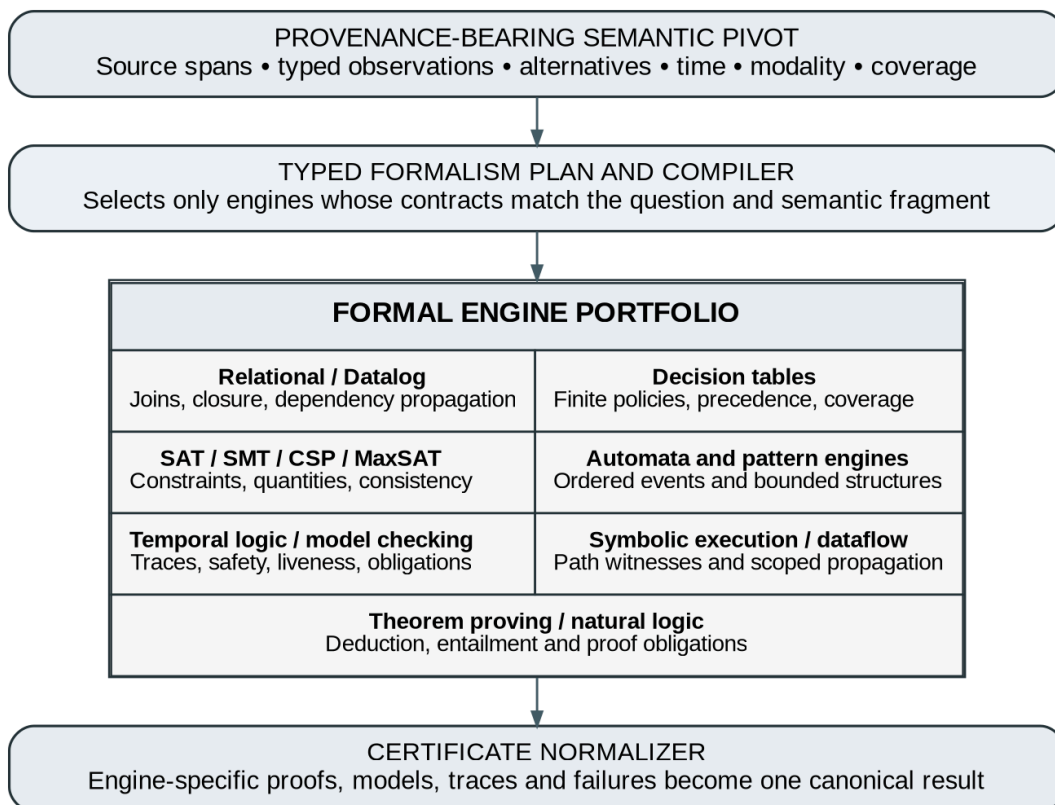
Architektury Query-first pozwalają uniknąć całkowitej formalizacji. Obwód lub pytanie deklaruje, których obserwacji i relacji potrzebuje. Czas działania zachowuje źródło strukturalnie, pobiera odpowiednie regiony i materializuje tylko wymagane typy semantyczne. Relacyjne zapytania, tabele i zasady decyzyjne działają w oparciu o skończony świat. Ten projekt lepiej skaluje się do długich dokumentów i dostosowuje obliczenia do celu.

Ściśle powiązana wizja systemów baz danych 2026 odróżnia zinterpretowanych operatorów semantycznych od tych skompilowanych. W zinterpretowanym projekcie, LLM jest wywoływany wewnątrz pętli przetwarzania danych dla każdego wiersza, rekordu lub pary kandydatów. W skompilowanym projekcie LLM jest wywoływany podczas oddzielnej fazy budowy, aby przełożyć operację semantyczną na deterministyczny kod lokalny, który można wykonywać wielokrotnie bez wywołania modelu. Wstępne eksperymenty wykazały znaczne zmniejszenie opóźnień i wykorzystania modelu przy zachowaniu dużej jakości [Dong and Wang 2026] wyjściowej. Propozycja jest ważna, ponieważ niezależnie dociera do tej samej intuicji architektonicznej opracowanej w tej książce: model językowy jest najbardziej przydatny jako kompilator lub konstruktor planu, podczas gdy powtarzający się czas działania powinien być stabilny i możliwy do sprawdzenia. Jego ograniczenie jest równie pouczające. Wygenerowany kod może skutecznie przybliżać operatora zdefiniowanego modelem bez dostarczenia dowodu semantycznego, że przechwytuje on zamierzoną koncepcję użytkownika. Kompilacja usuwa stochastyczność na sztukę; sama w sobie nie rozwiązuje uziemienia lub wierności specyfikacji.

Ryzyko to fałszywe zamknięcie. Poszukiwanie może przegapić odpowiednie dowody, a analiza popytu może pominąć potrzebną koncepcję. Systemy Query-first muszą śledzić zasięg i producentów. Jeśli negatywny wniosek wymaga przeszukania całego dokumentu pod kątem jakiegokolwiek wyjątku, kilka pobranych fragmentów jest niewystarczających. Architektura powinna odróżniać ograniczony wynik zapytania od gwarancji zamkniętego świata. Kontrakty na zgodność i pokrycie nllAgent są ważnym mechanizmem w tej rodzinie.

Architektura jednoformalizmowa obniża wszystkie zadania do jednej logiki, często logiki pierwszego rzędu, Datalog lub SMT. Upraszcza obowiązki wykonawcze i dowodowe. Dla wąskiej domeny może to być najlepszy wybór. Logic-LM i LINC zyskują znaczną część swojej

wartości ze stabilnego języka docelowego. System planowania może rozsądnie używać jednego języka ograniczeń. Weryfikator protokołu może użyć jednego rachunku procesowego.



Rys. 3. Przełącznik semantyczny nośny pochodzenia można skompilować w portfolio specjalistycznych silników formalnych i znormalizować w jeden wynik kanoniczny.

W przypadku szerokich zadań w języku naturalnym bardziej prawdopodobne jest portfolio solverów. Ocena relacyjna obsługuje zapytania dotyczące dokumentów. Datalog oblicza monotonne zamknięcie. Tabele decyzyjne wyrażają skończone zasady. SAT i SMT obsługują ograniczenia i arytmetykę. Kontrola modelu obsługuje zachowania czasowe. Symboliczne wykonanie konstruuje świadków ścieżki. Logika naturalna obsługuje pewne relacje pociągowe w pobliżu języka powierzchni. Automaty rozpoznają uporządkowane wzory. Semantyczny pivot dostarcza uziemione atomy, a plan formalizmu wybiera jeden lub więcej silników.

Portfel tworzy nowe obowiązki. Router musi znać ekspresyjne i operacyjne możliwości każdego silnika. Kompilator musi zachować znaczenie w przypadku obniżek. Wyniki z różnych silników wymagają wspólnego statusu i modelu pochodzenia. Jeśli dwa silniki powodują nadmierne nakładanie się na siebie, należy sprawdzić spójność lub wyraźnie pozostawić otwartą. Zasady dotyczące zasobów muszą uniemożliwić modelowi wybór niepotrzebnie drogiego solvera.

Przydatny kontrakt formalizmu określa typy wejściowe, wspierane operatory, semantykę, wymagane pokrycie, klasę gwarancji, oczekiwany certyfikat, model kosztów i znane ograniczenia. Silnik czasowo-logiczny może wymagać uziemionych zdarzeń i skończonego układu przejściowego. Silnik SMT może obsługiwać arytmetykę całkowitą, ale nie nieograniczone nieliniowe funkcje rzeczywiste. Tabela decyzyjna może wymagać spełnienia określonych warunków. Silnik naturalno-logiczny może wspierać monotoniczność, ale nie

arbitralne współodniesienie zdań. Router może następnie odrzucić proponowany plan, którego wymagania przekraczają silnik.

Pokolenie dowodzące to kolejna rodzina architektoniczna. Podstawowym artefaktem nie jest odpowiedź w języku naturalnym, ale wynik kanoniczny plus dowód, świadectwo, ślad egzekucji lub możliwe do zweryfikowania pochodzenie. Model językowy generuje wyjaśnienie z tego artefaktu. Każde zgłoszenie wyjaśniające jest powiązane z elementem rezultatu. Niezależny sprawdzający zapewnia, że wyjaśnienie nie przekracza formalnej treści.

Ten projekt jest cenny, ponieważ generowanie i weryfikacja mają różne mocne strony. LLM może napisać jasne wyjaśnienie kontrprzykładu lub niesatysfakcjonującego ograniczenia. Symboliczny wynik może zapewnić, że scenariusz jest realny w ramach przyjętego modelu. Wyjaśnienie może nadal wprowadzać w błąd, jeśli ukrywa założenia lub niepewność, więc wynik kanoniczny musi obejmować te wymiary, a umowa renderująca musi wymagać ich, gdy materiał.

Architektury abstrakcyjno-semantyczne umieszczają abstrakcyjną warstwę interpretacji przed specjalistycznym wykonaniem. Obserwacje kandydujące mogą i muszą mieć domeny. Tania propagacja określa, które nieruchomości są zdecydowanie zadowolone, zdecydowanie niemożliwe lub potencjalnie naruszone. Tylko potencjalne naruszenia są pogrążone w drogich problemach symbolicznych. Zmniejsza to obciążenie solvera i zapewnia pryncypialne nieznanie stany. Tworzy również naturalną architekturę przyrostową do edycji dokumentów.

Architektury z kontrprzykładowymi kierowanymi zamykają pętlę. Nieudany dowód lub fałszywy świadek nie jest tylko błędem; określa, które rozróżnienie semantyczne ma znaczenie. System mapuje fakty solvera do rozpiętości źródeł, prosi LLM lub użytkownika o udoskonalenie odpowiedniej interpretacji i odtwarza analizę. Ta architektura jest szczególnie obiecująca, ponieważ przekształca formalne informacje zwrotne w ukierunkowane rozumienie języka, a nie ogólną autorefleksję.

Architektury edukacyjne regulują sposób, w jaki nowe zasady, ontologie i obwody wchodzą w produkcję. Jedna skrajna pozwala agentowi LLM generować kod wykonywalny podczas każdego uruchomienia. To maksymalizuje elastyczność i minimalizuje pewność. Przeciwna skrajność pozwala tylko na odrębne, formalnie zweryfikowane zasady. Praktyczny środek wykorzystuje środki kodujące w miejscach pracy, generuje punkty odniesienia i mutacje, ogranicza artefakty do form deklaracyjnych i wymaga wyraźnej publikacji. Czas działania produkcji odczytuje niezmiennie wydania. Jest to model zarządzania wdrożony w nllAgent i powinien być uogólniony.

Poniższe porównanie podsumowuje podstawowy kompromis, nie twierdząc, że jeden projekt dominuje w każdym zadaniu.

Tabela 1. Główne rodziny architektoniczne i ich zasadnicze kompromisy.

Architektura	Niezbędna siła i główne ryzyko
Uniwersalna reprezentacja semantyczna	Ponowne wykorzystanie i globalna spójność; ryzykuje nadmierną złożoność i przedwczesne zaangażowanie semantyczne.
Obrót semantyczny plus specjalistyczne obniżki	Zachowuje wspólne dowody i obsługuje wiele silników; wymaga starannego kompilatora i kontraktów międzysilnikowych.
Wielokrotne formalizacje kandydatów	Reprezentuje dwuznaczność i umożliwia może/musić wnioski; ryzykuje równoczesny wzrost.

Architektura	Niezbędna siła i główne ryzyko
Kompilacja w języku kontrolowanym	Silna semantyka i formy sprawdzalne przez użytkownika; poświęca pokrycie i może wprowadzać znaczące zmiany podczas przepisywania.
Materializacja kwerenda-pierwsza	Skale do długich dokumentów i potrzeb związanych z zadaniami; mogą pomylić pobieranie z kompletnością, chyba że zasięg jest wyraźny.
Pojedynczy cel formalny	Prosta baza zaufania i ocena w ograniczonej dziedzinie; słabe dopasowanie do heterogenicznych zjawisk językowych.
Portfolio Solvera	Dopasowuje metodę obliczeniową do struktury problemowej; dodaje obowiązki routingu, interoperacyjności i normalizacji certyfikatów.
Pokonanie przenoszenia dowodów	Ogranicza odpowiedzi w języku naturalnym na zweryfikowane treści; wymaga solidnego sprawdzenia realizacji na poziomie roszczeń.
Analiza abstrakcyjna plus dopracowanie symboliczne	Równoważenie skali i precyzji; solidność pozostaje uzależniona od pokrycia interpretacji.
Rządzone uczone obwody	Pozwala na ewolucję bez samomodyfikacji w czasie wykonywania; zależy od silnych poziomów odniesienia, testowania mutacji i przeglądu publikacji.

Najbardziej wiarygodna architektura ogólna to hybryda opcji środkowych. Wykorzystuje semantyczny pivot nośny pochodzenie, zachowuje alternatywy tam, gdzie mają znaczenie, wykonuje materializację opartą na zapytaniach, trasy do portfela formalizmu i akceptuje tylko wyniki oparte na dowodach lub świadkach. Kontrolowany język może służyć jako interaktywny punkt kontrolny dla formalizacji o dużym wpływie. Abstrakcyjna interpretacja kontroluje skalę, podczas gdy dopracowanie naprowadzane na kontrprzykłady poprawia precyzję. Uczenie się pozostaje poza produkcją.

Wniosek ten jest nie tylko teoretyczny. Znajduje to odzwierciedlenie w literaturze empirycznej. PAL, SatLM, Logic-LM, LINC, translatory czasowo-logiczne, formalizatory planowania, SymGPT i certyfikowane systemy weryfikacji odnoszą sukces, zawężając formalny cel. Żaden wynik nie pokazuje, że jedna uniwersalna logika lub jeden model end-to-end może zastąpić portfel. Architektura powinna zatem obejmować niejednorodność, jednocześnie minimalizując zaufane interfejsy między komponentami.

6.2 Referencyjny pipeline od tekstu do symboli, wykonania i ograniczonych odpowiedzi w języku naturalnym

Potok wykonywalny powinien być rozumiany jako łańcuch narzędzi i narzędzi do weryfikacji, a nie monit agenta. Każdy etap materializuje niezmienny lub adresowany treścią artefakt. Bieg można powtórzyć bez proszenia modelu o zrekonstruowanie jego poprzedniego rozumowania. Błędy są przypisane do etapów. Uczenie się może poprawić producenta lub teorię bez milczenia zmiany znaczenia starych wyników.

Pierwszy artefakt to pakiet źródłowy. Zawiera tekst kanoniczny lub dane, przegląd trawienia, język, strukturę, kanały i stabilne rozpiętości. Akapity, nagłówki, tabele, notowania i bloki kodowe zachowują porządek i tożsamość. Ewidencja zewnętrzna ma podobnie stabilne klucze. Pakiet źródłowy jest autorytetem leksykalnym: ustanawia to, co było obecne, a nie co to znaczy.

Drugim artefaktem jest umowa o zadanie. Stwierdza pytanie, wydanie reguł, zakres źródłowy, pożądaną gwarancję, dozwolone założenia, świat i jurysdykcję, budżety i politykę produkcji. Zadanie określa zapotrzebowanie semantyczne. Prośba o znalezienie nieokreślonych akronimów wymaga różnych obserwacji z prośby o weryfikację zgodności czasowej. Umowa określa również, czy można zawrzeć nieobecność i które formy niepewności wymagają przeglądu przez ludzi.

Trzeci etap konstruuje wykres dowodowy. Deterministyczni producenci dodają obserwacje strukturalne. Parsery w języku kontrolowanym dodają obserwacje dla fragmentów, które rozpoznają. Producenci wspierani przez LLM dodają związane ze schematem wydarzenia kandydujące, relacje, tożsamości, sposoby lub ilości. Każda obserwacja rejestruje kotwicę, producenta, model lub wersję algorytmu, status, alternatywy i zasięg. Model nie zapisuje bezpośrednio na wykresie kanonicznym; materializator ponownie sprawdza rozpiętości, typy i integralność odniesienia.

Tożsamość i zarządzanie światem występują w tej samej warstwie, ale nie są po cichu rozwiązywane. Wzmianki są dowodami miejscowymi. Postawy podmiotów zbierają wzmianki pod światem i zakresem. Kandydaci na tożsamość stwierdzają takie same, różne lub nierozwiązane stosunki z poparciem. Wzajemnie wyłączne interpretacje należą do odrębnych światów lub strzeżonych alternatyw. Wykres może zatem reprezentować niezgodę bez korumpowania globalnej spójności.

Czwarty etap oblicza abstrakcyjny stan semantyczny. Abstrakcje specyficzne dla domeny podsumowują możliwe tożsamości, porządki czasowe, statusy normatywne, ilości i poziomy dowodów. Fakty muszą zawierać się we wszystkich przyjętych światach; mogą fakty zawierać co najmniej jeden. Nieznane rejestruje lukę, a nie probabilistyczne domysły. Stan abstrakcyjny obsługuje tanie propagacje reguł i określa, które pytania wymagają większej precyzji.

Piąty etap tworzy plan formalizmu. Planner nie jest decyzją o swobodnym formularzu LLM. Porównuje zadanie z zarejestrowanymi umowami formalizmu. Proste zapytanie o dokument może pozostać relacyjne. Polityka może zostać opracowana w tabelach i ograniczeniach decyzyjnych. Procedura może zostać złożona w system przejściowy. Naruszenie zależne od ścieżki może wywołać symboliczne wykonanie. Roszczenie dedukcyjne może wywołać twierdzenie twierdzenia. Nazwy planów wymagały wejść, obniżek, wersji silnika, ograniczeń zasobów i oczekiwanych certyfikatów.

LLM może zaproponować plan, ponieważ rozpoznaje strukturę problemu i język domeny. Deterministyczny kontroler potwierdza, że plan wykorzystuje wspierane formalizmy, że każda wymagana obserwacja ma kompatybilnego producenta i że żadne rozróżnienie semantyczne nie jest istotne dla zadania. Na przykład reguła obejmująca wyjątki i priorytet nie może być skompilowana w silnik monotony bez wyraźnego kodowania. Zasada dotycząca tożsamości nie może być kontynuowana, jeśli domena tożsamości jest nierozwiązana, a od niej zależy werdykt.

Szósty etap wykonuje kompilację typowaną. Kompilator ten powinien być mały, deterministyczny i mocno przetestowany. Przekształca obiekty semantyczne w zmienne solver, relacje, przejścia lub predykaty. Mapy źródłowe łączą każdy element formalny z powrotem do obserwacji i rozpiętości. Kompilatory powinny generować zobowiązania dowodowe, gdy obniżenie jest stratne lub zależy od założeń. Różne kompilatory można porównać poprzez mutacje semantyczne i modele wyróżniające.

Siódmy etap to formalne wykonanie. Silniki relacyjne i Datalog zamykają się. Tabele decyzyjne identyfikują dopasowane i sprzeczne wiersze. SAT/SMT/CSP solwery znajdują modele lub udowadniają niewykonalność w ramach swoich wspieranych teorii. Kontrolery modeli oceniają właściwości czasowe. Symboliczni wykonawcy badają warunki ścieżki i zwracają konkretnych świadków. Dowodowi asystenci lub twierdzenia dowodzą, że pochodzą z nich. Wyjście silnika nie jest jeszcze werdyktem skierowanym do użytkownika; jest to rekord wykonania łożysk certyfikacyjnych.

Ósmy etap to weryfikacja powtórki. Oddzielny weryfikator sprawdza świadka lub dowód przeciwko związanemu ze źródłem problemowi formalnemu i opublikowanej teorii. W przypadku znalezienia dokumentu potwierdza, że cytowane obserwacje istnieją, rozpiętości pasują do wersji źródła, statusy spełniają regułę, a ślad wykonania potwierdza roszczenie. Dla wyniku negatywnego potwierdza to, że założenia dotyczące pokrycia i kompletności są wystarczające. Powtórka jest cenna nawet wtedy, gdy zaufany jest bazowy solver, ponieważ potwierdza połączenie między wyjściem kandydackim a wydaną polityką.

Jeśli abstrakcyjna analiza przewidziała naruszenie, ale symboliczne wykonanie nie może zdawać sobie z tego sprawy, rozpoczyna się dopracowanie semantyczne. System wyodrębnia niezaspokojony rdzeń, sprzeczne predykaty lub niedowidzenie. Mapy źródłowe identyfikują odpowiednie rozpiętości i wybory semantyczne. LLM otrzymuje ograniczone zadanie: ponownie rozważyć to łącznie tożsamości, zakres czasowy, mapowanie wyjątków lub ustawienie predykacyjne. Proponuje poprawionych kandydatów; materializator i kompilator ich rewalidują; powtarza się wykonanie. Wnioskuje się o przegląd człowieka, gdy rozróżnienie ma duży wpływ lub pozostaje nierozwiązane.

Ta pętla powinna być monotonna w odniesieniu do dowodów, a nie tylko pewności siebie. Udoskonalenie może podzielić jednostkę, zwęzić przedział czasowy, dodać pomijany wyjątek lub zaznaczyć nieobsługiwane założenie. Nie powinien on aktualizować proponowanej obserwacji do faktów, ponieważ nowy wynik jest wygodny. Każda zmiana jest zapisana jako nowa rewizja semantyczna. Poprzednie ścieżki pozostają powtarzalne.

Dziewiąty artefakt jest wynikiem kanonicznym. Zawiera on tożsamość reguły lub zapytania, akceptowany status, założenia, musi i może wnioski, odrzucone kandydaty, niewiadome, zasięg, dowody źródłowe, odniesienia do dowodu lub świadka oraz granice egzekucji. Może również zawierać zwięzły wykres wyjaśnień do sprawdzenia maszyny. Ten artefakt jest autorytatywny nad każdym renderowaniem prozy.

Dziesiąty etap realizuje język naturalny. Renderer otrzymuje tylko wynik kanoniczny i wybrane fragmenty źródła. Nie powtarza pierwotnego rozumowania. Każde wygenerowane zdanie jest związane z jednym lub większą liczbą roszczeń wynikowych. Warcab porównuje zdanie z dozwolonymi roszczeniami, zapewnia zachowanie modalności i niepewności oraz odrzuca nieuzasadnione dodatki. Wynik może rozróżnić ustalenia, założenia, interpretacje i zalecenia.

Na przykład poprawna odpowiedź może brzmieć: „Pod interpretacją, że oba odniesienia do Zarządu oznaczają ten sam organ, reguła jest naruszana, ponieważ zatwierdzenie poprzedza zarejestrowany przegląd bezpieczeństwa. W dokumencie nie ustalono, czy odniesienia oznaczają ten sam organ; w związku z tym ogólny wynik jest wymagany do przeglądu.” Płynny, ale niebezpieczny renderer może pominąć stan i ogłosić naruszenie. Wynik kanoniczny zapobiega temu uproszczeniu.

Gazociąg wspiera również wytwarzanie, nie tylko kłębkowanie. Użytkownik dostarcza pomysł lub cel. Obwody planowania konstruują kontrolowaną specyfikację ze świadkami typu „reputacja-plan”. LLM realizuje projekt. Obwody audytu analizują projekt przy użyciu tych samych dowodów i ścieżki weryfikacji. Zmiana trwa do czasu zakończenia audytu kanonicznego, który przewiduje politykę uwolnienia lub wyczerpanie budżetu. Model generacji nigdy nie staje się ostateczną wyrocznią dla tego, czy jego własny tekst spełnia zasady.

Zarządzanie otacza łańcuch narzędzi. Ontologie, producenci, obwody, kompilatory, kontrakty formalizmu i rendery mają wersje i strawy. Agenci uczący się działają w jednorazowych obszarach inscenizacji. Publikacja wymaga wskaźników referencyjnych, mutacji semantycznych, kontroli zgodności i wyraźnego zatwierdzenia. Produkcja odczytuje niezmiennie wydania. Profile modeli na żywo są oceniane oddzielnie, ponieważ oprawy deterministyczne nie określają jakości dostawcy. Pakiety wiedzy w bieżącym świecie mają daty wejścia w życie i jurysdykcje.

Architektura powinna obnażać hierarchię gwarancji. Gwarancje strukturalne dotyczą dokładnych rozpiętości, uporządkowania i ważności schematu. Uziemienie gwarantuje dochodzenie w zakresie wyrównania dowodów i statusu tożsamości. Egzekucja gwarantuje konsekwencje wystąpienia problemu formalnego. Kompleksowe gwarancje łączą je warunkowo. Użytkownik powinien być w stanie zobaczyć najśłabszą warstwę. Wynik pochodzący z zdarzeń proponowanych przez model i przejścia zweryfikowanego mechanicznie nie powinien być określany po prostu „zweryfikowany”; powinien stwierdzać, że wykonanie jest zweryfikowane w związku z proponowanymi obserwacjami.

Małe zaufane jądro jest głównym celem projektowym. Jądro obejmuje kanonizację źródła, schemat i sprawdzanie typów, formalne kompilatory, zarejestrowane wykonawcy lub sprawdzanie dowodów, weryfikatory powtórek i walidację wyników kanonicznych. LLM monity, generatory kandydatów, heurystyki pobierania i modele wyjaśnień pozostają na zewnątrz. Zaufane komponenty mogą nadal zawierać błędy, ale ich zachowanie jest stabilne, możliwe do przetestowania i możliwe do przejrzania. Jest to znaczna poprawa w stosunku do zaufania do rozwijającego się modelu fundamentu jako monolitycznego rozumnika.

Proponowany rurociąg nie jest jedynym możliwym wdrożeniem, ale wychwytyuje obowiązki ujawnione przez literaturę. To wyjaśnia, dlaczego rozwiązanie jest konieczne, ale niewystarczające. Daje abstrakcyjną interpretację i symboliczne wykonanie odrębnych ról. Obsługuje wiele języków formalnych bez porzucania wspólnej warstwy dowodowej. Pozwala LLM na kreatywność w odkrywaniu, naprawie i wyjaśnieniu, zachowując jednocześnie

symboliczną władzę nad uziemieniem i akceptowanymi rezultatami. Co najważniejsze, zamienia porażkę w nazwany artefakt - nieobsługiwany typ, nierozwiązaną tożsamość, niepełne pokrycie, utratę kompilacji, niesatysfakcjonującą formalizację lub nielicencjonowane zdanie - zamiast niejasnego halucynacji.

7. LongTextJS, CircuitJS i nllAgent: krytyczne studium przypadku

Publiczne repozytorium NaturalLanguageLinterAgent jest jedną z niewielu prób wyraźnej granicy między interpretacją językową a wykonywaniem reguł w implementacji wykonywalnej. Projekt nie jest rozumnikiem w języku ogólnoustrojowym, a jego dokumentacja nie przedstawia go jako jednego. Jest to biblioteka Node.js 22+ ESM i czas pracy wiersza poleceń z dwoma trybami produktu w kontrolowanym języku w tej samej LongTextJS–CircuitJS architekturze. Tryb audytu stosuje teorię wersjonowaną do istniejącego dokumentu Markdown i emituje kanoniczny produkt audytu. Tryb specyfikacji stosuje teorie planowania do pomysłu na wysokim poziomie i emituje zweryfikowany plan generowania; opcjonalny model językowy może zdać sobie sprawę z tego planu jako prozy, ale niezmiennie obwody audytu pozostają wyrocznią [AssistOS-AI 2026] akceptacyjną.

Repozytorium jest przydatne jako studium przypadku, ponieważ ucieleśnia centralną tezę tej książki w formie oprogramowania: modele mogą proponować obserwacje semantyczne lub prozę, ale organ produkcyjny jest przypisany do niezmiennych wydań, wyraźnych kontroli zgodności, ograniczonego wykonywania obwodów, weryfikatorów i artefaktów kanonicznych. Projekt prowadzi również publiczny rejestr poważnych problemów. Ta praktyka ma znaczenie, ponieważ duża powierzchnia dokumentacji może w inny sposób sprawić, że intencje architektoniczne będą wyglądać jak wdrożone gwarancje. Poniższa dyskusja wyróżnia to, co jest poprawnie ustalone od tego, co pozostaje hipotezą badawczą.

Żadna niezależna ocena recenzowana lub przegląd techniczny nllAgent, LongTextJS lub CircuitJS nie został zlokalizowany poprzez przeszukanie nazw projektów, nazwy repozytorium, dokumentacji URL i kombinacji z „przeglądem”, „oceną”, „benchmark” i „replikacją” z dnia 2 sierpnia 2026 r. Wyniki wyszukiwania doprowadziły do samego projektu, a nie do oceny zewnętrznej. Wyrok w tym rozdziale jest zatem niezależnym przeglądem architektonicznym autora, opartym na publicznym repozytorium, dokumentacji, przykładach wykonywalnych, deklarowanym procesie wydania i aktualnych aktach poważnych problemów. Brak przeglądu zewnętrznego nie jest ani pochwałą, ani krytyką; oznacza, że roszczenia projektowe powinny pozostać wąskie, dopóki niezależnie się nie odtworzy.

7.1 Co wdraża architektura publiczna i dlaczego jej granice mają znaczenie

System definiuje trzy działania produkcyjne, które są celowo rozdzielone. Audyt produkcji odczytuje niezmiennie opublikowane wydanie, kompiluje dokument w LongTextJS, wykonuje obwody walidacyjne wydania, weryfikuje wyniki kandydujące i utrzymuje kanoniczny CNL / Audit-1 wynik z renderowanym raportem. Specyfikacja produkcyjna kompiluje pomysł w obserwacji LongTextJS, stosuje obwody planowania i wytwarza kanoniczny artefakt CNL / Plan-1; opcjonalna realizacja może następnie wygenerować Markdown i przesłać go do tej samej maszyny audytu. Uczenie się odbywa się w jednorazowej przestrzeni roboczej. Agent kodowania może generować artefakty i benchmarki autorów kandydatów, ale kontrole po stronie hosta przyjmują tylko pliki z białej listy, a opiekun musi wyraźnie opublikować i aktywować wydanie. To sprawia, że uczenie się jest analogiczne do zarządzania rozwojem oprogramowania, a nie samomodyfikacją czasu pracy.

Najważniejszą jednostką pracy nie jest raport końcowy, ale katalog biegów. Audyt zachowuje skopiowane dane wejściowe, program źródłowy, wybraną podstawę, wyniki w zakresie zgodności i zasięgu, śledzenie obwodów, wyniki weryfikacji, ustalenia, kanoniczny raport do odczytu maszynowego i widok czytelny dla człowieka. Planowanie działa, aby zachować odpowiednie artefakty planu. Bieg, który zatrzymuje się, ponieważ brakuje wiedzy, nadal rejestruje awarię i może stworzyć problem wielokrotnego użytku. Zasada projektowania, że brakująca wiedza nigdy nie jest zgłaszana jako zgodność, jest zasadnicza. Zapobiega powszechnej awarii agentów dokumentów: przekładając „nie znalazłem dowodów” na „dokument spełnia regułę”.

LongTextJS należy rozumieć jako program po stronie źródła tylko w ograniczonym sensie. Nie przypisuje zachowania imperatywnego do arbitralnych zdań. Kompiluje stałą rewizję źródła w skończony, dyskusyjny świat struktur i obserwacji. Cennymi właściwościami są tożsamość źródłowa, stabilne rozpiętości, wersjonowane typy obserwacji, tożsamość producenta, pokrycie, wyraźne luki i regrywalność. Artefakt po stronie dokumentu może być wykonany, ponieważ zarejestrowani konsumenci mogą odpytywać i przekształcać te relacje w ramach deklarowanej semantyki; jest on bliższy semantycznej reprezentacji pośredniej lub programu ładowania bazy danych niż nieograniczonemu kodowi aplikacji.

To rozróżnienie to coś więcej niż terminologia. Kiedy model tłumaczy zdanie, jego wynik nie powinien stać się autorytatywny tylko dlatego, że jest serializowany w języku programowania. Tekst źródłowy pozostaje dowodem; przetłumaczona obserwacja rejestruje jedną interpretację tych dowodów, wygenerowaną przez nazwanego producenta pod schematem i profilem modelu. Obwód pozostaje odrębną teorią. Odkrycie staje się możliwe do opublikowania dopiero po tym, jak weryfikator sprawdzi warunki oczekiwane przez wydanie. Warstwy te uniemożliwiają parserowi ciche stanie się sędzią i zapobiegają wyjaśnieniu, aby stał się jedynym zapisem wnioskowania.

Obecne opakowanie fundamentowe ilustruje zamierzony zakres semantyczny. Polecenia zwykłe audytu, planu i benchmarku wybierają ograniczony fundament-core@1.1.0, chyba że są wyraźnie wyłączone. Pakiet rozpoznaje udokumentowane kontrolowane-angielskie formy dla stanu, typu, dokładnego precedensu czasowego wyrażonego przez „przed”, dokładne twierdzenia arytmetyczne, ilościowe i dosłowne emocje. Pięć obwodów zweryfikowanych powtórnie sprawdza tylko opublikowane niezmienniki logiczne, czasowe, arytmetyczne,

elementarne fizyczne i emocjonalne / typowe. Wybrane opakowanie i przetrawienie są utrzymywane. Zmieniające się fakty polityczne, geograficzne, społeczne, ekonomiczne i prawne są celowo wykluczane z ponadczasowej prawdy fundamentowej. To jest możliwa do obrony granica modelu światowego. System, który koduje aktualne fakty bez skutecznych dat, jurysdykcji, źródeł i polityki konfliktu, nie może oferować powtarzalnych egzekucji semantycznych.

System wspiera deterministyczne i wspomagane modelem materializację. Wybór tłumaczenia może preferować skonfigurowany agent modelu językowego AchillesAgentLib, używać bieżącego ograniczonego adaptera kodowania lub uruchamiać bez modelu dla czynników deterministycznych. Każde wywołanie modelowe otrzymuje umiejętność specyficzną dla ról, a nie całą przestrzeń roboczą agenta. Jeśli krytyczna obserwacja nie może zostać zmaterializowana w ramach wymaganego schematu i budżetu zasobów, wynik jest niezgodny, niekompletny lub wyczerpany budżetem. Model nie kontroluje budowy źródła, polityki publikacji ani znaczenia weryfikatora. Jest to dokładnie właściwy kierunek dla federacyjnego systemu neuro-symbolicznego: model jest potężny na krawędzi, podczas gdy symboliczne artefakty i akceptacja kontroli polityki gospodarza.

Zasięg jest traktowany jako obowiązek wykonywalny, a nie nieformalny wynik zaufania. Kompilacja obwodów działa wstecz od węzłów osiągalnych w celu określenia, które obserwacje zewnętrzne mogą mieć wpływ na wynik. Umowy o rejestrach publikacji łączące te żądania z dostępnymi producentami. Kompatybilność z czasem pracy pyta następnie, czy konkretny dokument faktycznie przyniósł obserwacje z wymaganym typem, statusem, kardynalnością, strukturą, kanałem i zasięgiem. Deklarowana zdolność producenta nie jest mylona z dowodami na to, że obecny dokument został pomyślnie przetworzony. Kiedy krytyczne zapotrzebowanie jest niezaspokojone, bieg zatrzymuje się na nazwanej granicy.

Ta konstrukcja oparta na zapotrzebowaniu jest dobrze dopasowana do długich dokumentów. Unika proszenia modelu o stworzenie uniwersalnej interpretacji każdego zdania, zanim jakiegokolwiek pytanie zostanie poznane. Obwód może żądać potrzebnych wymiarów semantycznych: dokładnych terminów, twierdzeń stanu, ilości, relacji czasowych lub obserwacji domeny. Kompilator może materializować tylko obserwacje istotne dla wyjścia i utrzymywać to, co było, a co nie było objęte. Zaletą naukową jest nie tylko niższy koszt. Ogranicza niepotrzebne zobowiązania semantyczne i sprawia, że niepełna formalizacja jest widoczna.

CircuitJS jest reprezentacją po stronie teorii. Moduły obwodów edytowalnych są ograniczone do jednego wyrażenia deklaratywnego, znormalizowanego do zwykłych danych i przepuszczone przez ten sam kompilator używany do opisów obwodów niekodowych. Mogą wywoływać tylko zarejestrowane operatory i weryfikatory; nie mogą ukrywać arbitralnego zachowania wykonywalnego w wyuczonej regule. Aplikacja hosta może zainstalować jedno sprawdzone rozszerzenie runtime zawierające rzeczywiste algorytmy, ale wejścia i kontekst są zamrożone w zwykłych danych, wyniki muszą być prostymi danymi, tożsamość pamięci podręcznej obejmuje przetrawienie rozszerzenia i wydania, które zależą od blokady rozszerzenia, która trawi. Standardowy interfejs wiersza poleceń nie akceptuje dowolnych wykonywalnych ścieżek rozszerzeń.

Granica ta dotyczy jednego z najpoważniejszych zagrożeń praktycznych w systemach reguł „uporządkowanych agentami”. Jeśli agent kodujący może wygenerować dowolne JavaScript w każdym obwodzie, nauka litery staje się zdalnym generowaniem kodu, a semantyczne zachowanie wydania staje się trudne do sprawdzenia. Ograniczenie zwykłego tworzenia

obwodów do zarejestrowanych operacji tworzy skończone słownictwo operacyjne. Specjalistyczne algorytmy pozostają możliwe, ale wchodzi przez oddzielną ścieżkę inżynierii, przeglądu i wersjonowania. Architektura wyróżnia zatem ewolucję teorii od ewolucji zaufanego środowiska wykonawczego.

Równie ważna jest granica weryfikatora. Wykonanie obwodu może tworzyć ustalenia kandydatów, ale weryfikator kontroluje, czy kandydaci ci wchodzi do audytu kanonicznego lub planu. W ograniczonych przykładach weryfikator może ponownie odczytać kotwice źródłowe, obserwacje, metadane wydania i zastosować politykę. Nie wystarczy, że węzeł wykonawczy emituje obiekt, którego pola wyglądają jak ostrzeżenie. Weryfikator ustala określoną właściwość, o której oczekuje produkt. Nie jest to automatycznie udowodnione, że semantyczna interpretacja jest poprawna, ale powstrzymuje przypadkowe lub złośliwe wyjście obwodu przed maskowaniem jako akceptowane odkrycie.

Tryb specyfikacji pokazuje, w jaki sposób ta sama architektura może wspierać kontrolowane generowanie bez przyznawania autorytetu modelowi pisarza. Obwody planowania budują plan kanoniczny i wymagają świadków zgodnie z zasadami stosowania. Opcjonalna realizacja przekształca plan w prozę. Proza kandydująca przechodzi następnie przez tryb audytu, a model realizacji nie ocenia własnej zgodności. Jest to silniejsze niż szybkie, ograniczone generowanie, ponieważ autorytatywnym celem jest plan i ślad audytu, a nie samozwańcze przestrzeganie modelu.

Znaczący jest również eksperymentalny podzbiór kwerend. Canonical LongTextJS jest eksponowany jako skończony świat zapytań tylko do odczytu. Wpisane zapytania i wartości tabeli świadomej dowodów są obniżone do zwykłego wykonywania CircuitJS. Kompilator sprawdza wspiera relacje, wyrażenia, anty-joins, zamawianie i politykę trafień; umowy o zapytania dotyczące publikacji i mapy źródłowe; benchmarki porównują bezpośrednią ocenę referencyjną z obniżoną realizacją. Obecne wydanie redakcyjne produkcji pozostaje celowo prostsze, podczas gdy agregaty, uporządkowane wzorce, natywne indeksy i zaawansowane rozumowanie są obsługiwane za pośrednictwem bezpośrednich operatorów lub wykresów. Ta powściągliwość jest lepsza niż uzyskanie dojrzałego uniwersalnego języka zapytań, zanim semantyka i optymalizacja będą stabilne.

Model publikacji repozytorium wzmacnia odtwarzalność. Nazwani agenci są właścicielami oddzielnego materiału autorytetu, schematów, profili ekstrakcji, obwodów, pakietów porównawczych, kandydatów, wydań, biegów, problemów i informacji zwrotnych. Agent kodowania może zaproponować kandydatów, ale nie może ich aktywni. Produkcja odczytuje niezmiennie wydanie. Ta struktura umożliwia odtworzenie, która teoria, podstawa, profil producenta, przekop rozszerzenie i weryfikator wygenerowały wynik. Pozwala również kilku agentom kodowania lub modelować rodziny budować konkurencyjne teorie bez milczenia dzielenia się aktywną ontologią lub uwolnieniem.

Najmocniejsze zaimplementowane twierdzenie jest zatem architektoniczne, a nie semantyczne: nllAgent ma zdyscyplinowaną ścieżkę od niezmiennego źródła do wersjonowanych obserwacji, od zapotrzebowania na obserwacje do zgodności, od obwodów zastrzeżonych po publikację kontrolowaną przez weryfikatora oraz od produktu kanonicznego do raportu czytelnego dla człowieka. Ma wyraźne stany zatrzymania. Nie trzeba udawać, że każdy dokument jest ontologicznie kompatybilny. Traktuje wyniki modelowe jako propozycje związane z rolą. Są to znaczące osiągnięcia w porównaniu z konwencjonalnymi rurociągami

LLM-agent, w których szybki, interpretacyjny, rozumowanie i wyjaśnienie często istnieją tylko w jednym przejściowym kontekście.

Architektura nie stwierdza jednak, że język naturalny został prawidłowo ugruntowany. Stabilne rozpiętości dowodzą, że dowody istnieją w miejscu; nie dowodzą, że aktor, wydarzenie, wyjątek lub modalność zostały zinterpretowane prawidłowo. Wpisane obserwacje mogą nadal odnosić się do niewłaściwego podmiotu. Kompletny bieg może być kompletny tylko dla umowy obserwacji ograniczonej, a nie dla każdego istotnego znaczenia, jakie człowiek mógłby wnioskować. Własna dokumentacja projektu i rejestr emisji są w tym punkcie odpowiednio ostrożne. Wartość wdrożenia polega na tym, że te ograniczenia mają miejsca, które należy reprezentować, a nie być ukryte w ostatecznej odpowiedzi.

7.2 Niezależna ocena: mocne strony, aktualne poważne problemy i wymagana ewolucja

Moja ogólna ocena jest pozytywna co do granic zaufania i ostrożna w stosunku do roszczeń semantycznych. nllAgent to coś więcej niż owijarka błyskawiczna, więcej niż potok odzyskujący wektor i więcej niż zbiór schematów JSON. Jego artefakty po stronie źródła i teorii, niezmiennie publikacje, ograniczona granica autorska, weryfikacja powtórek, kontrole pokrycia i produkty kanoniczne stanowią wiarygodny substrat badawczy. Architektura jest szczególnie dobrze zgodna z wzorcem federacyjnym zidentyfikowanym w ostatnich neuro-symbolicznych recenzjach: komponent neuronowy proponuje lub realizuje język, podczas gdy komponent symboliczny zachowuje własne kryteria tożsamości, semantyki i akceptacji [Chatzikyriakidis and Lappin 2026].

Obecne dowody projektu potwierdzają jednak ograniczoną wykonalność, a nie ogólny możliwy do wykonania język naturalny. Domyślny fundament rozpoznaje mały kontrolowany fragment. Demonstracja redakcyjna wykorzystuje wąskich operatorów i weryfikatorów. Kompilacja Query-first jest eksperymentalna. Kontrolowana realizacja jest ograniczona. Żadna niezależna ocena nie ustanowiła wierności formalizacji, rozdzielczości tożsamości, transferu między domenami, kalibracji błędów lub przewagi nad prostszymi wartościami bazowymi. Właściwą strategią publikacji jest zatem określenie, które właściwości są wdrażane, oraz dołączenie każdego silniejszego roszczenia do poziomu odniesienia i puli gwarancji.

Obecny rejestr poważnych problemów identyfikuje sześć ograniczeń inżynierskich i semantycznych. Pierwszym z nich jest ograniczona stopniowość harmonogramu. Czas działania ma użyteczne mechanizmy zależności i buforowania, ale nie twierdzi jeszcze, że jest to w pełni ogólny przyrostowy silnik punktów stałych dla arbitralnych obwodów, rekursywnych relacji, alternatywnych światów i aktualizacji nie-monotonowych. Ma to znaczenie dla długich dokumentów, ponieważ niewielka edycja powinna unieważnić tylko artefakty semantyczne i stany obwodu, które są od niego zależne. Naiwna pełna powtórka jest poprawna, ale droga; niezdrowy przyrostowy harmonogram jest szybki, ale niebezpieczny. Rozwiązanie wymaga wyraźnych umów monotonicznych i efektowych, pochodzenia zależności, separacji epok dla skrótów i porównań referencyjnych między przyrostowym a pełnym wykonaniem.

Druga kwestia polega na tym, że wsparcie interpretacyjne nie przepływa jeszcze jednolicie przez każdą ścieżkę wykonania. Dokument może zawierać konkurencyjne odczyty, światy, konteksty lub warunki obecności, ale operator, klucz pamięci podręcznej, agregacja lub weryfikator mogą je przypadkowo złamać. Jest to główna przeszkoda w traktowaniu LongTextJS jako abstrakcyjnego stanu semantycznego. Każda wartość, która zależy od interpretacji, musi mieć swój kontekst. Łączenia muszą łączyć kompatybilne konteksty. Agregacja musi powiedzieć, czy obejmuje jeden świat, czy alternatywy. Tożsamość podręczna musi zawierać formułę kontekstu. Operatorów, którzy nie mogą zachować alternatyw, należy uznać za niezgodne, a nie po cichu wybrać jedną.

Ten problem nie jest niewielką luką. Podstawową obietnicą naukową możliwego do wykonania języka naturalnego jest często to, że system może powiedzieć, co jest prawdą w przypadku wszystkich dopuszczalnych odczytów, co jest możliwe w co najmniej jednym, a który wynik zależy od spornej interpretacji. Jeśli alternatywy zostaną utracone w wewnętrznej trasie wykonania, wypolerowany interfejs may / must będzie wprowadzający w błąd. Projekt powinien zatem sprawić, że zachowanie interpretacji stanie się niezmiennikiem całego środowiska i

przetestować go poprzez wygenerowane światy skończone, właściwości metamorficzne i powtórkę różnicową.

Trzecia kwestia to konserwatywna zgodność z nieprzejrystym zachowaniem proceduralnym. Zarejestrowane rozszerzenia i operatory są konieczne, ponieważ niektóre użyteczne algorytmy są niezręczne do wyrażenia deklaratywnie. Ale kompilator nie może wnioskować o precyzyjnych wymaganiach semantycznych, efektach, monotoniczności, wykorzystaniu zasobów lub zobowiązaniach świadków z nieprzezroczystych JavaScript. Konserwatywne odrzucenie jest bezpieczniejsze niż optymistyczna akceptacja, ale nadmierna nieprzezroczystość zmniejsza współczulność i sprawia, że materializacja oparta na popycie jest mniej skuteczna. Każdy procesowy prymitywny potrzebuje kontraktu na własność rejestru, w którym deklaruje typy wejściowe i wyjściowe, akceptowane statusy epistemiczne, wymagania dotyczące pokrycia, możliwe efekty, determinizm, monotoniczność, zachowanie pamięci podręcznej, klasę kosztów i czytelny świadek. Umowa powinna być bardziej autorytatywna niż komentarze napisane przez autora rozszerzenia.

Czwartą kwestią jest minimalne planowanie zapytań i indeksowanie czasowe. Skończony świat zapytań może działać poprawnie bez zaawansowanej optymalizacji, ale użycie w skali dokumentów będzie wymagało planowania połączeń, szacunków selektywności, indeksów rozpiętości źródła i podmiotów, wskaźników interwału czasowego i konserwacji przyrostowej. Te optymalizacje to nie tylko szczegóły wydajności. Planer, który przepisuje zapytania, musi zachować wielość dowodów, formuły kontekstu, porządkowanie, semantykę anty-join i zasięg. Indeksy natywne muszą pozostać jednorazowymi akceleratorami, a nie źródłami prawdy. Testy różnicowe powinny porównywać zoptymalizowaną i referencyjną ocenę nad generowanymi światami, szczególnie w przypadku zapytań o absencję, nakładania się czasu, interpretacji alternatywnych i zduplikowanych dowodów.

Piąta kwestia jest celowo wąskim kontrolowanym pokoleniem. Dokładna lub sprawdzona realizacja ograniczonych ustaleń i planów jest cenna, ale nie jest jeszcze równoznaczna ze zweryfikowaną długoformatową prozą. Wieloletnie generowanie wprowadza agregację, zaimki, elipsę, retoryczny nacisk, relacje dyskursu i implikacje stworzone przez kolejność zdań. Realizacja może zawierać tylko licencjonowane stwierdzenia atomowe i nadal implikować nieuzasadnioną przyczynowość lub normatywną narrację. Następnym krokiem powinno być raczej doświadczenie-niesienie niż szerszy nieskrępowany pisarz. Wynik kanoniczny powinien dostarczać identyfikatory roszczeń, miejsca na jednostki i ilości, sposób, niepewność i dozwolone relacje dyskursu. Wygenerowany tekst należy ponownie przeanalizować i porównać z tymi twierdzeniami. Krytyczne niedopasowania powinny odrzucić tekst; materiał stylistyczny poza sprawdzanym fragmentem powinien być oznakowany na niższym poziomie zapewnienia.

Szóstą kwestią jest specyficzne dla scenariusza wsparcie mutacji. Zwykła dokładność porównawcza może ukryć reguły, które są obojętne, zbędne lub nieprawidłowe. Obwód może przekazać wszystkie wybrane przykłady, nawet jeśli wyjątek zostanie zignorowany, ponieważ żadna obudowa go nie aktywuje. Poważna ocena powinna generować typowane mutanty semantyczne, które zmieniają progi, porównawcy, kwantyfikatory, negacje, modalność, powiązania tożsamości, kolejność czasową, status dowodów, przewidywania pokrycia, zakres wyjątków, priorytet reguł i trasy weryfikatorów. Zestaw testowy „zabija” mutantą, gdy oczekiwany wynik kanoniczny zmienia się w wymagany sposób. Przetrwali mutanci ujawniają słabe testy lub semantycznie zbędne klauzule. Mutacja powinna obejmować programy źródłowe, teorie obwodów, obniżki kompilatorów i ograniczenia realizacji, a nie tylko ciągi w przykładach.

Te sześć problemów jest bardziej pouczających niż ogólna etykieta „prototypowa”, ponieważ identyfikuje, gdzie obecna architektura może stracić skalę lub znaczenie. Proponują także spójną ścieżkę ewolucji. Pierwszym kamieniem milowym powinna być ograniczona formalna semantyka dla sklepu semantycznego i świata zapytań. Nie musi formalizować nieograniczonego angielskiego. Powinien on definiować niezmiennie światy źródłowe, tożsamość obserwacji, status epistemiczny, zasięg, alternatywy, transakcje, wyniki zapytań, publikację i wycofanie. Kluczowe inwarianty mogą być sprawdzane modelowo w stosunku do skończonych instancji i testowane pod względem właściwości w implementacji.

Drugim kamieniem milowym powinno być wykonanie świadome interpretacji. Warunki obecności lub konteksty faktoryzowane powinny towarzyszyć obserwacjom uzależnionym od alternatyw. Harmonogram, silnik zapytań, prymitywny rejestr, pamięć podręczna, weryfikator i produkty kanoniczne powinny zachować te konteksty lub zadeklarować niezgodność. Może, musi, nie może i nieznane powinno być obliczane z wyraźnych zbiorów lub abstrakcji dopuszczalnych światów, a nie z modelowej pewności siebie. Stworzyłoby to konkretny most z istniejącej architektury do abstrakcyjnej interpretacji.

Trzecim kamieniem milowym powinien być raczej rejestr formalizmu niż bezterminowo rozszerzający CircuitJS. CircuitJS najlepiej stosować jako język orkiestracji, dowodów i publikacji. Specjalistyczne silniki powinny posiadać specjalistyczną semantykę: relacyjną i Datalog ocenę zamknięcia i połączeń; tabele decyzyjne dotyczące pierwszeństwa polityki; SAT, SMT, CSP lub MaxSAT dla ograniczeń; automaty dla uporządkowanych wzorców; czasowe solvery i sprawdzanie modeli dla śladów; symboliczne wykonanie dla świadków ścieżki; ramy przepływu danych dla propagacji; Każdy adapter powinien zadeklarować wpisany fragment wejściowy, założenia tłumaczenia, certyfikat wyjściowy, sprawdzanie powtórek i tryby awarii.

Czwartym kamieniem milowym powinno być kontrprzykładowe dopracowanie semantyczne. Zgrubna abstrakcyjna analiza może zidentyfikować możliwe naruszenie. Rozwiązywacz lub symboliczny wykonawca szuka następnie spójnego świadka. Jeśli świadek jest zadowolony i możliwy do ponownego odtwarzania, system zgłasza to z rozpiętością źródłową. Jeśli jest to fałszywe, niesatysfakcjonujący rdzeń lub konflikt powinien określić, która tożsamość, czas, modal, ilość, wyjątek lub rozróżnienie pokrycia zostały utracone. Wniosek o dopracowanie powinien być skierowany wyłącznie do odpowiedniego źródła i producenta. Naprawiony artefakt semantyczny staje się nową wersją, a kontrprzykład staje się przypadkiem regresji. Ta pętla byłaby prawdziwie nowatorską syntezą propozycji semantycznej opartej na LLM, abstrakcyjnej interpretacji i symbolicznego wykonania.

Piątym kamieniem milowym powinna być silniejsza warstwa uziemienia. Bezpieczeństwo typu i kotwice źródłowe są konieczne, ale niewystarczające. Runtime potrzebuje zestawów tożsamości kandydatów, wyraźnych tych samych / różnych ograniczeń, terminowej ważności, kontroli jednostki i jurysdykcji, autorytetu źródła i polityki konfliktu. Wygenerowane przez model linki top-one nie powinny destrukcyjnie łączyć obiektów. Krytyczne podmioty powinny używać autorytatywnych identyfikatorów lub ludzkiego potwierdzenia. Wiedza zewnętrzna powinna zawierać tylko w czynnikach, pakiety związane z treścią ze źródłami, efektywne odstępy czasu, jurysdykcję, wygaśnięcie, rozwiązywanie konfliktów i pułapy gwarancji. Web retrieval może zaproponować aktualizacje pakietów, ale nie powinno stać się niejawną ponadczasową prawdą.

Szóstym kamieniem milowym powinna być niezależna ocena. Oprawy deterministyczne ustanawiają odtwarzalność oprogramowania, ale nie jakość tłumaczenia semantycznego na żywo. Profil oceny opt-in powinien przypiąć model, dostawcę, podpowiedzi, wersje narzędzi,

politykę temperatury i przechwytywania; powtarzać przypadki w celu pomiaru wariancji; rekord kosztów i latencji; i zabraniać cichego awaryjnego zwrotu. Odrębnie zaimplementowany oceniający odniesienia powinien porównać wyniki zapytań i obwodów tam, gdzie jest to możliwe. Adnotacja człowieka powinna koncentrować się na wierności od źródła do semanty i niejasnościach istotnych dla rezultatu. Linie bazowe powinny obejmować bezpośredni LLM osąd, LLM plus pobieranie, LLM-to-one-formalizm, kompilację w kontrolowanym języku i deterministyczne systemy wzorców. Projekt powinien wykazać, kiedy jego dodatkowa architektura wytwarza mierzalną wartość.

Użyteczną ocenę można podsumować bez zawyżania twierdzeń. Zachowanie źródła, niezmiennie wydania, utrzymywanie się śladu, oddzielenie obserwacji dokumentów od teorii wielokrotnego użytku, wyraźny zasięg, ograniczone tworzenie obwodów, publikacja kontrolowana przez weryfikatora i uczenie się regulowane są silnymi decyzjami projektowymi. Ogólne zrozumienie semantyczne, wykonanie kompletne, planowanie zapytań na skalę produkcyjną, długotrwałe generowanie długokształtne, kompleksowa mutacja i niezależna walidacja empiryczna pozostają otwarte. Wiarygodność architektury wynika częściowo z przyznania się do tej granicy.

Główną okazją naukową nie jest dodanie większej DSL składni. Ma to wykazać warunkową semantyczną dźwięk dla ograniczonego, ale nietrywialnego fragmentu. Wynik możliwy do opublikowania może określać język dokumentu zawierający podmioty, zdarzenia, relacje czasowe, ilości, modalność, wyraźne wyjątki i alternatywne odniesienia; zdefiniować dopuszczony zestaw interpretacji; udowodnić lub przetestować mechanicznie, że każdy opublikowany wynik może / musi być poprawny w stosunku do tego zestawu; pokazują, że symboliczne udoskonalenie eliminuje mierzalną klasę fałszywych ostrzeżeń; i sprawdzić, czy wygenerowane wyjaśnienia zachowują wynik kanoniczny. Taki wynik byłby węższy niż „wykonywalny angielski”, ale znacznie ważniejszy.

Główną możliwością praktyczną jest nowa klasa systemów dokumentowych i agentów, które są skłonne przestać. Normalny asystent LLM jest zoptymalizowany do dalszego tworzenia odpowiedzi. Symbolicznie ugruntowany czas działania może zwrócić niezgodną ontologię, nierozwiązaną tożsamość, niewystarczający zasięg, nieuzasadniony formalizm, wyczerpanie budżetu, fałszywy kontrprzykład lub nielicencjonowaną realizację. Wyniki te mogą wydawać się mniej imponujące w demonstracji, ale są one warunkami wstępnymi do niezawodnego wykorzystania w nauce, regulacji, inżynierii i podejmowaniu decyzji instytucjonalnych.

Moim niezależnym osądem jest zatem to, że nllAgent wybrał wiele właściwych granic architektonicznych i powinien oprzeć się naciskom, aby poszerzyć swoje twierdzenia szybciej niż poszerzy swoje dowody semantyczne. Jest to już wiarygodny prototyp uporządkowanej, źródłowej, oddzielonej teorii analizy językowej. Nie jest jeszcze dowodem na to, że nieograniczony język naturalny może być wykonany z formalnymi gwarancjami. Jego najlepszą ścieżką naprzód jest stanie się eksperymentalną platformą dla węższego i bardziej możliwego do obrony twierdzenia opracowanego w całej tej książce: LLMs odkryć formalnych kandydatów, symboliczne jądro kontroluje uziemienie i konsekwencje, a wyjście w języku naturalnym pozostaje podporządkowane powtarzalnemu wynikowi kanonicznemu.

8. Budowa i walidacja neuro-symbolicznego środowiska wykonawczego

8.1 Strategia implementacji: minimalny, lecz naukowo uczciwy system języka wykonywalnego

Architektura proponowana w tej książce jest ambitna tylko wtedy, gdy jest opisana niejasno. Po oddzieleniu jego zobowiązań pierwszy możliwy do wdrożenia system jest stosunkowo skromny. Nie musi rozumieć języka arbitralnego, konstruować uniwersalnej ontologii ani udowadniać każdego wniosku. Musi przyjąć ograniczoną klasę dokumentów, zachować dokładne dowody, zmaterializować małą rodzinę obserwacji, skompilować ograniczoną rodzinę zasad w jeden lub dwa formalne silniki, zwrócić świadków lub wyraźne niewiadome i zapobiec zmianie symbolicznego wyniku ostatecznej warstwy języka naturalnego. Wartość naukowa pochodzi z tego, że każde ograniczenie jest widoczne i mierzalne.

Pierwsza decyzja projektowa jest deklaracją docelową. Prototyp badawczy nie powinien zaczynać się od „system rozumie i wykonuje język naturalny”. Powinno się zacząć od roszczenia, takiego jak: dane dokumenty z deklarowanego gatunku, ontologia wersjonowana, skończona rodzina producentów obserwacji i zestaw opublikowanych zasad, czas pracy określa, czy każda zasada jest spełniona, naruszona, potencjalnie naruszona, niewspierana lub poza semantyczną relacją wydania. Rezultat jest regrywalny i połączony z rozpiętością źródłową. Twierdzenie to jest wystarczająco wąskie, aby przetestować i wystarczająco szerokie, aby wspierać przegląd zgodności, analizę wymagań, kłaczenie dokumentacji, inspekcję protokołu i kontrolowane planowanie.

Warstwa źródłowa musi być celowo nudna. Każdy dokument jest przechowywany jako niezmienna rewizja ze stabilnym przetrwaniem. Jednostki strukturalne - nagłówki, akapity, zdania, elementy listy, komórki tabeli, podpisy, przypisy i wbudowane obiekty - otrzymują trwałe identyfikatory. Normalizacja jest rejestrowana, a nie cicho stosowana. Przesunięcia znaków powinny odnosić się do kanonicznej reprezentacji źródła, podczas gdy odwzorowania zachowują pozycje w oryginalnym pliku. Oświadczenie w tabeli i identyczne wizualnie zdanie w akapicie może wiązać się z różnymi rolami strukturalnymi; to rozróżnienie musi przetrwać ekstrakcję. Gdy OCR jest nieunikniony, wyjście OCR i obraz strony są oddzielnymi obiektami dowodowymi, a niepewność w transkrypcji nie jest zapadana w zwykły tekst.

Ta dyscyplina źródłowa nie jest nad głową administracyjną. To właśnie sprawia, że symboliczne wnioski są kontestowalne. Odkrycie, które przytacza tylko wygenerowane podsumowanie, nie może być niezależnie sprawdzone. Odkrycie powiązane z dokładnym tekstem, komórkami tabeli i rekordami transformacji można przejrzeć, nawet jeśli zmienił się każdy model językowy. Pakiet źródłowy należy zatem uznać za część zaufanego rejestru audytu, chociaż ekstrakторы, które go tworzą, pozostają omylne i wymagają testów specyficznych dla formatu.

Druga warstwa jest raczej wykresem dowodowym niż pojedynczym parsem. Wykres zawiera wzmianki, podmioty, wydarzenia, stany, ilości, wyrażenia czasowe, modalność, relacje i linki źródłowe. Co najważniejsze, reprezentuje również alternatywy i status. Powiązanie podmiotu zaproponowanego przez model nie jest przechowywane jako bezwarunkowa równość. Jest to relacja tożsamości kandydata z producentem, punktacja, rozpiętość poparcia, sprzeczne

dowody i status epistemiczny, taki jak proponowany, potwierdzany, akceptowany, odrzucany lub nierozwiązany. Relacja czasowa wywnioskowana z „wkrótce po” nie powinna być normalizowana do arbitralnego dokładnego czasu trwania; powinna zachować stosunek jakościowy i dowolną deklarowaną interpretację domeny.

Warstwa ta powinna pozostać skończona i ograniczona zadaniem. System nie buduje kompletnego modelu świata. Materializuje rozróżnienia wymagane przez opublikowane obwody i przez bieżące zapytanie. Materializacja Query-first, już zbadana w nllAgent, jest prawidłowym domyślnym. Obwód oświadcza, że potrzebuje tożsamości aktora, relacji dokument-porządek, deontycznej modalności i skutecznych dat. Kompilator wywodzi te wymagania przed kosztowną interpretacją. Następnie producenci generują tylko obserwacje wymagane do decydowania o zgodności i wykonania reguły. Zmniejsza to koszty i, co ważniejsze, ogranicza powierzchnię semantyczną, na której mogą wystąpić błędy.

Trzecia warstwa to symboliczne jądro uziemienia. „Uziemienie” jest często używane luźno, aby oznaczać, że odpowiedź przytacza fragment. Tutaj ma to bardziej rygorystyczne znaczenie: każdy symbol przyznany do formalnego wykonania ma wpisany referent, pochodzenie, zakres i status wystarczający dla formalizmu docelowego. Przewidywania takie jak zatwierdzone (x, y, t) nie mogą być wprowadzane tylko dlatego, że LLM wytworzył te tokeny. Jądro sprawdza, czy x i y odnoszą się do dopuszczalnych typów jednostek, że t jest wspieraną wartością czasową lub odstępem czasu, że producent obserwacji jest upoważniony do tego przewidywania, że rozpiętość źródłowa istnieje w deklarowanej rewizji i że status spełnia wymóg gwarancji reguły.

Umowy o uziemienie powinny odróżniać dowody od ontologii. Dowody mówią, że rozpiętość wspiera obserwację. Ontologia mówi, jaki rodzaj rzeczy oznacza obserwacja i jakie relacje są dopuszczalne. Aksjomy domeny mówią, co wynika z tych relacji. Światowa wiedza dostarcza zewnętrznie pozyskiwane roszczenia. Te warstwy nie mogą być scalane. Rejestr spółek może ustanowić, że dwie nazwy oznaczają jeden podmiot prawny w określonym okresie. Ontologia domeny może oświadczyć, że osoby prawne mogą podpisywać umowy. Teoria polityki może oświadczyć, że jako zatwierdzenie liczy się podpis upoważnionego przedstawiciela. Model może sugerować wszystkie trzy połączenia, ale każde połączenie należy do innego organu i powinno być akceptowane lub odrzucane osobno.

Tożsamość zasługuje na własną służbę, ponieważ jest najczęstszą ukrytą zależnością w rozumowaniu w długim dokumencie. Usługa powinna zarządzać wzmiankami o klastrach, wyraźnych aliasach, identyfikatorach, rolach organizacyjnych, ważności czasowej i wybranych światach. Powinien pozwolić kilku nierozwiązanym kandydatom, a nie zmuszać do wczesnego zjednoczenia. W pierwszym wydaniu tożsamość może być celowo zachowawcza: akceptowane są wyraźne identyfikatory i dokładne deklarowane aliasy; linki oparte na rolach i kontekstowe pozostają proponowane; nieobsługiwana tożsamość miedzydokumentowa zatrzymuje zasady, które tego wymagają. Ta konserwatywna polityka obniży pozorny zasięg, ale znacznie poprawi znaczenie przyjętych wyników.

Czas potrzebuje podobnie wyraźnej domeny. Praktyczna początkowa reprezentacja może łączyć dokładne momenty, zamknięte lub otwarte interwały, kolejność jakościową i nieznane granice. Czas działania musi odróżniać kolejność dokumentów od kolejności zdarzeń i czasu publikacji od efektywnego czasu. Zwroty czasowe, takie jak „przed wdrożeniem”, „w ciągu trzydziestu dni”, „podczas gdy autoryzacja pozostaje ważna” i „po jakiegokolwiek zmianie materiału” wywołują różne struktury. Kompilator powinien odrzucić regułę czasową, jeśli wykres dowodów ma tylko obserwacje porządku dokumentu, gdy wymagana jest semantyka zdarzenia.

Cicha substytucja między domenami czasowymi jest klasycznym źródłem wiarygodnych, ale nieprawdziwych wniosków.

Modalność i siła normatywna wymagają raczej małej sieci niż Boolean. System powinien co najmniej rozróżnić twierdzenie, raport, przekonanie, możliwość, zdolność, pozwolenie, obowiązek, zakaz, zalecenie, intencję, stan i kontrfaktyczne. W stosownych przypadkach powinna również rejestrować posiadacza i beneficjenta normy. „Dostawca może ujawnić dane” i „dostawca musi ujawnić dane” udostępnia słownictwo, ale mają przeciwne konsekwencje zgodności. „Dokument stwierdza, że dostawca przestrzegał” jest dowodem raportu, niekoniecznie dowodem zdarzenia leżącego u podstaw. Oficjalny czas działania staje się przydatny właśnie poprzez odrzucenie tych upadków.

Po uziemieniu istnieją obserwacje, czwarta warstwa oblicza abstrakcyjny stan semantyczny. Minimalna domena może reprezentować, dla każdej propozycji istotnej dla zapytania, niezależnie od tego, czy jest prawdziwa we wszystkich dopuszczalnych interpretacjach, prawdziwa w niektórych, fałszywa w sumie, sprzeczna, czy nierozwiązana z powodu brakującego pokrycia. Implementacja nie musi zaczynać się od wyrafinowanej biblioteki kratowej. Może zacząć się od wyraźnych skończonych alternatyw i mechanicznie wyprowadzonego może / must summary. Wraz ze wzrostem skali, domeny abstrakcyjne mogą podsumować kandydatów tożsamości, zakresy liczbowe, interwały czasowe, statusy modalne i zestawy pochodzenia bez wyliczania każdego świata.

Państwo abstrakcyjne wykonuje dwie prace. Po pierwsze, zapewnia konserwatywną propagację ogólnodokumentową. Jeśli każda przyjęta interpretacja łączy definicję z terminem, relacja ta jest faktem obowiązkowym. Jeśli tylko jedno z trzech prawdopodobnych przydziałów tożsamości powoduje konflikt interesów, konflikt jest raczej możliwym odkryciem niż zweryfikowanym naruszeniem. Po drugie, decyduje o tym, gdzie droższa egzekucja jest uzasadniona. Wynik obowiązkowy może zakończyć się wcześniej. Wynik mocnego naruszenia powoduje przeszukanie świadka. Niewiadoma spowodowana przez nieobecny zasięg producenta wywołuje problem z kompatybilnością semantyczną, a nie wezwanie solvera.

Piąta warstwa to plan formalizmu. Plan nie jest prozą o swobodnym kształcie napisaną przez agenta. Jest to wpisana decyzja kompilacyjna, która określa, jakie pytanie zostanie zadane, który silnik, jakie obserwacje są wejściami, co gwarantuje zwrot silnika i jak jego wynik mapuje do kanonicznego słownictwa wynikowego. Wybór powinien być napędzany raczej przez kształt semantyczny niż modę.

Ocena relacyjna jest pierwszym silnikiem, który został wdrożony, ponieważ wiele przydatnych kontroli dokumentów zmniejsza się do połączeń, porównań zamówień, grup i ustawia różnic. Zasady takie jak „każdy zdefiniowany termin techniczny jest używany konsekwentnie”, „każde przyjęte roszczenie ma co najmniej jedno źródło”, „żadny identyfikator wymagań nie jest powielany” lub „wszystkie wymienione ryzyka mają właściciela łagodzenia” nie wymagają twierdzenia. Rdzeń relacyjny jest przejrzysty, wydajny i łatwy do przetestowania. Datalog rozszerza ten rdzeń, gdy wymagane jest przejściowe zamknięcie lub zależności rekurencyjne. Przykłady obejmują dziedziczenie, propagację częściową, łańcuchy zależności i osiągalność poprzez cytowanie lub wykresy wymagań.

Tabela decyzyjna jest drugim użytecznym formalizmem dla ograniczonej polityki. Tabela decyzyjna sprawia, że kombinacje warunków i wyniki są wyraźne, ujawnia nakładające się lub brakujące wiersze oraz wspiera analizę zasięgu. Jest to szczególnie właściwe w przypadku kwalifikowalności, trasowania, klasyfikacji i zasad politycznych, których złożoność wynika raczej

z kombinacji niż nawrót. Obwód może zestawić politykę języka naturalnego w tabelę kandydacką, ale publikacja powinna wymagać przykładów na poziomie wiersza, wykrywania nakładania się, kontroli kompletności i zatwierdzania przez człowieka partycji semantycznej.

Silniki SAT, SMT, CSP i MaxSAT stają się przydatne, gdy zadanie zawiera oddziaływanie ograniczeń, arytmetykę, harmonogramy, zadania lub optymalizację. Kompilator musi wybrać wspierane teorie i unikać ukrywania nieobsługiwanej semantyki w nieinterpretowanych ciągach. Wynik SMT może udowodnić niewykonalność pod zakodowanymi ograniczeniami, wytworzyć satysfakcjonujący model lub odsłonić niezaspokojony rdzeń. Rdzeń należy odwzorować z powrotem do obowiązków i założeń źródłowych. To mapowanie jest bardziej pouczające niż proszenie LLM o wyjaśnienie błędu generycznego solvera; identyfikuje najmniejsze formalne zobowiązania, które nie mogą wspólnie utrzymać.

Logika czasowa i sprawdzanie modeli są odpowiednie dla procedur, przepływów pracy, protokołów i reguł cyklu życia. Wykres dowodów dostarcza zdarzeń i propozycji państwowych; kompilator konstruuje system przejściowy i własność czasową; sprawdzający zwraca satysfakcję, kontrprzykładowy ślad lub niejednoznaczny wynik pod granicami. Trudna część to nie czasowy warcab. Decyduje się, czy źródło definiuje kompletny system przejściowy, czy pominięte przejścia są niemożliwe, czy tylko nieudokumentowane, oraz czy uzasadnione są założenia dotyczące sprawiedliwości lub środowiska. Wynik kanoniczny musi zatem obnażyć granicę modelu.

Symboliczne wykonanie jest odpowiednie, gdy reguła odnosi się do możliwych ścieżek poprzez kod wykonywalny, procedurę stanową lub artefakt semantyczny podobny do programu. Jego wydajność powinna być konkretnym świadkiem: przydziałów, warunków gałęzi, przejść stanowych i łączy źródłowych. W ustawieniach tylko dokumentów, symboliczne wykonanie może zbadać alternatywne powiązania lub gałęzie proceduralne, ale nie należy go wywoływać tylko dlatego, że system zawiera symbole. Termin ten powinien zachować swoje wrażliwe na ścieżki znaczenie. SymGPT jest pouczający, ponieważ reguła języka naturalnego jest kompilowana w stan naruszenia, a następnie testowana pod rzeczywistymi ścieżkami kontraktu, a nie przeciwko bardziej wygenerowanej prozie [Xia et al. 2026].

Naturalna logika, skończona automata, parsowanie wykresów, obliczenia półprzestępcze i specjalistyczne twierdzenia twierdzą, że są dowodem na istnienie portfela, gdy ich założenia strukturalne pasują. Logika naturalna może skutecznie radzić sobie z monotonicznością i pewnymi zaciągnięciami leksykalnymi w pobliżu języka powierzchniowego. Automaty są doskonałe do zamawianych wzorów lokalnych. Metody wykresów i półrejestracji mogą utrzymywać wiele pars z obliczeniami współdzielonymi. Asystenci dowodowi są odpowiedni, gdy niewielka liczba roszczeń o wysokiej wartości uzasadnia ręczną formalizację i sprawdzone przez jądra dowody. Architektura powinna sprawić, że dodanie silnika jest rozszerzeniem typowanym, a nie zaproszeniem do osadzenia dowolnego kodu wykonywalnego w obwodach.

Szósta warstwa jest kontrprzykładowym udoskonaleniem semantycznym. Może zostać opracowany w problem wykonalności. Jeśli solver zwróci świadka, czas działania odtwarza go na wykresie dowodowym i informuje o dokładnych wyborach interpretacyjnych. Jeśli ścieżka jest niewykonalna, silnik zwraca niesatysfakcjonujący rdzeń, nieudany warunek wstępny lub konflikt ustawiony. System śledzi, który sprzeciwia się kompilacji do obserwacji źródłowych i mapowań kandydatów, które go wytworzyły. Dopiero wtedy z prośbą jest LLM o pomoc.

Monit naprawy powinien być lokalny i typowany. Może zapytać, czy dwie wzmianki zostały błędnie ujednolicone, czy warunek był przeznaczony jako konieczny lub wystarczający, czy ma

zastosowanie wyjątek, czy też relacja czasowa ma niewłaściwy punkt końcowy. Powinien on przedstawiać odpowiednie rozpiętości źródeł, umowy ontologiczne, aktualne alternatywy i konflikt symboliczny. LLM proponuje jeden lub więcej poprawionych artefaktów semantycznych. Propozycje te przechodzą przez takie same kontrole uziemienia i kompilacji jak oryginały. Człowiek może być wstawiony, gdy dwuznaczność jest normatywnie lub naukowo materialna.

Proces ten zasadniczo różni się od ogólnej autorefleksji. Model nie jest informowany, że jego odpowiedź była błędna i zapraszana do improwizacji innego łańcucha myślenia. Podaje się mechanicznie zidentyfikowany konflikt w trwałym artefaktach semantycznych. Naprawa może być przetestowana przez regresję. Jeśli naprawa zmienia ontologię lub producenta, publikacja wymaga nowych przypadków referencyjnych i separatywnej różnicy. Kontrprzykład staje się stałym elementem korpusu oceny.

Siódma warstwa to wynik kanoniczny. Powinien to być stabilny, niezależny od rozwiązania produktu, który oddziela werdykt, dowody, założenia, interpretacje, wykonanie i ograniczenia. W przypadku reguły audytu rejestruje wersję reguły, obowiązujący zakres, stan zgodności, pokrycie, przyjęte uwagi, odrzucone obserwacje, wynik wykonania, odniesienie do świadka lub dowodu, wynik weryfikacji i nierozwiązane problemy. W przypadku planu rejestruje cele, ograniczenia, kroki, zależności, zasoby, alternatywy i dokładne kontrole, które spełnia plan. Produkt kanoniczny jest autorytetem dla każdej późniejszej odpowiedzi w języku naturalnym.

Droga powrotu do języka musi być traktowana jako kompilacja, a nie wolne pokolenie. Renderer otrzymuje skrócony zestaw licencjonowanych roszczeń. Każde twierdzenie ma modalność, biegunowość, ilość, odniesienia do podmiotu, niepewność i pochodzenie. Renderujący może wybrać kolejność dyskursu i wyrażenie wyjaśniające, ale może nie wprowadzać nowych jednostek, wzmacniać może, aby musi, przekształcić brakujące dowody w brak, stłumić materialne założenie lub wymyślić wyjaśnienie przyczynowe. Identyfikatory roszczeń na poziomie zdania powinny pozostać dostępne do kontroli. Drugi sprawdzacz powinien porównać renderowany tekst z wynikiem kanonicznym, przy użyciu testów deterministycznych dla nazw, ilości, dat, negacji i modalności oraz, w razie potrzeby, oddzielnego modelu ograniczonego do zaręczyn tekstowych, a nie oceny otwartej.

Skrępowana dowód realizacja jest szczególnie ważna, ponieważ użytkownicy będą oceniać system po jego prozie. Doskonały symboliczny ślad może zostać podważony przez eleganckie, ale niedokładne podsumowanie. Architektura powinna zatem uczynić wszechobłąknienie opcjonalną, ale wiernością obowiązkową. W trybie wysokiej pewności system może generować prosty, powtarzalny język, ponieważ każde zdanie podąża za szablonem licencjonowanym przez typ wyniku. W trybie wyjaśniającym LLM może poprawić czytelność, ale niezwiązane zdania są odrzucane i regenerowane. Kanoniczny artefakt, a nie proza, pozostaje zapisem decyzji.

Bezpieczeństwo i zarządzanie muszą być zintegrowane od początku. Dokumenty w języku naturalnym są niezaufanymi wejściami i mogą zawierać szybkie wstrzykiwanie, przykłady kontradiktoryjne, formatowanie wprowadzające w błąd lub fragmenty kodu. Producenci semantyczni powinni otrzymywać dane źródłowe za pośrednictwem wpisanych interfejsów, a nie uprzywilejowanego monitu systemowego, który może zostać nadpisany przez zawartość dokumentu. Wygenerowane artefakty formalne muszą być analizowane przez restrykcyjne gramatyki. Rozwiązywacze powinni działać w granicach zasobów. Rozszerzenia Runtime powinny być sprawdzane, wersjonowane i blokowane przez trawienie. Nauka obszarów

roboczych nie powinna publikować bezpośrednio w produkcji. Granice te są już częściowo obecne w nllAgent i powinny być wzmocnione, a nie omijane dla wygody [AssistOS-AI 2026].

Realistyczny program realizacji może być realizowany w pięciu wersjach. Pierwsze wydanie powinno wspierać niezmiennie pobieranie Markdown lub DOCX, rozpiętości źródeł, obserwacje strukturalne, silnik zapytań relacyjnych, ograniczone obserwacje kontrolowane-angielskie, stany zasięgu, obwody wielokrotnego użytku, weryfikację powtórki i ustalenia kanoniczne. Powinien rozwiązywać zadania ograniczone, takie jak kompletność pochodzenia, spójność definicji, pokrycie wymagań, uporządkowane zasady sekcji i proste rozróżnienia modalne. Nie należy twierdzić ogólnej tożsamości, doniesień czasowych ani wyszukiwania w świecie bieżącym.

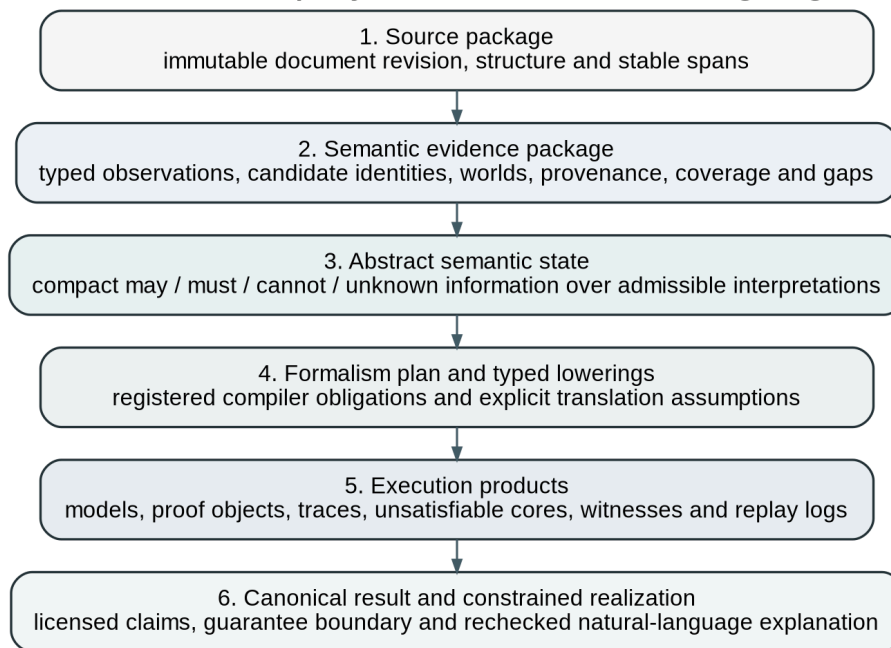
Drugie wydanie powinno dodać typowaną tożsamość i czas. Tożsamość zaczyna się od wyraźnych identyfikatorów, aliasów i ról deklarowanych przez dokument; semantyka czasowa zaczyna się od dokładnych dat, odstępów czasu, kolejności dokumentów i małego zestawu wzorców kolejności zdarzeń. Benchmarki powinny zawierać niejednoznaczne aliasy, zmieniające się role, nakładające się interwały i celowo brakujące ogniwa. System powinien się zatrzymać, a nie zgadywać, kiedy obwód wymaga silniejszej tożsamości niż zapewnia wydanie.

Trzecie wydanie powinno dodać tabele decyzyjne i SMT/CSP wykonanie, z niesatysfakcjonującym mapowaniem rdzenia i kontrprzykładowym udoskonaleniem semantycznym. Niniejsze wydanie może dotyczyć zasad kwalifikowalności, harmonogramów, ograniczeń przydziału, spójności ilościowej i wykonalności procedury ograniczonej. Każdy wynik solvera musi być odtwarzany za pomocą niezależnego weryfikatora lub mniejszego sprawdzania, tam gdzie to możliwe. Testowanie mutacji powinno zmienić ograniczenia i obserwacje, aby wykazać, że pakiety porównawcze wykrywają regresje semantyczne.

Czwarte wydanie powinno dodać adapter czasowy / sprawdzający model i czuły na ścieżkę adapter symboliczno-wykonawczy dla jednej domeny betonowej, taki jak specyfikacje przepływu pracy lub zasady smart-contract. Celem nie jest szeroki zasięg, ale dowód na to, że rejestr formalizmu i interfejs certyfikatu kanonicznego obsługują naprawdę różną semantykę. Testy spójności międzyformalizmowe powinny sprawdzić, czy wspólne uzasadnione fakty zachowują tożsamość, czas, modalność i pochodzenie w zakresie obniżenia.

Piąte wydanie powinno dodać planowanie i kontrolowaną realizację. Pomysł jest kompilowany w plan kanoniczny, sprawdzany pod kątem formalnych ograniczeń, realizowany w prozę i kontrolowany przez ten sam system reguł. Ocena musi rozróżnić ważność planu, wierność realizacji, jakość językową i wydajność naprawy. Plan nie staje się godny zaufania tylko dlatego, że proza została wygenerowana z obiektu ustrukturyzowanego; staje się bardziej audytowalny, ponieważ obiekt i jego kontrole są trwałe.

Artifact stack for a replayable executable-language runtime



Rys. 4. Stos artefaktów dla odtwarzalnego środowiska wykonawczego w języku naturalnym. Każda warstwa jest wersjonowana, mapowana i niezależnie sprawdzana.

To etapowe wdrożenie uosabia ważną zasadę metodologiczną: ambicje semantyczne powinny rosnąć tylko wtedy, gdy system może wykryć, że jego poprzednia reprezentacja jest niewystarczająca. Czas działania nie powinien ukrywać nieobsługiwanego języka za zaufaniem modelki. Powinien on ujawnić luki w kompatybilności jako wyniki pierwszej klasy i przekształcić powtarzające się luki w nowych producentów, ontologię, silnik i porównawcze prace. Wykonalny język naturalny postępuje poprzez uporządkowaną ekspansję koperty solidności, a nie poprzez jeden skok do powszechnego zrozumienia.

8.2 Pytania badawcze, benchmarki i program publikacji

Architektura staje się wkładem naukowym tylko wtedy, gdy jej twierdzenia są zbuntowane. Demonstracja, w której LLM generuje wiarygodny artefakt formalny, a solver zwraca odpowiedź, jest niewystarczająca. Ten sam model może wygenerować benchmark, tłumaczenie i wyjaśnienie, tworząc zamkniętą pętlę skorelowanego błędu. Poważna ocena musi rozłożyć rurociąg i przetestować każdy obowiązek niezależnie. Musi również obejmować warunki awarii, w których proponowane podejście powinno zostać odrzucone lub zawężone.

Pierwsze pytanie badawcze dotyczy reprezentacji: czy utrzymanie kilku dopuszczalnych interpretacji semantycznych poprawia wiarygodność w wyborze jednego tłumaczenia? Odpowiednia hipoteza jest taka, że semantyka kandydująca, w połączeniu z analizą *may/must*, zmniejsza fałszywą pewność na dwuznacznych i niedostatecznie określonych wejściach. Może obniżyć pozorną odpowiedź, ponieważ więcej przypadków staje się warunkowych lub nieznanych. Niekoniecznie jest to wada. Odpowiednie środki porównawcze, czy przyjęte wnioski muszą być częściej poprawne, oraz czy ustalenia mogą uchwycić istotnie prawdopodobne alternatywy bez przytłaczających recenzentów.

Punkt odniesienia dla tego pytania powinien zawierać teksty z zestawami interpretacji z adnotacjami ekspertów, a nie jedną złotą formę logiczną. Minimalne pary powinny różnić zakres kwantyfikatorów, negację, przywiązanie wyjątku, koordynację, koreferencje, granice czasowe, siłę modalną i jurysdykcję. Niektóre pozycje powinny mieć jedną wyraźnie zamierzoną lekturę, kilka legalnych odczytów i kilka niewystarczających dowodów na jakąkolwiek określoną odpowiedź. Systemy są oceniane pod kątem dopuszczalnych odczytów, wykluczenia niedopuszczalnych odczytów, kalibracji statusu majowego/obowiązkowego oraz werdyktów niższego szczebla. Dokładne dopasowanie wykresu jest zbyt łamliwe; równoważność semantyczną należy przetestować za pomocą pakietów zapytań lub dwukierunkowego zaangażowania w ramach ontologii porównawczej.

Drugie pytanie badawcze dotyczy dopracowania: czy konflikty oparte na solverach są bardziej przydatne do naprawy formalizacji niż ogólna samokrytyka modelu? Hipoteza jest taka, że kontrprzykłady, niezaspokojone rdzenie i nieudane zobowiązania dowodowe zapewniają bardziej zlokalizowane, stabilne i wydajne na próbkach naprawy, ponieważ identyfikują zależność odpowiedzialną za wynik. Kontrolowany eksperyment powinien porównywać cztery warunki: bezpośrednią próbę, autorefleksję słowną, sprzężenie zwrotne z błędów egzekucyjnych i dopracowanie semantyczne z mapowaniem źródłowym. Ten sam model, temperatura i budżet powinny być stosowane tam, gdzie jest to możliwe.

Jakość naprawy powinna być mierzona nie tylko przez to, czy docelowy przykład staje się prawidłowy. Naprawa musi zachować niepowiązane przypadki, zmniejszyć różnicę semantyczną i uniknąć wprowadzania nieuzasadnionych założeń. Pakiety regresji oparte na mutacji są niezbędne. Naprawa, która rozwiązuje jeden przykład poprzez osłabienie reguły na całym świecie, nie jest sukcesem. Eksperyment powinien rejestrować liczbę iteracji, żetonów, interwencji ludzkich, ocalałych mutantów, zmienionych obserwacji i tego, czy ostateczny artefakt był niezależnie możliwy do ponownego odtwarzania.

Trzecie pytanie badawcze dotyczy umowy na dowody. Czy typowane oddzielenie rozpiętości źródłowych, obserwacji, twierdzeń ontologicznych, aksjomatów domeny i pomieszczeń wiedzy światowej poprawia transfer i możliwość audytu w porównaniu z płaskim wykresem wiedzy lub łańcuchem myślenia o swobodnej formie? Hipoteza jest taka, że typowane kontrakty

zmniejszając ciche wprowadzenie i sprawiają, że ponowne użycie między domenami jest bardziej przewidywalne. Prawdopodobnym kosztem jest tworzenie kosztów ogólnych i więcej zatrzymanych biegów.

Punkt odniesienia powinien zawierać tę samą regułę, która ma zastosowanie w dokumentach z kilku dziedzin, z kontrolowaną zmiennością słownictwa i ontologii. Na przykład ogólny wymóg, że każde przyjęte twierdzenie ma identyfikowalne dowody, może zostać przetestowany na dokumentach badawczych, dokumentach polityki, specyfikacjach technicznych i planach. Uwagi strukturalne mogą zostać przeniesione bezpośrednio; podmiot specyficzny dla danej dziedziny i producenci relacji nie mogą. Rejestry oceny, które sprawdzają zgodność, czy przyczyna zatrzymania jest poprawna i ile nowego materiału semantycznego jest wymagane do wznowienia wykonania. Udana przeniesienie to nie tylko uzyskanie odpowiedzi; to dokładne rozpoznanie tego, co można, a czego nie można ponownie wykorzystać.

Czwarte pytanie badawcze dotyczy wyboru formalizmu. Czy trasowanie do portfolio specjalistycznych silników przewyższa uniwersalny język pośredni pod równym nadzorem semantycznym? Beiser i Penz pokazują, że wybór w języku pośrednim może zmienić ogólną dokładność wykonania i dokładność wykonania w zadaniach zrozumienia neuro-symbolicznego [Beiser and Penz 2025]. Hipoteza jest tutaj bardziej konkretna: typowany router, który wybiera najprostsz odpowiedni formalizm, poprawi sukces kompilacji, jakość wyjaśnień i wydajność bez poświęcania spójności między zadaniami.

Istotne porównanie nie powinno pozwolić systemowi portfela na więcej ukrytych informacji. Każde zadanie otrzymuje te same dowody źródłowe i ontologię. Poziom odniesienia dla pojedynczego formalizmu może wykorzystywać logikę pierwszego rzędu, ASP, SMT lub ogólny język zapytania grafowego. System portfela może wybrać zapytania relacyjne, Datalog, tabele decyzyjne, SMT, logikę czasową lub wykonanie symboliczne. Wskaźniki obejmują ważność formalizacji, równoważność semantyczną, sukces solvera, czas pracy, jakość świadków i koszty utrzymania. Przypadki międzyformalizmowe powinny wymagać dwóch silników i sprawdzić, czy wspólne symbole pozostają spójne.

Eksperyment powinien również szukać kontrdowodów. Powszechna reprezentacja może przewyższać portfolio w wąskich domenach, ponieważ routing wprowadza błędy, a wiele kompilatorów zwiększa zaufaną bazę. Oczekiwany wniosek nie jest taki, że portfele są zawsze lepsze. Chodzi o to, że pluralizm jest uzasadniony tylko wtedy, gdy rozkład zadań rzeczywiście zawiera różne struktury obliczeniowe i gdy interfejsy zachowują semantykę.

Piąte pytanie badawcze dotyczy ostatecznej warstwy językowej. Czy ograniczona dowodowo realizacja zmniejsza nieobecne lub wzmocnione semantycznie roszczenia, zachowując jednocześnie akceptowalną czytelność? Hipoteza jest taka, że wynik kanoniczny plus pokolenie licencjonowane przez roszczenia znacznie obniża halucynację i modalność dryfu w stosunku do bezpośredniego generowania odpowiedzi lub generowania odpowiedzi z surowego śladu solvera. Kosztem może być mniej naturalna proza, a większa regeneracja.

Wskaźnik referencyjny powinien obejmować wyniki kanoniczne z subtelnością niepewnością, sprzeczne dowody, warunki zagnieżdżone, ilości i daty. Ludzcy oceniający oceniają wierność faktyczną, kompletność ograniczeń materialnych, czytelność i użyteczność. Deterministyczne sprawdzanie mierzy jednostkę, liczbę, datę, polaryzację i zachowanie modalności. Zestaw kontrydiktoryjny powinien skusić renderer do uzyskania silniejszego, prostszego wniosku niż wynik. Centralną metryką nie jest preferencja stylistyczna, ale tempo, w jakim proza przekracza autorytet produktu symbolicznego.

Szóste pytanie badawcze dotyczy samej architektury federacyjnej. Systematyczny przegląd neuro-symbolicznych NLP raportów z 2026 r. Obiecuje zyski dla systemów federacyjnych, ale ostrzega, że dowody są zmieszane z różnicami [Chatzikyriakidis and Lappin 2026] w zadaniach. Bezpośredni eksperyment powinien porównywać potok federacyjny z wstrzykiwaną lub end-to-end wyuczoną alternatywą w tych samych zadaniach, danych i skali modelu. Odpowiednie korzyści mogą wydawać się mniej surowej dokładności niż w przypadku audytu, wstrzymania się od głosu, lokalizacji błędów i naprawy.

Badanie powinno zatem przedstawiać kilka wymiarów. Potrzebna jest surowa dokładność zadania. Powinno mu towarzyszyć dokładność formy semantycznej, dokładność uwarunkowana rozwiązywaniem, nieznana kalibracja, odtwarzalność w różnych wersjach modelowych, wierność wyjaśnień, czas przeglądu ludzkiego oraz ułamek błędów przypisywanych ekstrakcji, uziemieniu, kompilacji, wykonaniu lub realizacji. Modułowy system, który pasuje, a nie przebija dokładność end-to-end, może być nadal preferowany w przypadku pracy o wysokiej pewności, jeśli zamieni ciche błędy w możliwe do zdiagnozowania artefakty. Odwrotnie, modułowość nie powinna być broniona jako wewnętrzna cnota, jeśli tylko dodaje biurokrację bez poprawy dowodów.

Siódme pytanie badawcze dotyczy abstrakcyjnej interpretacji alternatyw semantycznych. Czy można zdefiniować użyteczną domenę abstrakcyjną, która jest konserwatywna w stosunku do zestawu interpretacji porównawczej i bardziej skalowalna niż wrywkowa wyliczenie? Pierwsze eksperymenty mogą wykorzystywać zestawy skończone jako konkretne odniesienie. Abstrakcyjne domeny tożsamości, interwały czasowe, modalność, ilości i zasięg są wdrażane osobno, a następnie łączone. Dźwiękowość jest sprawdzana mechanicznie pod kątem wyliczenia w małych przypadkach. Precyzja jest mierzona przez wskaźnik możliwych ustaleń, które można wyeliminować za pomocą wyszukiwania świadków lub udoskonalenia.

Ten program powinien być ostrożny ze słowem zdrowym. Jeśli adnotacja eksperta definiuje skończony zestaw interpretacji, interpreter abstrakcyjny może być zdrowy w stosunku do tego zestawu. Nie może w ten sposób udowodnić, że zestaw wyczerpuje ludzkie znaczenie. Wkład jest wykonalną semantyką warunkową i dowodem na jej użyteczność. Późniejsza praca teoretyczna może definiować relację dopuszczalności, funkcje abstrakcji i konkretyzacji, warunki transferu i operatory udoskonalania. System empiryczny nie powinien czekać na kompletną semantykę języka, ale powinien wyraźnie wyrażać warunkowy charakter swoich twierdzeń.

Ósme pytanie badawcze dotyczy symbolicznego wykonania programów semantycznych. Czy wrażliwa na ścieżkę konstrukcja świadków poprawia zrozumienie recenzenta i odkrywanie defektów w stosunku do wyników reguł statycznych? W obszarze przepływu pracy lub domeny zasad, runtime może porównać prosty ewaluator reguł z symboliczną eksploracją ścieżki. Ewaluator mówi, że naruszenie jest możliwe; wykonawca zwraca konkretną sekwencję zdarzeń i zadań. Ludzcy recenzenci oceniają, czy świadek jest zrozumiały i użyteczny. Wady gruntu i mutacje rozsiane decydują o tym, czy świadek odpowiada prawdziwemu przypadkowi.

Wybuch ścieżki musi być częścią oceny, a nie traktowany jako niedogodność implementacji. Badanie powinno różnić się rozgałęzieniem, ograniczeniami pętli, alternatywami tożsamości i wyborami czasowymi. Wytyczne LLM mogą priorytetowo traktować ścieżki, jak sugeruje niedawna LLM praca z wykonaniem symbolicznym, ale silnik musi odróżnić pokrycie heurystyczne od formalnej kompletności. Wskaźniki pokrycia powinny raportować zbadane stany, stany przycinane, limity czasu solvera i nieobsługiwane zachowania zewnętrzne.

Świadek jest silnym dowodem na istnienie; brak znalezienia takiego nie jest dowodem nieobecności, chyba że związane z eksploracją i model uzasadniają ten wniosek.

Architektura wzorcowa powinna być warstwowa. Warstwa źródłowa testuje spożycie i zakotwiczenie. Warstwa obserwacyjna sprawdza ekstrakcję i status. Warstwa uziemienia sprawdza tożsamość, czas, modalność, ilości i autorytet. Warstwa formalizacji sprawdza równoważność semantyczną i ważność języka docelowego. Warstwa wykonania testuje wyjścia i certyfikaty solvera. Warstwa wynikowa testuje skład werdyktu i niepewność. Warstwa realizacji sprawdza wierność odpowiedzi w języku naturalnym. Każdy przykład powinien identyfikować najwcześniejszą warstwę, przy której należy wykryć wstrzykniętą usterkę.

Przypadki referencyjne powinny być wystarczająco małe, aby walidować ręcznie i komponować się w dłuższe dokumenty. Małe przykłady dostarczają wiarygodnej prawdy gruntowej dla subtelnej logiki. Długie kompozyty testują pobieranie, indeksowanie, ciągłość tożsamości i skalowalność. Ta sama podstawowa reguła może pojawić się w kilku formach językowych i strukturach dokumentów. Rodziny parafrazy testują niezmienność; zestawy kontrastu zmieniają jedną cechę semantyczną i czułość testu. Punkt odniesienia musi unikać powszechnej awarii, w której system osiąga dobre wyniki, zapamiętując wzorce powierzchni, ignorując zamierzone rozróżnienie.

Testy mutacyjne powinny być centralne. Mutacje mogą zmienić kwantyfikator, zanegować warunek, przenieść wyjątek, zmienić granicę daty, połączyć tożsamości, usunąć dowody, zmienić siłę modalną, zamienić aktora i obiektu, zduplikować wymóg lub zmodyfikować próg obwodu. Solidny pakiet powinien wykryć oczekiwaną zmianę i pozostawić niepowiązane wyniki stabilne. Wynik mutacji nie jest kompletną miarą jakości, ale ujawnia testy, które tylko potwierdzają jedną szczęśliwą ścieżkę. W przypadku nllAgent mutacje wykonywalne bezpośrednio dotyczyłyby jednej z uznanych luk [AssistOS-AI 2026] w repozytorium.

Ocena powinna obejmować szybki wstrzykiwanie i separatycznych przeciwników. Dokumenty mogą zawierać instrukcje skierowane do modelu, wprowadzające w błąd nagłówki, cytowany tekst polityki, kontryktoryjny Unicode, ukrytą treść lub oświadczenia, które przypominają deklaracje ontologii. Parser źródłowy musi zachować je jako dane. Producenci semantyczni powinni działać w ramach stałego organu systemowego. Punkt odniesienia powinien sprawdzić, czy tekst dokumentu może zmienić obwód, stłumić ustalenia lub spowodować wywołania uprzywilejowanych narzędzi. Wyniki bezpieczeństwa powinny być rejestrowane oddzielnie od dokładności semantycznej.

Ocena modelu na żywo musi być wystarczająco odtwarzalna, aby poprzeć twierdzenia naukowe. Każdy bieg powinien być dostawcą pinów, identyfikatorem modelu, schematem żądania, temperaturą, nasionami, gdy są obsługiwane, budowę kontekstu i politykę ponownego próbowania. Wkłady i wyjścia powinny być przechwytywane z zastrzeżeniem ograniczeń prywatności. Powtarzające się próby szacują wariancję. Deterministyczne podwójne elementy pozostają przydatne w testach oprogramowania, ale nie mogą ustalić jakości tłumaczenia. Co najmniej dwie rodziny modelowe powinny zostać przetestowane tam, gdzie jest to możliwe, aby odróżnić efekty architektoniczne od zachowania jednego dostawcy.

Ocena człowieka jest konieczna, ale powinna być wąsko zaprojektowana. Nie należy pytać ekspertów, czy odpowiedź „wygląda dobrze”. Powinni osądzić konkretne obowiązki: czy formalizacja zachowuje źródło, czy tożsamości są uzasadnione, czy świadek jest wykonalny, czy wyjaśnienie określa wszystkie założenia materialne i czy nieznane jest odpowiednio

zwracane. Międzystawowa niezgoda jest sama w sobie dowodem semijskiej dwuznaczności i nie powinna być zmuszana do jednej złotej etykiety bez analizy.

Podstawowe wskaźniki ilościowe powinny obejmować dokładność kotwicy źródłowej; dokładność obserwacji i przywoływanie według statusu; ontologia i ważność typu; precyzja wiązania tożsamości, przywołanie i kalibracja; dokładność relacji czasowych i modalnych; pokrycie zestawu kandydatów; może / musi brzmieć w stosunku do interpretacji adnotowanych; składnia formalizacji i równoważność semantyczna; sukces solvera; ważność świadka; sukces sprawdzania dowodów; fałszywy wskaźnik wiarygodności; odpowiednie wynagrodzenie; Pojedynczy wynik zbiorczy ukryłby mechanizm i powinien być drugorzędny.

Fałszywe zapewnienie zasługuje na szczególny nacisk. Mierzy przypadki, w których system przedstawia określony zweryfikowany wniosek, chociaż źródło, uziemienie lub zestaw interpretacji go nie uzasadnia. W aplikacjach o wysokiej pewności zmniejszenie fałszywego zapewnienia może mieć więcej znaczenia niż maksymalizacja odpowiedzi na pytania. System powinien otrzymać kredyt na dokładny stan zatrzymania, gdy bezpośredni LLM pewnie wymyśli most. Odpowiednia niewiadoma jest pozytywnym wynikiem.

Wartości bazowe powinny obejmować bezpośrednie LLM odpowiadanie, łańcuch myślenia lub równoważne rozumowanie, narzędzia rozszerzone LLM wykonanie bez trwałych artefaktów formalnych, tłumaczenie jednoformalizmowe, tłumaczenie kandydata bez abstrakcyjnej interpretacji, symboliczne wykonanie bez dopracowania semantycznego i pełną architekturę. W przypadku studium przypadku nllAgent obecne ograniczone wydanie fundamentów może służyć jako wdrożona podstawa, podczas gdy proponowana tożsamość, abstrakcyjna semantyka i rozszerzenia wieloformalizmu są oceniane stopniowo. Pozwala to uniknąć porównania pełnej wizji badawczej tylko ze słabszymi systemami zewnętrznymi.

Program publikacji może być zorganizowany wokół pięciu wkładów, a nie jednego ponadwymiarowego rozsczenia. Pierwszy papier powinien być papierem pozycji i architektury, który definiuje możliwy do wykonania język naturalny, granicę zaufania, lukę formalizacyjną, otoczkę solidności i projekt multiformalizmu. Jego wkładem jest jasność koncepcyjna i architektura odniesienia, poparta systematycznym badaniem i małym działającym demonstrantem.

Drugi papier powinien sformalizować referencyjny obrót i warunkową solidność łożyska pochodzenia. Definiuje obiekty źródłowe, obserwacje typowane, interpretacje kandydatów, semantykę may/must i kompatybilność. Dowodzi to solidności abstrakcyjnych obliczeń w stosunku do skończonego dopuszczalnego zestawu interpretacji i raportuje eksperymenty dotyczące tożsamości, czasu, modalności i pokrycia. To jest język programowania i rdzeń formalno-semantyczny.

Trzecia praca powinna wprowadzić dopracowanie semantyczne na przykładowo. Określa mapowanie od konfliktów solver do zobowiązań semantycznych powiązanych ze źródłem i porównuje dopracowanie z autorefleksją i ogólnymi informacjami zwrotnymi z wykonania. Najsilniejszym rezultatem byłoby niższe fałszywe zapewnienie, mniejsze naprawy, lepsze zachowanie regresji i zmniejszony wysiłek ludzki. Nawet negatywny wynik byłby cenny, gdyby zidentyfikował, który sprzężenie zwrotne solvera jest zbyt niskie, aby kierować naprawą semantyczną.

W czwartym artykule należy przedstawić portfolio formalizmu i kontrakty na silniki typowane. Ocenia adaptory relacyjne, Datalog, z tabeli decyzyjnej, SMT, czasowe i symboliczne na

powszechnym modelu artefaktu. W pracy należy podkreślić raczej zachowanie semantyczne i certyfikaty niż liczbę integracji. Mały zestaw prawdziwie różnych silników jest bardziej przekonujący niż wiele płytkich owijek.

Piątym dokumentem powinien być system nllAgent i dokument oceny. Powinien wyraźnie odróżnić wdrożone wydanie od szerszego środowiska wykonawczego. Może dokumentować niezmiennie korekty źródeł, LongTextJS, CircuitJS, obserwacje oparte na popycie, zgodność, zasięg, ograniczone wykonanie, zarządzanie uczeniem się i publikacją, weryfikację powtórek, audyt kanoniczny i planowanie produktów oraz proces poważnych problemów. Ocena powinna obejmować mutację wykonalną, zapisy modelu live, przypadki tożsamości, bieżące zarządzanie pakietami, benchmarki planowania oraz niezależny lub oddzielnie wdrożony weryfikator, w miarę możliwości.

Szósty późniejszy artykuł może studiować ograniczoną dowodowo realizację. Porównałby bezpośrednie generowanie, renderowanie szablonów, ograniczone LLM renderowanie i weryfikację post-hoc. Centralnym wkładem byłaby reprezentacja licencyjna dla roszczeń i dowody na to, że czytelny język może być generowany bez zmiany sposobu, niepewności lub treści faktycznych. Niniejsza praca łączy formalne wykonanie z praktyczną potrzebą dostępnej komunikacji międzyludzkiej.

Projekt powinien określać wyraźne kryteria awarii. Należy ponownie rozważyć podejście oparte na kandydacie, jeśli daje tak wiele alternatyw, które mogą / muszą wyniki są nieinformacyjne, a dopracowanie nie może odzyskać precyzji. Portfolio formalizmu powinno zostać zawężone, jeśli błędy routingu i krzyżowo-formalizmu przekraczają korzyści wynikające ze specjalizacji. Formalizacja wspomagana LLM powinna być ograniczona do wsparcia, jeśli równowaga semantyczna pozostaje słaba w ramach realistycznych dokumentów. Skrępowana korekta powinna wrócić do szablonów, jeśli proza generowana przez model wielokrotnie przekracza jej licencję. Rozumowanie w obecnym świecie powinno pozostać poza zaufanym rdzeniem, jeśli nie można egzekwować kontroli źródła, czasu, jurysdykcji i konfliktów.

Uczciwość naukowa wymaga również porównania z prostszymi alternatywami. Wiele kontroli dokumentów można rozwiązać za pomocą deterministycznych parserów, wyrażeń regularnych, walidacji schematu, zapytań do baz danych lub reguł autorstwa człowieka. LLM jest uzasadniony, gdy zmienność języka, mapowanie ontologii lub propozycja semantyczna istotnie zmniejsza koszty tworzenia lub zwiększa zasięg. Symboliczny rdzeń jest uzasadniony, gdy dokładna stan, powtórka, świadkowie lub niezależna weryfikacja mają znaczenie. Praca badawcza powinna dotyczyć sytuacji, gdy prostszy system działa równie dobrze. Wykonywalny język naturalny nie jest powodem do sformalizowania każdej preferencji redakcyjnej lub wstawienia solvera do zadań, które wymagają tylko niezawodnego parsera.

Prawdopodobnym praktycznym rezultatem nie jest maszyna, która wykonuje cały język naturalny. Jest to rodzina zarządzanych kompilatorów semantycznych dla domen dokumentów i agentów, źródła udostępniania, uziemienia, wykonania i infrastruktury certyfikatów. Każdy kompilator posiada deklarowaną kopertę semantyczną. LLMs sprawiają, że kompilatory te są łatwiejsze do autorowania, adaptacji i wyjaśniania. Formalne metody sprawiają, że ich akceptowane wyniki są odtwarzalne i kontrowersyjne. Połączenie jest przydatne właśnie dlatego, że żaden z komponentów nie jest proszony o dostarczenie tego, czego nie może niezawodnie zapewnić w pojedynkę.

Program badawczy jest zatem zarówno skromniejszy, jak i bardziej znaczący niż hasło „język naturalny jako kod”. Język naturalny pozostaje otwartym, społecznym, zależnym od kontekstu

medium. Runtime tworzy jego projekcje wykonywalne dla zadeklarowanych pytań. Prognozy te mogą być wystarczająco rygorystyczne, aby wspierać rzeczywiste audyty, plany, protokoły i kontrole naukowe, pod warunkiem, że system rejestruje odległość między obiektem źródłowym a formalnym, zamiast udawać, że odległość zniknęła.

Konkluzja: od formy probabilistycznej do konsekwencji symbolicznej

Niedawne odrodzenie neurosymbolicznego AI wytworzyło rozpoznawalny wzór architektoniczny. Duże modele językowe są używane tam, gdzie przestrzeń wyszukiwania jest językowa, niejednorodna lub trudna do wyliczenia. Systemy formalne są stosowane tam, gdzie ma to miejsce dokładny stan, spójność, wyczerpujące ograniczone przeszukiwanie, świadkowie lub dowody. Interesujące pytanie nie polega już na tym, czy można łączyć metody neuronowe i symboliczne. Po prostu mogą. Ważne pytanie, gdzie leży autorytet i co jest niezależnie sprawdzane.

Z badania wynika, że najbardziej wiarygodne systemy nie przyznają ostatecznej władzy modelowi językowego nad wynikiem. Języki kontrolowane tłumaczą ograniczoną powierzchnię na semantykę formalną. Parsery semantyczne generują zapytania lub programy wykonywalne. PAL deleguje obliczenia do Pythona. SatLM deleguje ograniczenia do solvera. Logic-LM i LINC dedagują odliczenie do silników logicznych. NL2TL i nl2spec przekładają wymagania na specyfikacje czasowe. Systemy planowania wykorzystują formalne modele i narzędzia weryfikacji do zachowania spójności o długim poziomie horyzontu. SymGPT kompiluje zasady języka ERC w ograniczenia i wykorzystuje symboliczne wykonanie do budowy konkretnych ścieżek kontraktowych. SAIL wykorzystuje LLMs do wyszukiwania abstrakcyjnych transformatorów, ale przyjmuje tylko kandydatów, którzy spełniają obowiązki mechaniczne. Systemy te znacznie się różnią, ale zbiegają się one z zasadą, którą proponuje pokolenie i decyduje niezależny mechanizm.

Zbieżność ta nie powinna być zawyżona. Solver use nie weryfikuje zrozumienia. Główną luką formalizacyjną pozostaje główny problem. Twierdzenie twierdzenia może wyprowadzić ważny wniosek z przewidywań, które błędnie przedstawiają tekst. Program może zwrócić oczekiwaną odpowiedź przez fałszywą ścieżkę. Model SMT może spełniać ograniczenia, które pomijają decydujące wymagania użytkownika. Symboliczny wykonawca może odsłonić prawdziwą ścieżkę oprogramowania dla błędnie przetłumaczonej reguły. Najsilniejsze systemy prądowe zmniejszają błąd dedukcyjny i obliczeniowy, jednocześnie przenosząc ryzyko do parsowania semantycznego, uziemienia, ontologii i zakresu.

Odpowiednią odpowiedzią nie jest porzucenie narzędzi formalnych, ponieważ tłumaczenie jest niedoskonałe. Ma to sprawić, że tłumaczenie będzie trwałym, wpisanym, recenzowanym artefaktem. Źródło musi pozostać dostępne. Każda obserwacja semantyczna musi identyfikować swojego producenta, dowody, status i ontologię. Tożsamość, czas, modalność, ilości i autorytet muszą być reprezentowane, a nie wywnioskowane na nowo podczas każdej odpowiedzi. Alternatywne interpretacje muszą pozostać alternatywą, gdy rozróżnienie wpływa na wynik. Symboliczny silnik powinien wykonywać tylko uziemione obiekty, które spełniają wyraźne umowy.

Abstrakcyjna interpretacja stanowi użyteczną teorię dla globalnej strony tej architektury. Pyta, co można zawrzeć konserwatywnie z dużego zestawu możliwych państw. Stosowane do języka naturalnego, stany konkretne nie są nieokreślonymi „znaczeniami”, ale ograniczonymi strukturami semantycznymi dopuszczalnymi w ramach deklarowanej gramatyki, ontologii, rewizji źródeł i polityki uziemienia. Stan abstrakcyjny może wtedy reprezentować to, co musi mieć, co może mieć, nie może utrzymać, konfliktów lub pozostaje nieznane. Gwarancja jest uzależniona

od zestawu interpretacji, ale warunkowa solidność jest nadal znacząca, gdy warunki są wyraźne i przetestowane.

Symboliczne wykonanie zapewnia lokalne uzupełnienie. Gdy abstrakcyjna analiza zgłasza możliwe naruszenie, symboliczne wykonanie lub inna osoba ograniczająca może wyszukać konkretną interpretację i ścieżkę. Udań wyszukiwanie zwraca świadka, którego recenzent może sprawdzić. Nieudane wyszukiwanie obnaża zbyt gruboskórną abstrakcję lub niezgodny wybór semantyczny. Mapowanie, że porażka z powrotem do rozpiętości źródła tworzy kontrprzykładowe dopracowanie semantyczne: model lub człowiek rewiduje najmniejsze istotne formalne zaangażowanie, a analiza jest odtwarzana. Jest to bardziej zdyscyplinowane wykorzystanie informacji zwrotnych LLM niż ogólna samokrytyka, ponieważ sygnał błędu jest powiązany z formalną zależnością.

Żaden pojedynczy język docelowy nie będzie prawdopodobnie wspierał każdej użytecznej formy prozy wykonywalnej. Literatura już pokazuje zależność od wyboru w języku pośrednim, a współczesne systemy odnoszą sukces dzięki zastosowaniu Pythona, SQL, logiki pierwszego rzędu, logiki czasowej, PDDL, języków ograniczeń lub gramatyki specyficznej dla domeny zgodnie z zadaniem. Praktyczny czas działania powinien zatem wykorzystywać przełącznik semantyczny o znaczeniu proweniencyjnym i typowane portfolio silników formalnych. Relacyjna i Datalog ocena obsługują zapytania i zamknięcia dokumentów; tabele decyzyjne obsługują skończone zasady; SAT/SMT/CSP w zakresie interakcji ograniczeń; sprawdzanie modelu obsługuje zachowania czasowe; symboliczne wykonanie obsługuje świadków ścieżki; automatyczne uchwytów uporządkowane wzorce; naturalne uchwytów logiczne wybrane zarzuty; asystenci dowodowi obsługują roszczenia formalne o wysokiej wartości.

Plurality wprowadza własne ryzyka inżynieryjne. Każdy kompilator może zniekształcić semantykę, każdy silnik rozszerza zaufaną bazę, a trasowanie może się nie udać. Portfolio jest uzasadnione tylko wtedy, gdy interfejsy są wpisywane, symbole współdzielone zachowują tożsamość i pochodzenie, gwarancje silnika są wyraźne, a wyniki są normalizowane w kanoniczny produkt nośny certyfikatu. Uniwersalna logika może pozostać preferowana w wąskiej dziedzinie. Twierdzenie architektoniczne nie jest takie, że mnogość jest zawsze lepsza; jest tak, że szeroki język wykonywalny wymaga różnych struktur obliczeniowych i nie powinien ich ukrywać za jednym niejasnym DSL.

Powrót z symboli do języka jest częścią granicy zaufania. Ostateczny LLM nie może być dopuszczony do przekształcenia wyniku warunkowego w bezwarunkowe zalecenie, możliwe naruszenie w potwierdzone naruszenie lub brak dowodów w dowód nieobecności. Wynik kanoniczny powinien określać skończony zestaw licencjonowanych roszczeń z modalnością, ilościami, podmiotami, założeniami i pochodzeniem. Renderujący może poprawić czytelność, ale powinien pozostać podporządkowany tej władzy. Skrępowana dowodem realizacja jest naturalnym ukończeniem architektury.

Publiczne nllAgent wdrożenie jest znaczące, ponieważ ucieleśnia już kilka z tych dyscyplin. LongTextJS zachowuje źródłowy program dokumentów; CircuitJS reprezentuje teorie wielokrotnego użytku; zapotrzebowanie na obserwacje pochodzi z obwodów; zgodność i zasięg są wyraźne; ograniczone moduły i przegląd rozszerzenia ograniczają ukryte wykonanie; uczenie się odbywa się w jednorazowej inscenizacji; publikacja jest ręczna i niezmienna; kontrola weryfikacji powtórek kanoniczne ustalenia; a brakująca wiedza nie jest traktowana jako zgodność. Decyzje te sprawiają, że projekt jest wiarygodną platformą do badań, mimo że nie ustanawia ogólnego zrozumienia semantycznego.

Te same materiały publiczne identyfikują ważną niedokończoną pracę: ogólną materializację tożsamości, kompletne pisanie prymitywnych kontraktów, mutację semantyczną podlegającą wykonaniu, niezależną ocenę cienia, benchmarking modelu na żywo, benchmarki planowania i realizacji oraz zarządzane pakiety wiedzy w bieżącym świecie. Dla tej edycji nie została zlokalizowana żadna niezależna ocena nllAgent. Oferowany tutaj wyrok jest zatem architektoniczny: wdrożenie wybrało niezwykle rozsądne granice, ale jego najsilniejsze twierdzenia o obronie pozostają ograniczoną analizą strukturalną i kontrolowaną. Jego wartość wzrośnie, jeśli szerokość semantyczna zostanie rozszerzona tylko poprzez producentów wyraźnych, kontrakty, wskaźniki referencyjne i dowody.

Proponowanym kolejnym krokiem nie jest zastąpienie LongTextJS lub CircuitJS większym językiem uniwersalnym. LongTextJS powinien dojrzewać jako skończony, kwerendalny, niosący pochodzenie semantyczny świat. CircuitJS powinien pozostać ograniczoną orkiestracją i warstwą kontraktu dowodowego. Specjalistyczne zarejestrowane silniki powinny wykonywać wykonanie relacyjne, ograniczające, czasowe, przepływ danych, automaty lub wrażliwe na ścieżkę. Semantyka majowa/musta powinna stać się sprawna. Kontrprzykłady powinny napędzać skoncentrowane dopracowanie semantyczne. Kanoniczne CNL produkty powinny kontrolować końcowe wyjście językowe.

Rygorystyczny program badawczy może przetestować każdą część. Semantykę zestawów kandydatów można porównać z systemami jednottłumieńczymi. Udoskonalenie pochodzące z Solvera można porównać z autorefleksją. Wpisane umowy dowodowe mogą być testowane pod kątem transferu i fałszywego zapewnienia. Portfel formalizmu można porównać z uniwersalnym językiem pośrednim. Domeny abstrakcyjne można sprawdzić w oparciu o wyraźne zestawy interpretacji. Symboliczne świadki można ocenić pod kątem wykrywania wad i przydatności recenzenta. Dowód ograniczony realizacją może być testowany pod kątem wierności. Architektury modułowe i end-to-end można porównać w tych samych zadaniach, a nie w różnych rodzinach benchmarkowych.

Decydującą metryką powinna być fałszywa pewność, a nie tylko stopa odpowiedzi. System, który zwraca „nieznany, ponieważ tożsamość jest nierozwiązana” może być bardziej wartościowy niż ten, który wydaje pewny werdykt, cicho łącząc dwie osoby. System, który zatrzymuje się, ponieważ dowody w świecie bieżącym są nieświeże, jest bardziej godny zaufania niż taki, który odzyskuje nieużywany fakt. System, który eksponuje trzy interpretacje, jest lepszy niż taki, który wybiera najbardziej płynny. Odpowiednie wstrzymanie się od głosu jest częścią semantyki egzekucyjnej.

Wykonalny język naturalny należy zatem rozumieć jako regulowaną relację kompilacyjną, a nie wewnętrzną własność prozy. Tekst źródłowy pozostaje bogatszy niż jakakolwiek projekcja wykonywalna. System wybiera zadanie, materializuje odpowiednią semantykę, zachowuje niepewność, symbole uziemienia, kompiluje do odpowiedniego formalizmu, wykonuje pod wyraźnymi granicami, weryfikuje wynik i zwraca ograniczone wyjaśnienie. To, co staje się wykonalne, to nie język w całości, ale deklarowana projekcja semantyczna, której założenia i granice pozostają widoczne.

Ta węższa koncepcja jest wystarczająca do znacznego praktycznego wpływu. Twierdzenia naukowe można sprawdzić pod kątem pochodzenia i spójności. Wymagania mogą być powiązane z testami i stanami cyklu życia. Zasady mogą być kompilowane do struktur decyzyjnych. Plany można zweryfikować przed realizacją. Zasady protokołu można mapować do analiz stanu i ścieżki. Długie dokumenty można składać przez teorie wielokrotnego użytku

bez proszenia modelu o zapamiętanie każdego szczegółu. Agenci mogą używać języka naturalnego jako warstwy autorskiej i koordynacyjnej, jednocześnie delegując dokładne konsekwencje do symbolicznych czasów pracy.

Centralną zasadę można określić bez formalnej symboliki: pozwól LLM odkryć możliwy formularz; pozwól, aby jądro uziemienia zdecydowało, co symbole mogą oznaczać; pozwól, aby formalne silniki określiły, co następuje; i pozwól, aby ostateczne LLM wyjaśniło tylko, co symboliczne licencje rekordów. Zasada ta nie rozwiązuje każdego problemu języka. Tworzy granicę obrony między interpretacją a konsekwencjami, która jest niezbędnym fundamentem dla języka naturalnego, aby bezpiecznie uczestniczyć w systemach wykonywalnych.

Bibliografia

- Abzianidze, Lasha, Rik van Noord, Hessel Haagsma i Johan Bos. 2019. „Pierwsze wspólne zadanie dotyczące analizowania i oceny struktury reprezentacji dyskursu”. Materiały Wspólnego Zadania IWCS na Parsowaniu Semantycznym. Stowarzyszenie na rzecz Lingwistyki Obliczeniowej. <https://aclanthology.org/W19-1201/>
- Abzianidze, Lasha. 2017. „LangPro: Twierdzenie o języku naturalnym”. Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing: Demonstracje systemowe. Stowarzyszenie na rzecz Lingwistyki Obliczeniowej. <https://aclanthology.org/D17-2022/>
- Angeli, Gabor i Christopher D. Manning. 2014. „NaturalLI: Naturalne wnioskowanie logiczne dla zdrowego rozsądku”. Materiały EMNLP 2014. Stowarzyszenie na rzecz Lingwistyki Obliczeniowej. <https://aclanthology.org/D14-1209/>
- AssistOS-AI. 2026. „NaturalLanguageLinterAgent (nllAgent): Architektura, dokumentacja i poważne problemy LongTextJS-CircuitJS”. Publiczne repozytorium i wygenerowana dokumentacja, sprawdzona 2 sierpień 2026. <https://github.com/assistos-ai/nllAgent>
- Badreddine, Samy, Artur d'Avila Garcez, Luciano Serafini i Michael Spranger. 2022. „Sieci tensorów logicznych”. Sztuczna inteligencja 303: 103649. <https://doi.org/10.1016/j.artint.2021.103649>
- Baldoni, Roberto, Emilio Coppa, Daniele Cono D'Elia, Camil Demetrescu i Irene Finocchi. 2018. „Przegląd symbolicznych technik egzekucji”. ACM Badania Obliczeniowe 51(3): 1–39. <https://doi.org/10.1145/3182657>
- Beg, Arshad, Diarmuid P. O'Donoghue i Rosemary Monahan. 2025. „Krótkie badanie na temat formułowania wymagań dotyczących oprogramowania przy użyciu dużych modeli językowych”. arXiv:2506.11874. <https://doi.org/10.48550/arXiv.2506.11874>
- Beiser, Alexander i David Penz. 2025. „Tworzenie LLMs Powód? Problem języka pośredniego w podejściu neurosymbolicznym”. arXiv:2502.17216. <https://arxiv.org/abs/2502.17216>
- Bos, Johan. 2008. „Analiza semantyczna o szerokim pokryciu z bokserem”. Proceedings of the 2008 Conference on Semantics in Text Processing. <https://aclanthology.org/W08-2222/>
- Chatzikyriakidis, Stergios i Shalom Lappin. 2026. „Neuro-symboliczny NLP: taksonomia, ocena i wskazówki”. Granice w sztucznej inteligencji 9. <https://doi.org/10.3389/frai.2026.1797587>
- Chen, Lingfeng, Tao Xiao, Masanari Kondo i Yasutaka Kamei. 2026. „Kierowane badanie symboliczne dla wykrywania luk w zabezpieczeniach: podejście z przewodnikiem LLM w KLEE”. arXiv:2607.21676. <https://arxiv.org/abs/2607.21676>
- Chen, Yongchao, Rujul Gandhi, Yang Zhang i Chuchu Fan. 2023. „NL2TL: Przekształcenie języków naturalnych w logikę czasową przy użyciu modeli dużego języka”. Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing. Stowarzyszenie na rzecz Lingwistyki Obliczeniowej. <https://aclanthology.org/2023.emnlp-main.985/>
- Clarke, Edmund, Orna Grumberg, Somesh Jha, Yuan Lu i Helmut Veith. 2000. „Dopracowanie Abstrakcji Na Przykładowo”. Weryfikacja wspomagana komputerowo, LNCS 1855, 154–169. https://doi.org/10.1007/10722167_15
- Cosler, Matthias, Christopher Hahn, Daniel Mendoza, Frederik Schmitt i Caroline Trippel. 2023. „nl2spec: Interaktywne tłumaczenie nieustrukturyzowanego języka naturalnego na logikę czasową z modelami wielojęzycznymi”. Weryfikacja wspomagana komputerowo, LNCS 13964, 383–396. https://doi.org/10.1007/978-3-031-37703-7_18
- Cousot, Patrick i Radhia Cousot. 1977. „Interpretacja abstrakcyjna: Ujednolicony model kratownicy do statycznej analizy programów według konstrukcji lub zbliżenia punktów mocowania”. Materiały POPL 1977, 238–252. <https://doi.org/10.1145/512950.512973>
- Delvecchio, Giovanni Pio, Lorenzo Molfetta i Gianluca Moro. 2025. „Neuro-symboliczna sztuczna inteligencja: przegląd skierowana do zadań w erze modeli Black-Box”. Materiały IJCAI 2025, Ścieżka badania, 10418–10426. <https://doi.org/10.24963/ijcai.2025/1157>
- Dong, Wenkai i Yifan Wang. 2026. „Od interpretacji do kompilacji: Kompilacja oparta na wykonywaniu operacji semantycznych”. arXiv:2607.13407. <https://doi.org/10.48550/arXiv.2607.13407>

- Ferrari, Alessio i Paola Spoletini. 2025. „Formalne wymagania Inżynieria i duże modele językowe: dwukierunkowa mapa drogowa”. *Technologia informacyjno-oprogramowania* 181: 107697. <https://doi.org/10.1016/j.infsof.2025.107697>
- Fuchs, Norbert E. i Rolf Schwitter. 1996. „Attempto Controlled English (ACE) - Kto odpowiedział?”. *Proceedings of the First International Workshop on Controlled Language Applications*. <https://attempto.ifi.uzh.ch/site/pubs/papers/ace96.pdf>
- Fuchs, Norbert E., Kaarel Kaljurand i Tobias Kuhn. 2008. „Próba kontrolowanego języka angielskiego dla reprezentacji wiedzy”. *Rozumowanie Web, LNCS 5224*, 104–124. https://doi.org/10.1007/978-3-540-85658-0_3
- Gao, Luyu, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan i Graham Neubig. 2022. „PAL: Modele Językowe wspomagane programowo”. *arXiv:2211.10435*. <https://arxiv.org/abs/2211.10435>
- Goldman, Omer, Veronica Latcinnik, Udi Naveh, Amir Globerson i Jonathan Berant. 2018. „Słabo nadzorowane parsowanie semantyczne z przykładami abstrakcyjnymi”. *Materiały ACL 2018. Stowarzyszenie na rzecz Lingwistyki Obliczeniowej*. <https://aclanthology.org/P18-1168/>
- Grimaldi, Pasquale. 2026. „pamaldi w SemEval-2026 Zadanie 11: Neuro-symboliczne rozumowanie sylogistyczne poprzez LLM-kierowaną strukturę ekstrakcji i deterministycznej walidacji”. *Proceedings of the 20th International Workshop on Semantic Evaluation. Stowarzyszenie na rzecz Lingwistyki Obliczeniowej*. <https://doi.org/10.18653/v1/2026.semeval-1.186>
- Gu, Qiuhuan, Avaljot Singh i Gagandeep Singh. 2026. „SAIL: Dźwiękowe abstrakcyjne interpretatory z LLMs”. *Materiały ACM w językach programowania 10 (PLDI): 1534–1559*. <https://doi.org/10.1145/3808308>
- Guu, Kelvin, Panupong Pasupat, Evan Liu i Percy Liang. 2017. „Od języka do programów: pomostowanie uczenia się wzmocnień i maksymalne prawdopodobieństwo marginalne”. *Materiały ACL 2017. Stowarzyszenie na rzecz Lingwistyki Obliczeniowej*. <https://aclanthology.org/P17-1016/>
- Hamilton, Kyle, Aparna Nayak, Bojan Božić i Luca Longo. 2024. „Czy neurosymboliczne AI spełniają swoje obietnice w zakresie przetwarzania języka naturalnego? Przegląd strukturalny”. *Semantyczna Sieć 15(4): 1265–1306*. <https://doi.org/10.3233/SW-223228>
- Han, Simeng, Hailey Schoelkopf, Yilun Zhao, Zhenting Qi, Martin Riddell, Wenfei Zhou, James Coady, David Peng, Yujie Qiao, Luke Benson, Lucy Sun, Alex Wardle-Solano, Hannah Szabó, Ekaterina Zubova, Matthew Burtell, Jonathan Fan, Yixin Liu, Brian Wong, Malcolm Sailor, Ansong Ni, Linyong Nan, Jungo Kasai, Tao Yu, Rui Zhang, Alexander Fabbri, Wojciech Kryściński, Semih Yavuz, Ye Liu, Xi Victoria Lin, Shafiq Joty, Yingbo Zhou, Caiming Xiong, Rex Ying i Dragomir Radev. 2024. „FOLIO: Rozumowanie języka naturalnego z logiką na pierwszym rzędzie”. *Materiały EMNLP 2024. Stowarzyszenie na rzecz Lingwistyki Obliczeniowej*. <https://aclanthology.org/2024.emnlp-main.1229/>
- Hao, Yilun, Yongchao Chen, Yang Zhang i Chuchu Fan. 2025. „Duże modele językowe mogą szybko rozwiązywać rzeczywiste planowanie za pomocą formalnych narzędzi weryfikacji”. *Materiały NAACL 2025*, 3434–3483. <https://aclanthology.org/2025.naacl-long.176/>
- Hu, Ruikang, Shaoyu Lin, Yeliang Xiu i Yongmei Liu. 2025. „LTRAG: Wzmocnienie autoformalizacji i samodopracowania dla logicznego rozumowania z myślami z przewodnikiem RAG”. *Wyniki ACL 2025*, 2483–2493. <https://aclanthology.org/2025.findings-acl.126/>
- Huang, Pei, Jiayu Li, Yuhao Zhang, Changliu Liu i Shih-hsi Alex Liu. 2026. „Parametryzowana abstrakcyjna interpretacja weryfikacji transformatora”. *Materiały AAAI 2026*. <https://doi.org/10.1609/aaai.v40i42.40860>
- Jia, Robin, Aditi Raghunathan, Kerem Göksel i Percy Liang. 2019. „Certyfikowana solidność kontradykcyjnych podstawek słów”. *Materiały EMNLP-IJCNLP 2019. Stowarzyszenie na rzecz Lingwistyki Obliczeniowej*. <https://aclanthology.org/D19-1423/>
- Karthikeyan, T, Om Dehlan, Mausam i Manish Gupta. 2025. „LRPLAN: Wieloagentowa współpraca dużych modeli językowych i rozumowania do planowania z domyślnymi i wyraźnymi ograniczeniami”. *Wyniki EMNLP 2025*, 8280–8310. <https://doi.org/10.18653/v1/2025.findings-emnlp.440>
- Kartáč, Ivan, Kristýna Onderková, Jan Bronec, Zdeněk Kasner, Mateusz Lango i Ondřej Dušek. 2026. „UFAL-CUNI w SemEval-2026 Zadanie 11: Wydajna modułowa metoda neuro-symboliczna do nauczania sylogistycznego”. *Proceedings of the 20th International Workshop on Semantic Evaluation. Stowarzyszenie na rzecz Lingwistyki Obliczeniowej*. <https://aclanthology.org/2026.semeval-1.418/>
- King, James C. 1976. „Symboliczne wykonanie i testowanie programu”. *Komunikacja ACM 19(7): 385–394*. <https://doi.org/10.1145/360248.360252>

- Kirtania, Shashank, Priyanshu Gupta i Arjun Radhakrishna. 2024. „LOGIC-LM++: Wielostopniowa dopracowanie dla formuł symbolicznych”. *Proceedings of the 2nd Workshop on Natural Language Reasoning and Structured Explanations*, 56–63. <https://aclanthology.org/2024.nlrse-1.6/>
- Kuhn, Tobias. 2014. „Badanie i klasyfikacja kontrolowanych języków naturalnych”. *Językoznawstwo Obliczeniowe* 40(1): 121–170. https://doi.org/10.1162/COLI_a_00168
- Lalwani, Vivek, et al. 2025. „Autoformalizacja języka naturalnego do logiki pierwszego rzędu: studium przypadku w logicznym wykrywaniu błędów”. *Wyniki IJCNLP-AACL 2025*, 132–147. <https://aclanthology.org/2025.findings-ijcnlp.8/>
- Lee, Kang-il, Segwang Kim i Kyomin Jung. 2023. „Słabo nadzorowane parsowanie semantyczne z fałszywym filtrowaniem programów opartych na egzekucji”. *Materiały EMNLP 2023*, 6884–6894. <https://aclanthology.org/2023.emnlp-main.425/>
- Li, Yihe, Ruijie Meng i Gregory J. Duck. 2025. „Duży model językowy zasilany symboliczną egzekucją”. *Materiały ACM w zakresie języków programowania 9 (OOPSLA2)*, Article 385, 3148–3176. <https://doi.org/10.1145/3763163>
- Liang, Chen, Jonathan Berant, Quoc Le, Kenneth D. Forbus i Ni Lao. 2017. „Neural Symbolic Machines: Uczenie się parserów semantycznych na Freebase ze słabym nadzorem”. *Materiały ACL 2017. Stowarzyszenie na rzecz Lingwistyki Obliczeniowej*. <https://aclanthology.org/P17-1003/>
- Liu, Jiangming, Shay B. Cohen i Mirella Lapata. 2018. „Struktura reprezentacji dyskursu Parsing”. *Materiały ACL 2018. Stowarzyszenie na rzecz Lingwistyki Obliczeniowej*. <https://aclanthology.org/P18-1040/>
- Locascio, Nicholas, Karthik Narasimhan, Eduardo DeLeon, Nate Kushman i Regina Barzilay. 2018. „Neuralne generowanie wyrażeń regularnych z języka naturalnego z minimalną wiedzą na temat domeny”. *Materiały EMNLP 2018. Stowarzyszenie na rzecz Lingwistyki Obliczeniowej*. <https://aclanthology.org/D18-1436/>
- MacCartney, Bill i Christopher D. Manning. 2007. „Naturalna logika do wnioskowania tekstowego”. *Proceedings of the ACL-PASCAL Workshop on Textual Entailment and Paraphrasing*. <https://aclanthology.org/W07-1401/>
- MacCartney, Bill i Christopher D. Manning. 2009. „Rozszerzony model logiki naturalnej”. *Materiały IWCS 2009*. <https://aclanthology.org/W09-3714/>
- Manhaeve, Robin, Sebastijan Dumančić, Angelika Kimmig, Thomas Demeester i Luc De Raedt. 2018. „DeepProbLog: Neuronowe programowanie logiki probabilistycznej”. *Postępy w systemach przetwarzania informacji neuronowej* 31. <https://proceedings.neurips.cc/paper/2018/hash/dc5d637ed5e62c36ecb73b654b05ba2a-Abstract.html>
- Mao, Ziyu, Jingyi Wang, Jun Sun, Shengchao Qin i Jiawen Xiong. 2025. „LLM-Aided Automatyczne modelowanie dla weryfikacji protokołu bezpieczeństwa”. *Proceedings of 47. IEEE/ACM Międzynarodowa Konferencja Inżynierii Oprogramowania, 2025*. https://ink.library.smu.edu.sg/sis_research/10297/
- Mitchell, Jacqueline L., Brian Hyeonseok Kim, Chenyu Zhou i Chao Wang. 2025. „Czy LLMs może formalnie uzasadnić jako abstrakcyjne interpretatory do analizy programu?”. *arXiv:2503.12686*. <https://arxiv.org/abs/2503.12686>
- Mou, Lili, Zhengdong Lu, Hang Li i Zhi Jin. 2017. „Sprzężenie rozproszone i symboliczne wykonywanie zapytań dotyczących języka naturalnego”. *Materiały ICML 2017*. <https://proceedings.mlr.press/v70/mou17a.html>
- Olausson, Theo X., Alex Gu, Benjamin Lipkin, Cedegao E. Zhang, Armando Solar-Lezama, Joshua B. Tenenbaum i Roger Levy. 2023. „LINC: Neurosymboliczne podejście do logicznego rozumowania poprzez łączenie modeli językowych z urządzeniami logicznymi na pierwszym rzędzie”. *Materiały EMNLP 2023. Stowarzyszenie na rzecz Lingwistyki Obliczeniowej*. <https://aclanthology.org/2023.emnlp-main.313/>
- Pan, Liangming, Alon Albalak, Xinyi Wang i William Yang Wang. 2023. „Logic-LM: Wzmocnienie pozycji w dużych modelach językowych za pomocą symbolicznych rozwiązujących dla wiernego logicznego rozumowania”. *Wyniki EMNLP 2023. Stowarzyszenie na rzecz Lingwistyki Obliczeniowej*. <https://aclanthology.org/2023.findings-emnlp.248/>
- Pasupat, Panupong i Percy Liang. 2016. „Wnioskowanie form logicznych z denotacji”. *Materiały ACL 2016. Stowarzyszenie na rzecz Lingwistyki Obliczeniowej*. <https://aclanthology.org/P16-1004/>
- Pei, Yu, Yongping Du i Xingnan Jin. 2025. „FoVer: Weryfikacja logiczna pierwszego rzędu dla rozumowania języka naturalnego”. *Transakcje Stowarzyszenia Językoznawstwa Obliczeniowego* 13: 1340–1359. <https://aclanthology.org/2025.tacl-1.61/>

- Tafjord, Oyvind, Bhavana Dalvi Mishra i Peter Clark. 2021. „ProofWriter: Generowanie implikacji, dowodów i oświadczeń uprowadzenia w języku naturalnym”. Wyniki ACL-IJCNLP 2021. Stowarzyszenie na rzecz Lingwistyki Obliczeniowej. <https://aclanthology.org/2021.findings-acl.317/>
- Tantakoun, Nat, Christian Muise i Yilun Zhu. 2025. „Modele dużego języka jako formalizatory planowania: przegląd”. Wyniki ACL 2025. <https://aclanthology.org/2025.findings-acl.1291/>
- Tziafas, Georgios i Hamidreza Kasaei. 2026. „Neuro-symboliczne planowanie zadań i manipulacja obiektami z modelami wielkojęzycznymi”. Autonomiczne Roboty 50. <https://doi.org/10.1007/s10514-026-10230-6>
- Wang, Bailin, Richard Shin, Xiaodong Liu, Oleksandr Polozov i Matthew Richardson. 2018. „Dekodowanie programu neuronowego prowadzonego przez egzekucję”. Proceedings of the First Workshop on Learned Program Synthesis. <https://arxiv.org/abs/1807.03100>
- Wang, Boxin, Chejian Xu, Xiangyu Liu, Yu Cheng i Bo Li. 2023. „Solidność-Świadoma osadzanie słów poprawia certyfikowaną solidność do kontradiktoryjnych podstawień słów”. Wyniki ACL 2023. Stowarzyszenie na rzecz Lingwistyki Obliczeniowej. <https://aclanthology.org/2023.findings-acl.620/>
- Wang, Jiayimei, Tao Ni, Wei-Bin Lee i Qingchuan Zhao. 2025. „Współczesna przegląd nad analizą programu wspomaganego modelem dużego języka”. Trendy w sztucznej inteligencji. <https://arxiv.org/abs/2502.18474>
- Wang, Wenguan, Yi Yang i Fei Wu. 2025. „W kierunku danych i wiedzy AI: przegląd na temat obliczeń neuro-symbolicznych”. IEEE Transakcje na analizie wzorców i inteligencji maszyn 47(2): 878–899. <https://doi.org/10.1109/TPAMI.2024.3483273>
- Wang, Yu, You Zhang, Hao Zhang i Dan Xu. 2026. „YNU-NLP w SemEval-2026 Zadanie 11: Neuro-symboliczne podejście z mechanizmem refleksyjnym Rozplątywanie treści i formalne rozumowanie w modelach językowych”. Proceedings of the 20th International Workshop on Semantic Evaluation. Stowarzyszenie na rzecz Lingwistyki Obliczeniowej. <https://doi.org/10.18653/v1/2026.semeval-1.126>
- Weng, Ke, Lun Du, Sirui Li, Wangyue Lu, Haozhe Sun, Hengyu Liu i Tiancheng Zhang. 2025. „Autoformalizacja w erze modeli dużych języków: przegląd”. arXiv:2505.23486. <https://arxiv.org/abs/2505.23486>
- Xia, Shihao, Mengting He, Shuai Shao, Tingting Yu, Yiyang Zhang, Nobuko Yoshida i Linhai Song. 2026. „SymGPT: Audyt inteligentnych kontraktów poprzez łączenie symbolicznych wykonań z modelami wielkojęzycznymi”. Materiały ACM w językach programowania 10 (OOPSLA1), Article 109. <https://doi.org/10.1145/3798217>
- Yang, Zonglin, et al. 2024. „Autoformalizacja problemów rozumowania języka naturalnego do logiki pierwszego rzędu”. Materiały EMNLP 2024. Stowarzyszenie na rzecz Lingwistyki Obliczeniowej. <https://aclanthology.org/2024.emnlp-main.1132/>
- Ye, Mao, Chengyue Gong i Qiang Liu. 2020. „SAFER: Bezstrukturalne podejście do certyfikowanej części do kontradiktoryjnych podstawień słów”. Materiały ACL 2020. Stowarzyszenie na rzecz Lingwistyki Obliczeniowej. <https://aclanthology.org/2020.acl-main.317/>
- Ye, Xi, Qiaochu Chen, Isil Dillig i Greg Durrett. 2023. „SatLM: modele językowe wspomagane spełnialnością z wykorzystaniem promptingu deklaratywnego”. Postępy w systemach przetwarzania informacji neuronowej 36. <https://arxiv.org/abs/2305.09656>
- Zheng, Zihan, Tianle Cui, Chuwen Xie, Jiahui Pan, Qianglong Chen i Lewei He. 2025. „PlanningArena: Modułowy wskaźnik oceny wielowymiarowej planowania i uczenia narzędziowego”. Materiały ACL 2025, 31047–31086. <https://doi.org/10.18653/v1/2025.acl-long.1499>